



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

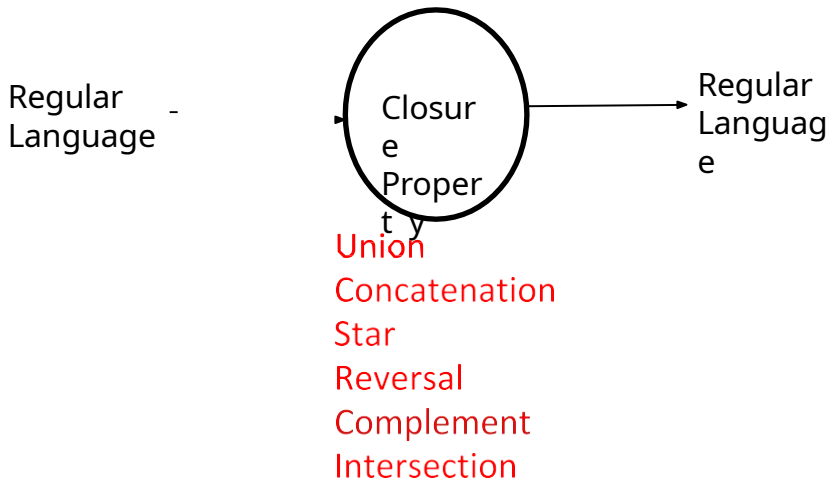
Automata Formal Languages & Logic

Unit 2

Preet Kanwal

Department of Computer Science Engineering

Unit 2 - Properties of Regular Languages



Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages



For regular languages L_1 and L_2 we will prove that:

Union : $L_1 \cup L_2$

Concatenation: $L_1 L_2$

Star: L_1^*

Reversal: L_1^R

Complement: $\overline{L_1}$

Intersection: $L_1 \cap L_2$

Are regular languages

Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

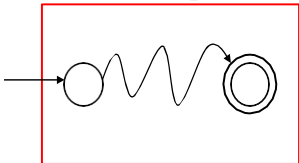
Take two languages

Regular language

L_1

$$L(M_1) = L_1$$

NFA M_1



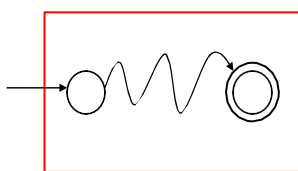
Single accepting
state

Regular language

L_2

$$L(M_2) = L_2$$

NFA M_2



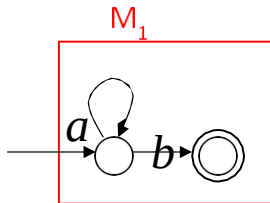
Single accepting
state

Automata Formal Languages and Logic

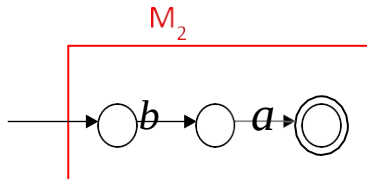
Unit 2 - Properties of Regular Languages

Example

$$L_1 = \{a^n b\}$$



$$L_2 = \{ba\}$$

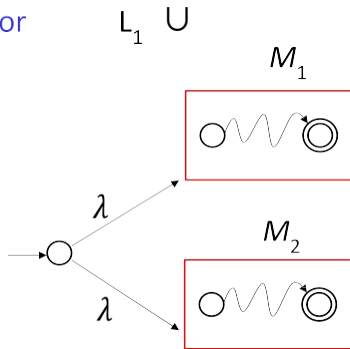


Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

Union

NFA for
 L_2

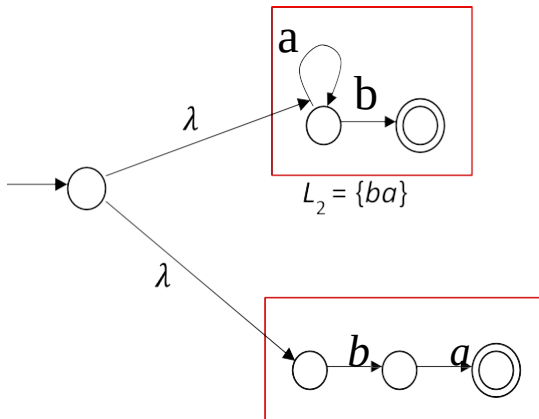


$$w \in L_1 \cup L_2 \iff w \in L_1 \text{ or } w \in L_2$$

Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

NFA for $L_1 \cup L_2 = \{a^n b\} \cup \{ba\}$
 $L_1 = \{a^n b\}$

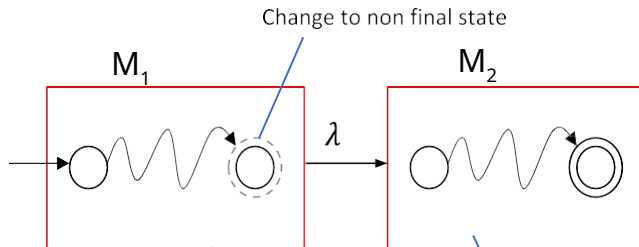


Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

Concatenation

NFA for $L_1 L_2$



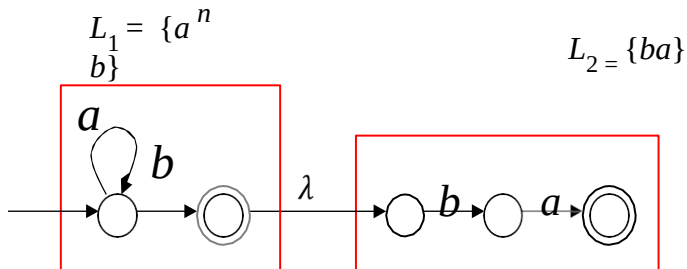
$$w \in L_1 L_2 \iff w = w_1 w_2 : w_1 \in L_1 \text{ and } w_2 \in L_2$$

Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

Example

NFA for $L_1 L_2 = \{a^n b\} \{ba\} = \{a^n bba\}$

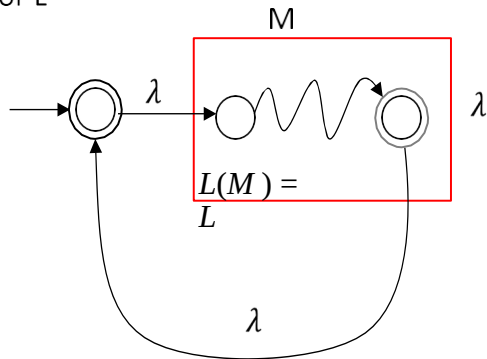


Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

Star Closure

NFA for L^*



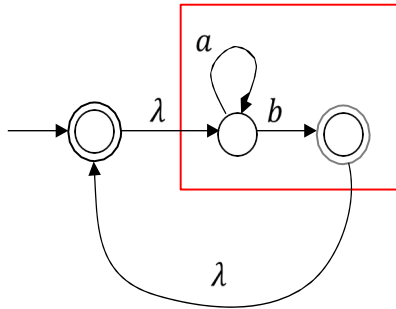
Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

Example

NFA for $L^* = \{a^n b\}^*$

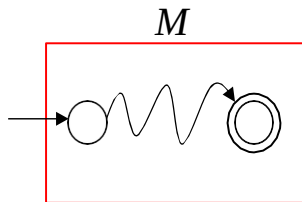
$$L_1 = \{a^n b\}$$



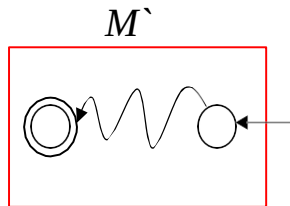
Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

Reverse



$$L(M) = L$$

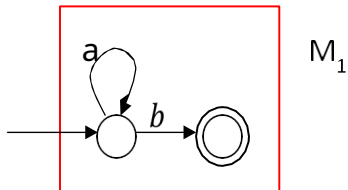


$$L(M^r) = L^R$$

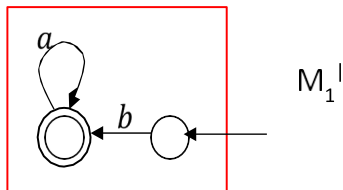
Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

$$L_1 = \{a^n b\}$$

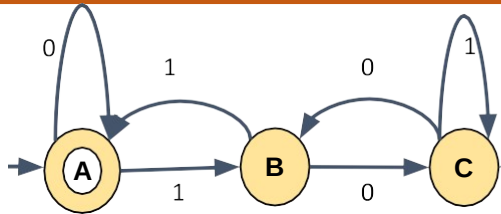


$$L^R = \{ba^n\}$$



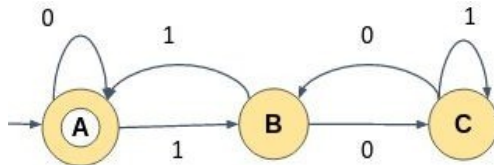
Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages



M

$L =$ Binary strings divisible by 3.



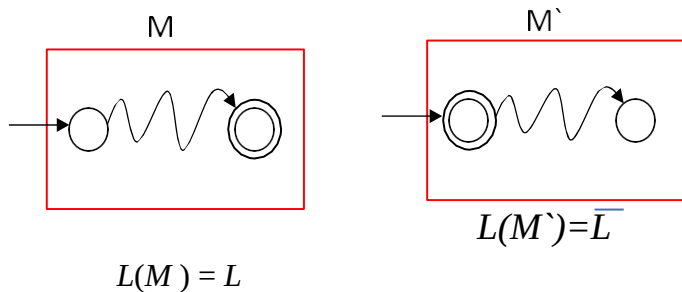
Both M and M_R is same because it is palindrome

M^R

Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

Complement



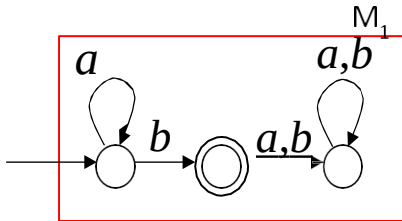
1. Take the DFA that accepts L
2. Make accepting state has non accepting and vice versa

Automata Formal Languages and Logic

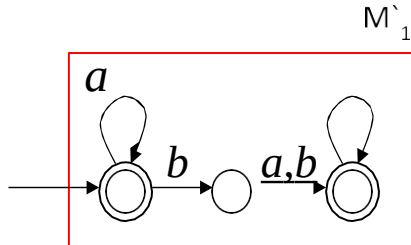
Unit 2 - Properties of Regular Languages

Example

$$L_1 = \{a^n b\}$$



$$\overline{L_1} = \{a,b\}^* - \{a^n b\}$$



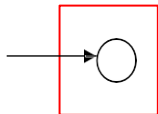
Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

NFAs cannot be used for complement

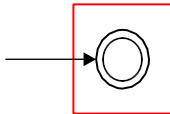
Make accepting state has non accepting and vice versa

NFA M



$$L(M) = \{ \}$$

NFA M'



$$L(M') = \{ \lambda \} \neq \overline{L(M)}$$

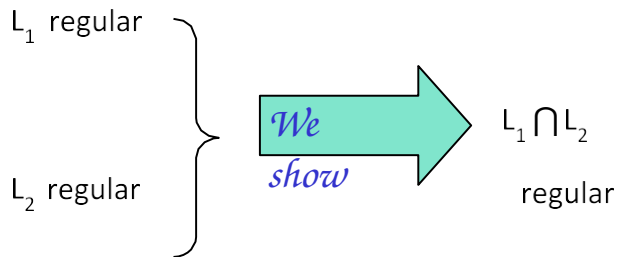
$$\overline{L(M)} = \Sigma^* = \{a,b\}^*$$

It is not the complement

Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

Intersection



DeMorgan's Law: $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$

L_1, L_2 regular, regular

→ $\overline{L_1}, \overline{L_2}$ regular, regular

→ $\overline{L_1} \cup \overline{L_2}$ regular

→ $\overline{\overline{L_1} \cup \overline{L_2}}$ regular

→ $L_1 \cap L_2$ regular

Example

$$\left. \begin{array}{l} L_1 = \{a^n b\} \text{ regular} \\ L_2 = \{ab, ba\} \text{ regular} \end{array} \right\} \Rightarrow L_1 \cap L_2 = \{ab\} \text{ regular}$$

Another Proof for Intersection

Machine M_1

DFA for L_1

Machine M_2

DFA for L_2

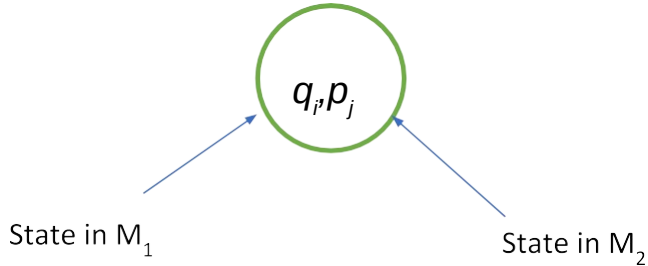
Construct a new DFA M that accepts $L_1 \cap L_2$

M simulates in parallel M_1 and M_2

Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

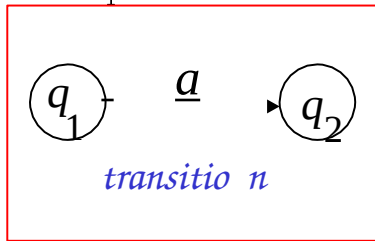
States in M



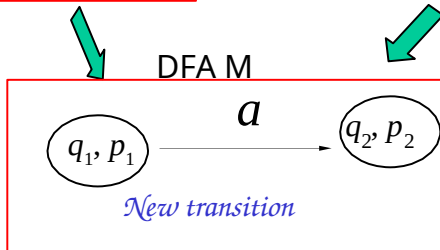
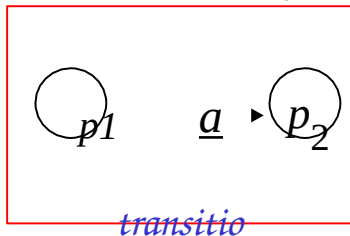
Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

DFA M_1

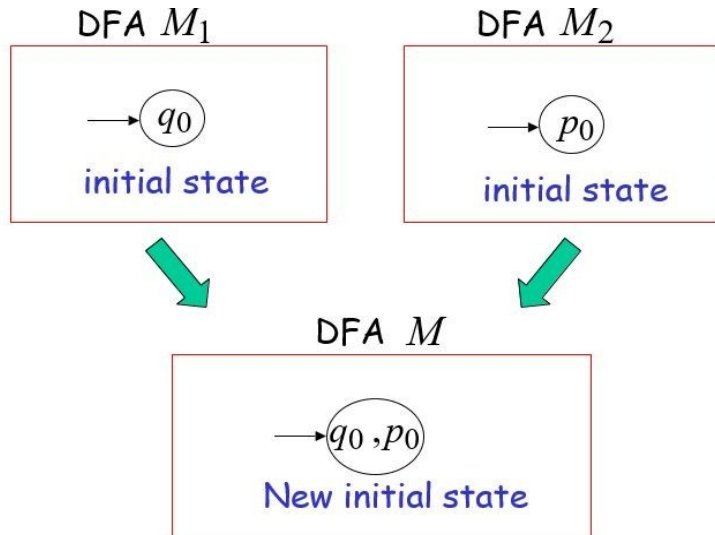


DFA M_2



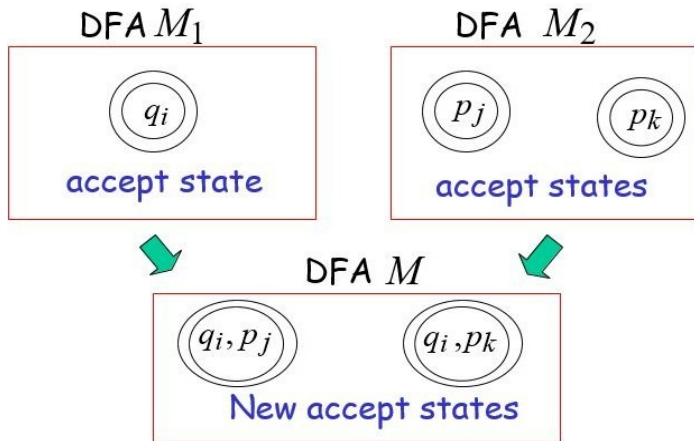
Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages



Automata Formal Languages and Logic

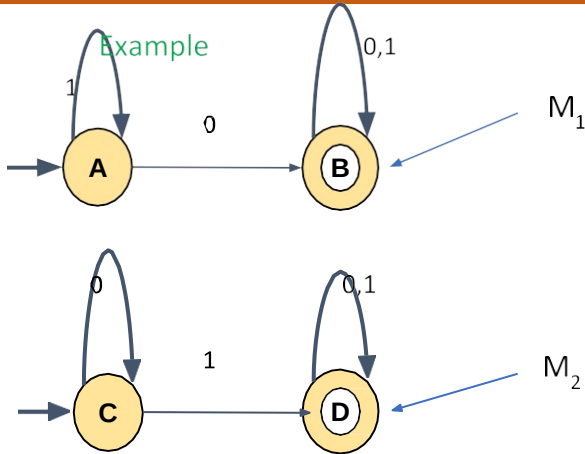
Unit 2 - Properties of Regular Languages



Both constituents must be accepting states

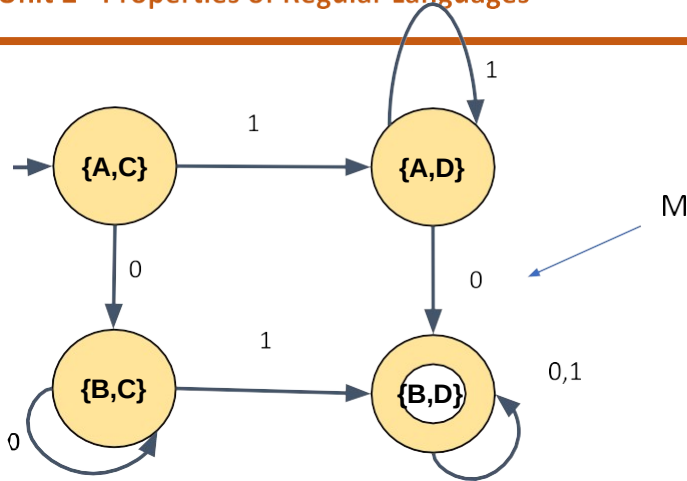
Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages



Automata Formal Languages and Logic

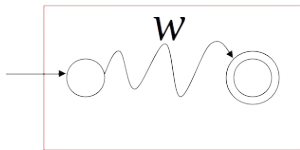
Unit 2 - Properties of Regular Languages



Decidable properties of regular language

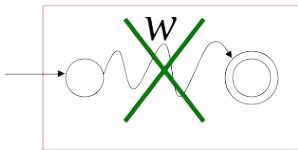
1. Membership Question

DFA



$w \in L$

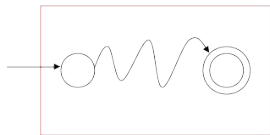
DFA



$w \notin L$

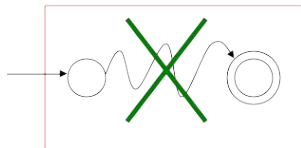
2. Testing Emptiness

DFA



$$L \neq \emptyset$$

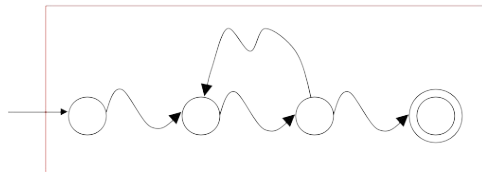
DFA



$$L = \emptyset$$

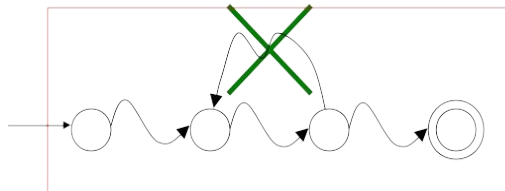
3. Is a given regular language finite or Infinite?

DFA



L is infinite

DFA



L is finite

4. Given a regular language L_1 and L_2 how can we check if $L_1 = L_2$?

Regular Language L_1

Regular Language L_2

Regular Language L_1



Regular expression R_1

Regular Language L_2



Regular expression R_2

Regular Language L_1



Regular expression R_1



Equivalent
NFA

Regular Language L_2



Regular expression R_2



Equivalent
NFA

Regular Language L_1



Regular expression R_1



Equivalent
NFA



Construct DFA

Regular Language L_2



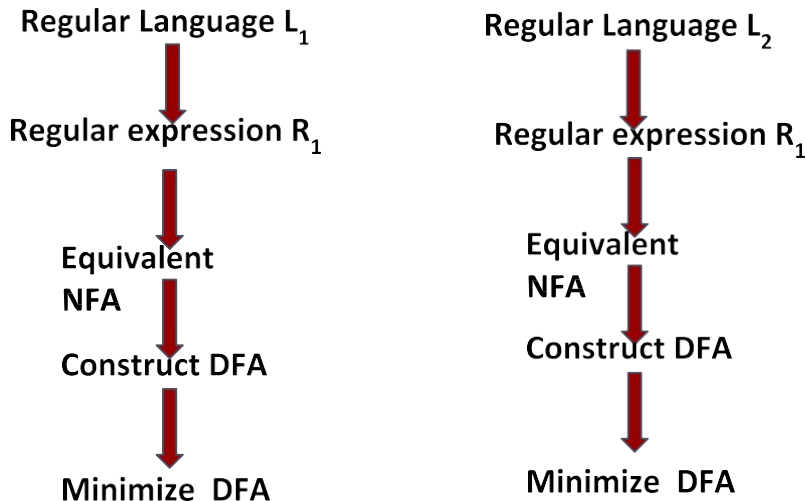
Regular expression R_2

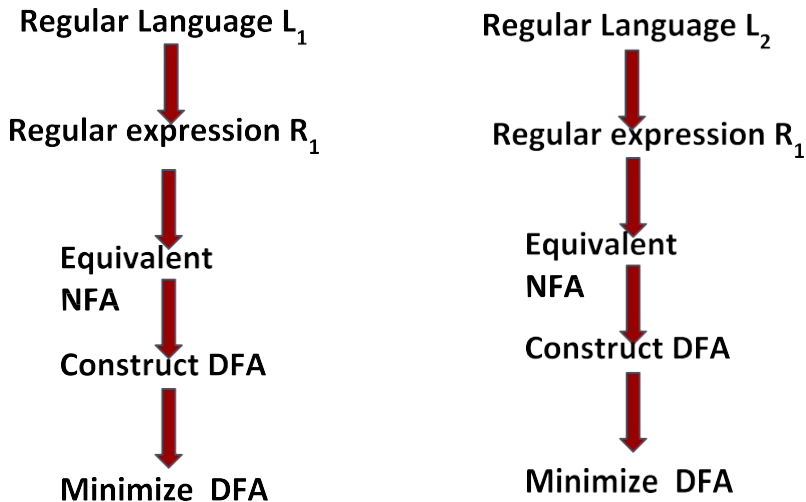


Equivalent
NFA



Construct DFA





RE (A) = RE (B) iff the minimized DFA of both the expression are same as the minimized DFA is unique

Example 1:

$$L_1(R_1) = R_1 = (0+1)^*(0+\lambda) \qquad L_2(R_2) = R_2 = (1+\lambda)(1+0)^*$$

Example 1:

$(0+1)^*(0+\lambda)$



R_1

$(1+\lambda)(1+0)^*$



R_2

Example 1:

$(0+1)^*(0+\lambda)$



R_1

$(1+\lambda)(1+0)^*$



R_2

Convert the regular expression R_1 and R_2 to its equivalent NFA

Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages



Example 1:

Formal
Method

$(0+1)^*(0+\lambda)$

$(1+\lambda)$
 $(1+0)^*$

Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

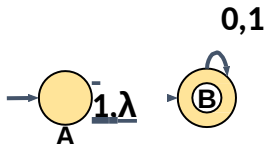
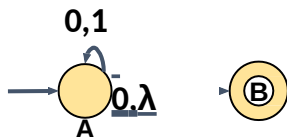
Example 1:

Formal Method

$(0+1)^*(0+\lambda)$

$(1+\lambda)$
 $(1+0)^*$

NFA



Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

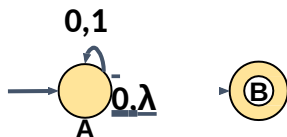
Example 1:

Formal Method

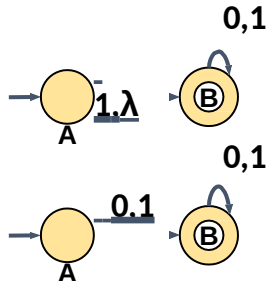
$(0+1)^*(0+\lambda)$

$(1+\lambda)$
 $(1+0)^*$

NFA



DFA



Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

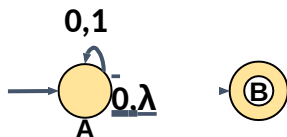
Example 1:

Formal Method

$(0+1)^*(0+\lambda)$

$(1+\lambda)$
 $(1+0)^*$

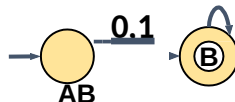
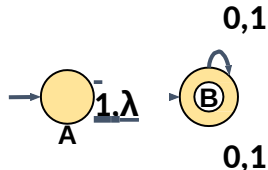
NFA



DFA



Minima
I DFA



Automata Formal Languages and Logic

Unit 2 - Properties of Regular Languages

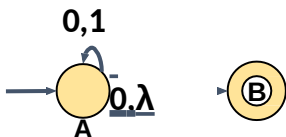
Example 1:

Formal Method

$(0+1)^*(0+\lambda)$

$(1+\lambda)$
 $(1+0)^*$

NFA



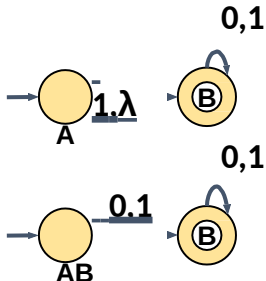
DFA



Minima
I DFA



Equal



Multiple Choice Questions

1. Which of the following is always true for a **regular language**?
 - a) It can represent only finite languages
 - b) It is closed under intersection
 - c) It is closed under subtraction
 - d) It is not closed under union

2. If $L = \{a, b\}^* - \{ab\}$, which of the following strings belongs to the language L ?
 - a) ab
 - b) ba
 - c) aa
 - d) aba

Multiple Choice Questions

3. For the language $L = (\text{binary strings divisible by } 3)$, what can we infer about its regularity?
- a) L is not regular
 - b) L is regular
 - c) L is a context-free language only
 - d) L must be infinite
4. Consider the language where both M and M_R are the same because the string is a palindrome. What does this imply about the string?
- a) The string is always non-regular
 - b) The string reads the same forwards and backwards
 - c) The string starts with the same symbol
 - d) The string ends with 'b'

Multiple Choice Questions

5. Let L be the set of all binary strings divisible by 3. Which closure property of regular languages guarantees that the intersection of L with another regular language R is also regular?
- a) Closure under union
 - b) Closure under complementation
 - c) Closure under intersection
 - d) Closure under reversal
6. Given $L = \{a, b\}^* - \{ab\}$, which of the following statements is correct?
- a) Any string containing 'ab' as a substring is included in L
 - b) Only strings that do not contain 'ab' as a substring are in L
 - c) Strings consisting solely of 'ab' are exclusively in L
 - d) L is not a regular language

Answers

1. B
2. C
3. B
4. B
5. C
6. B



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

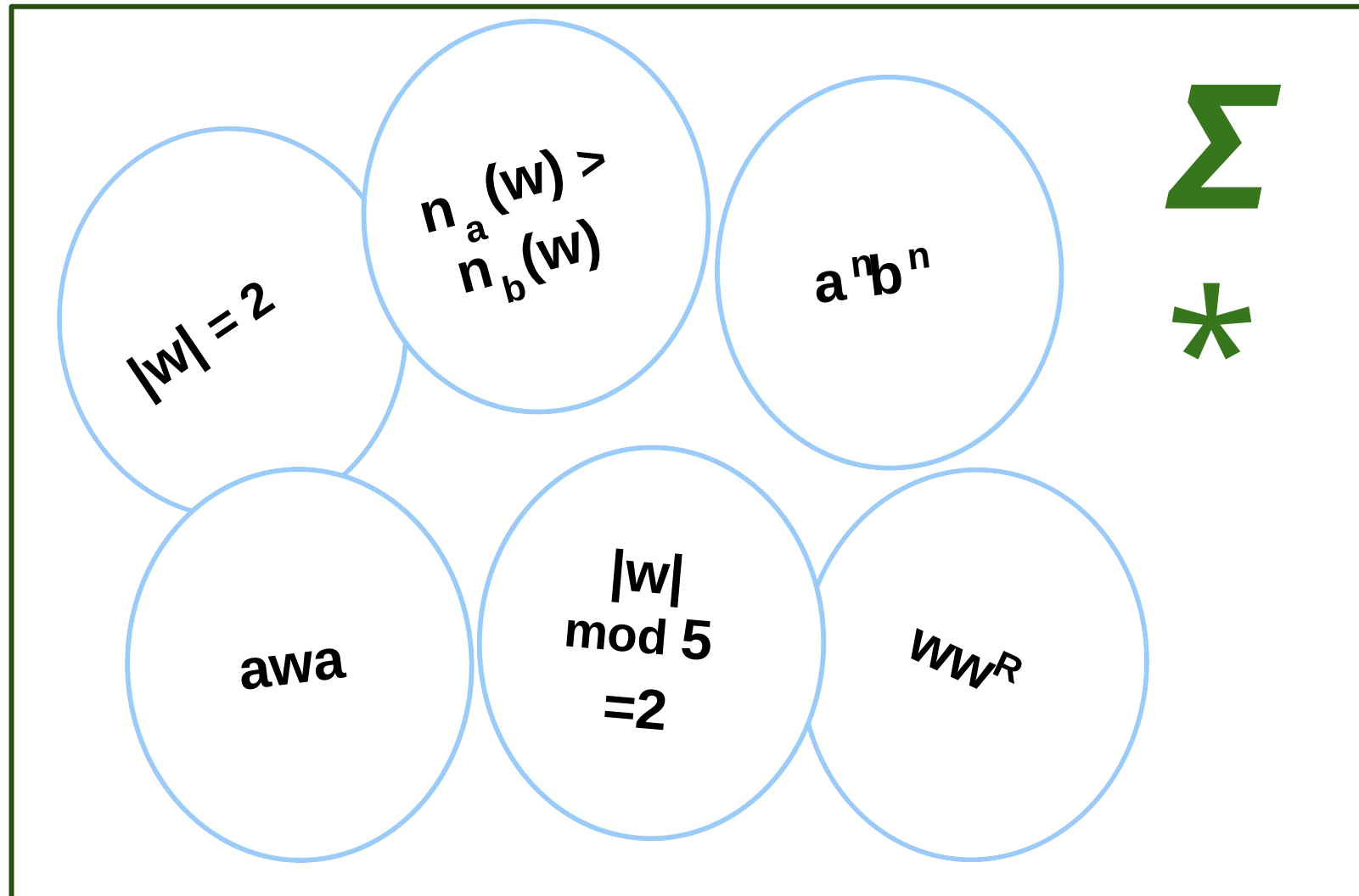
Unit 2

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages and Logic

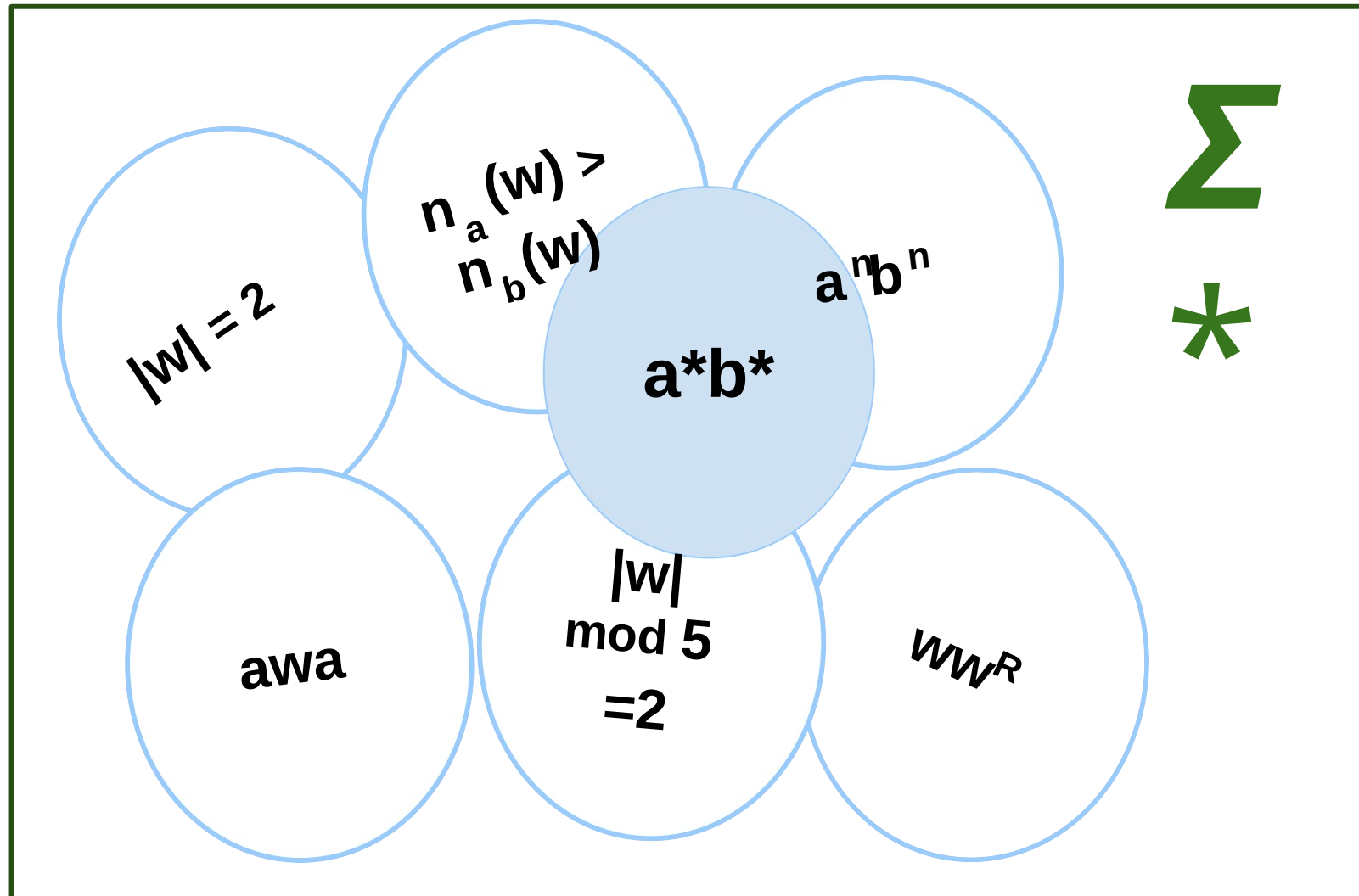
Unit 2 - Pumping Lemma for Regular Languages



$\Sigma = \{a, b\}$

Automata Formal Languages and Logic

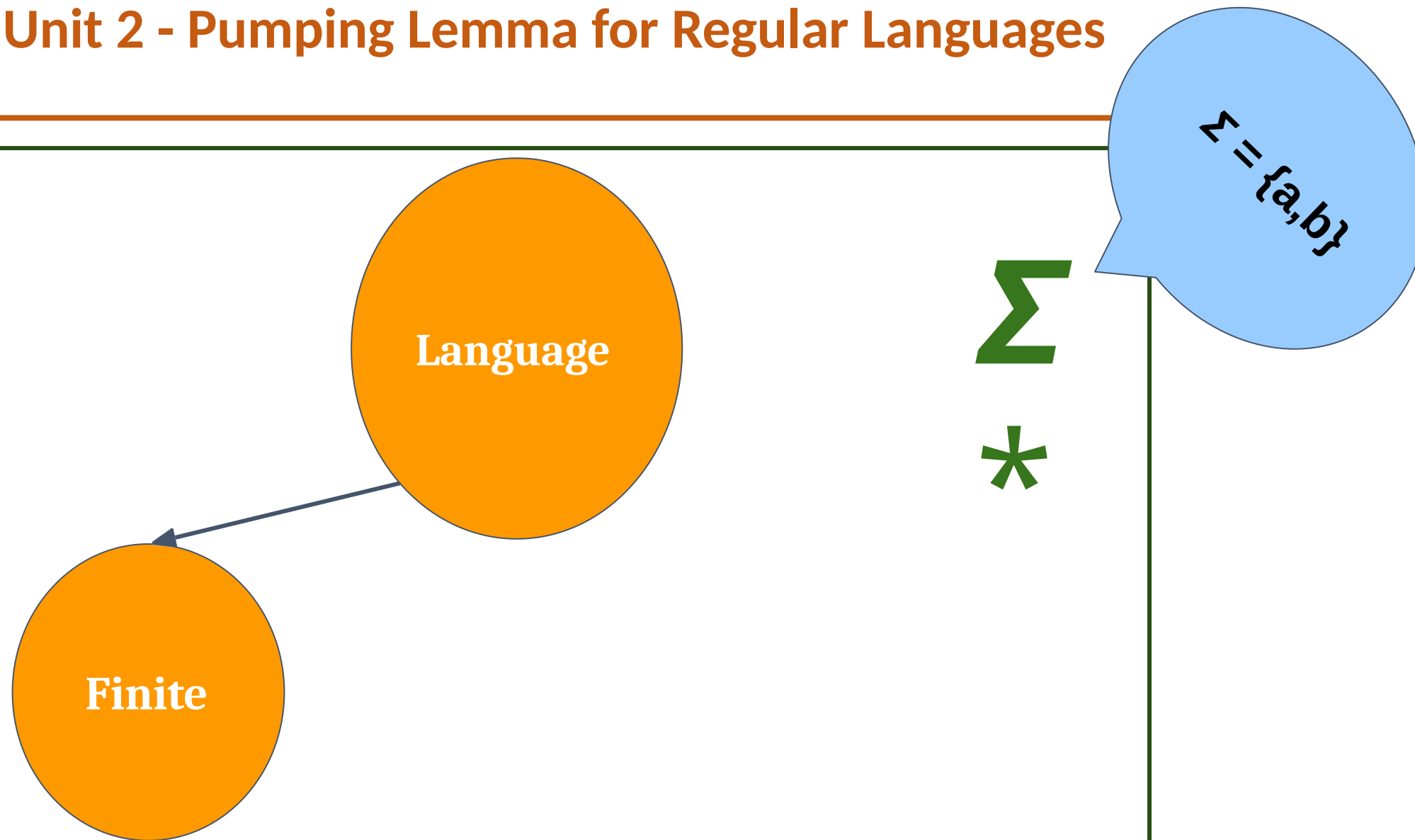
Unit 2 - Pumping Lemma for Regular Languages



$\Sigma = \{a, b\}$

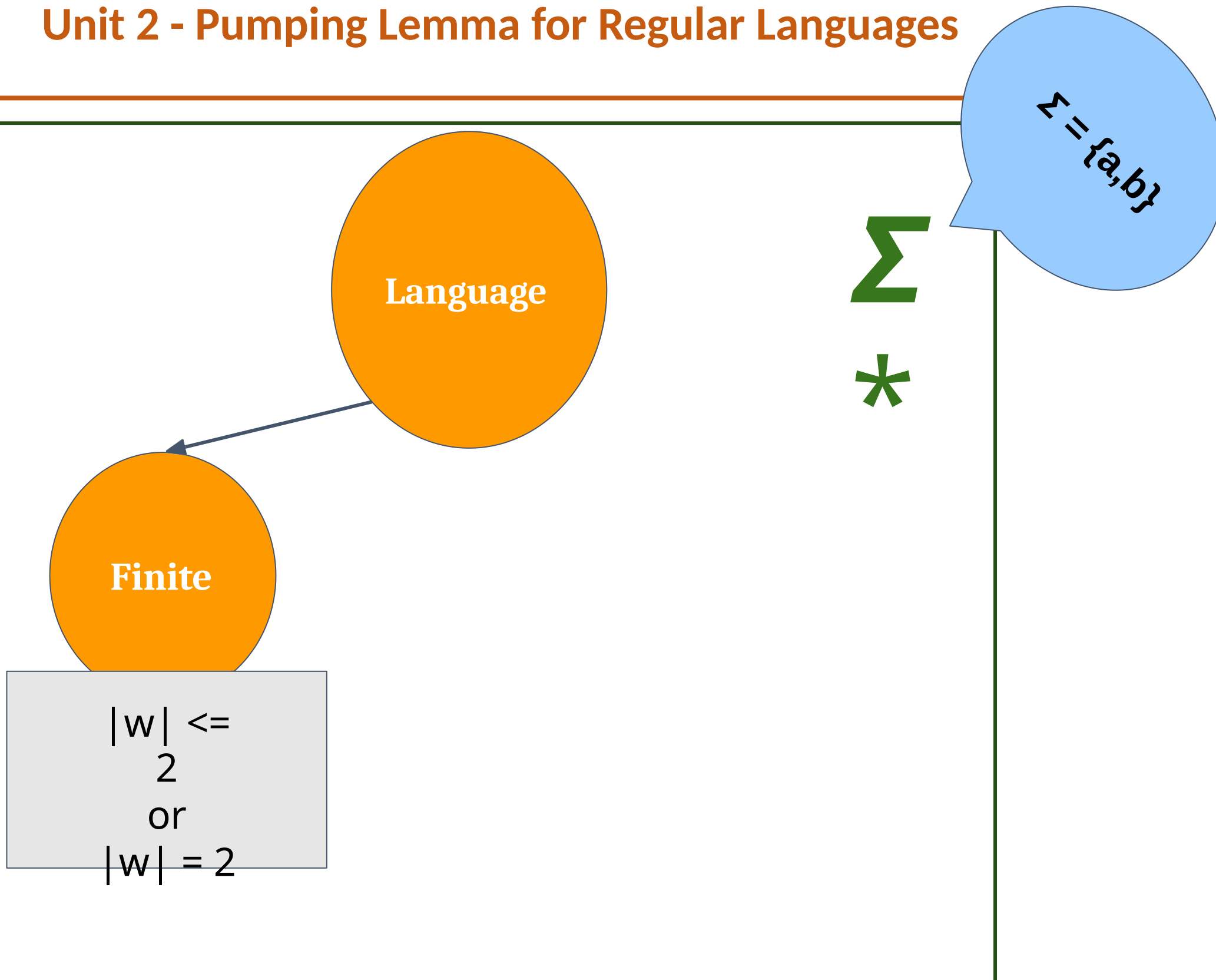
Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages



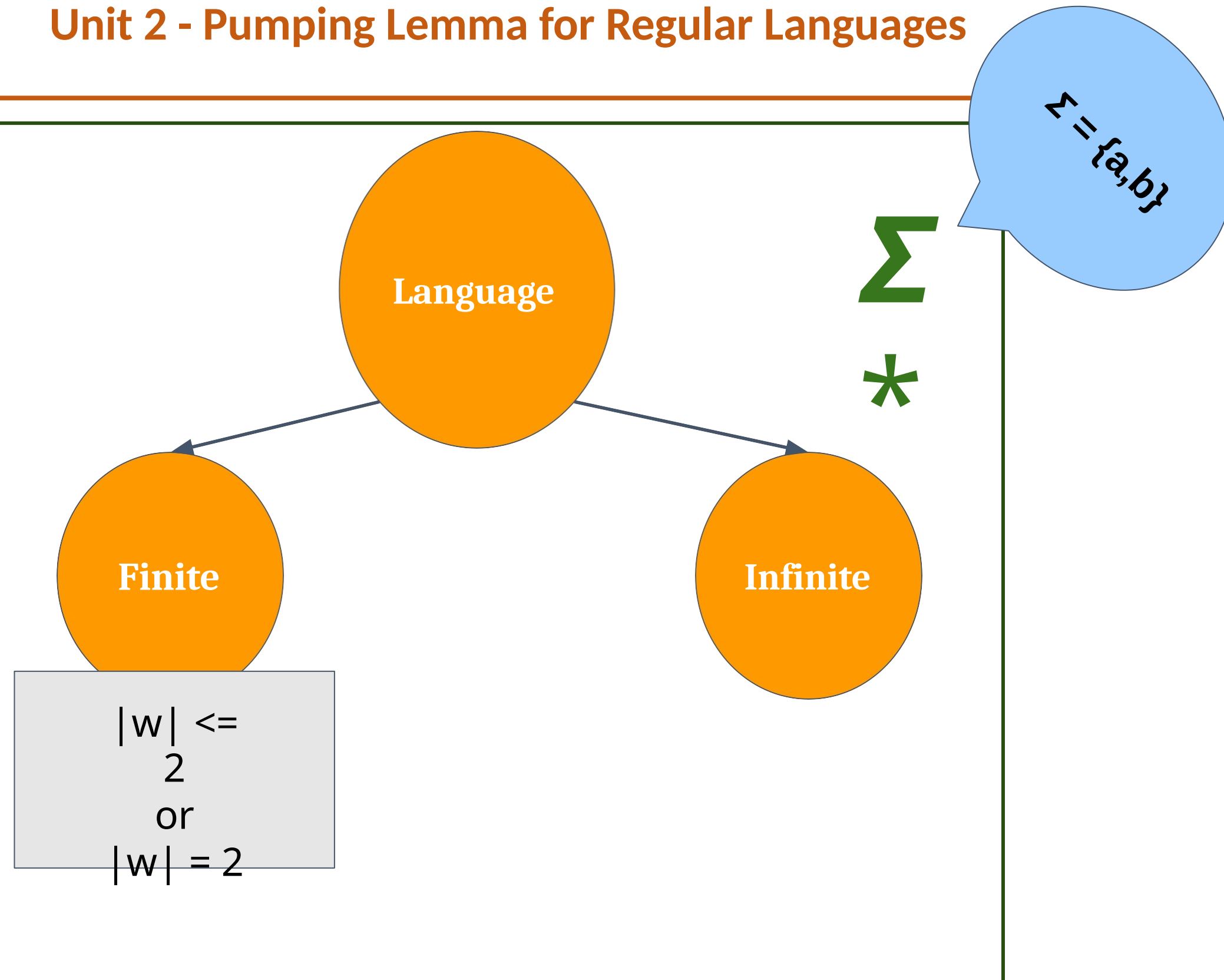
Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages



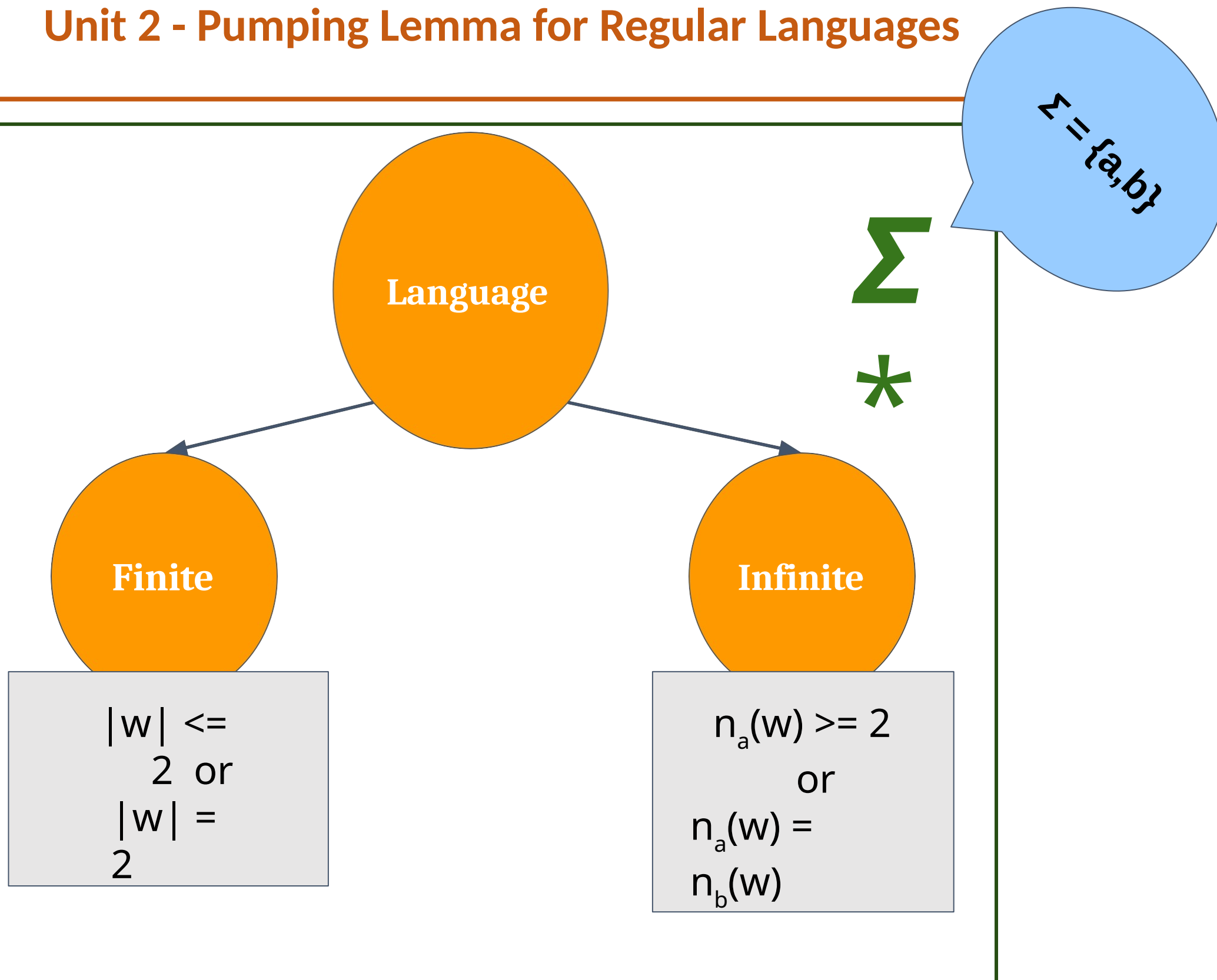
Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages



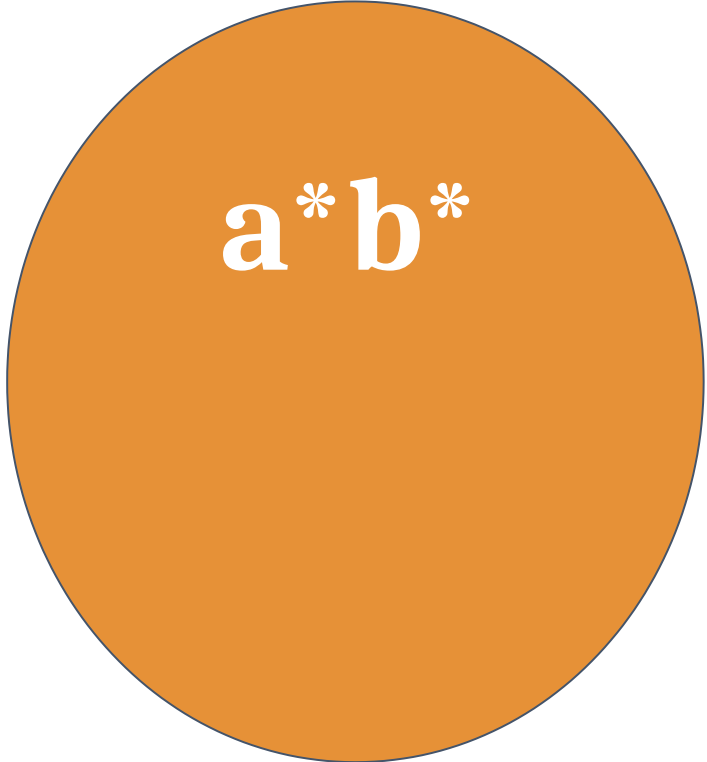
Is there any infinite language for which we cannot construct a Finite Automata?

That means, a language which is not regular?

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

Let's look at an example :

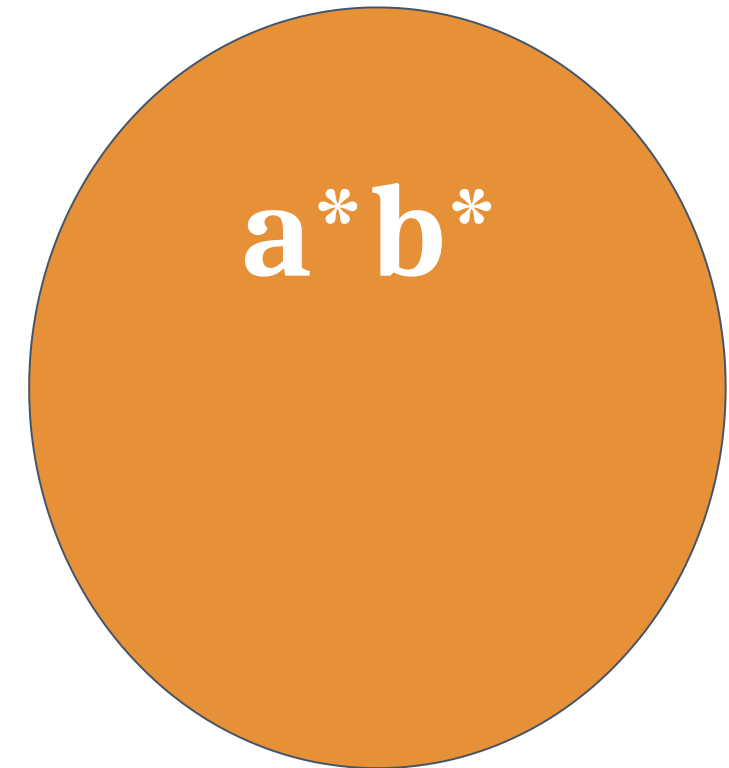
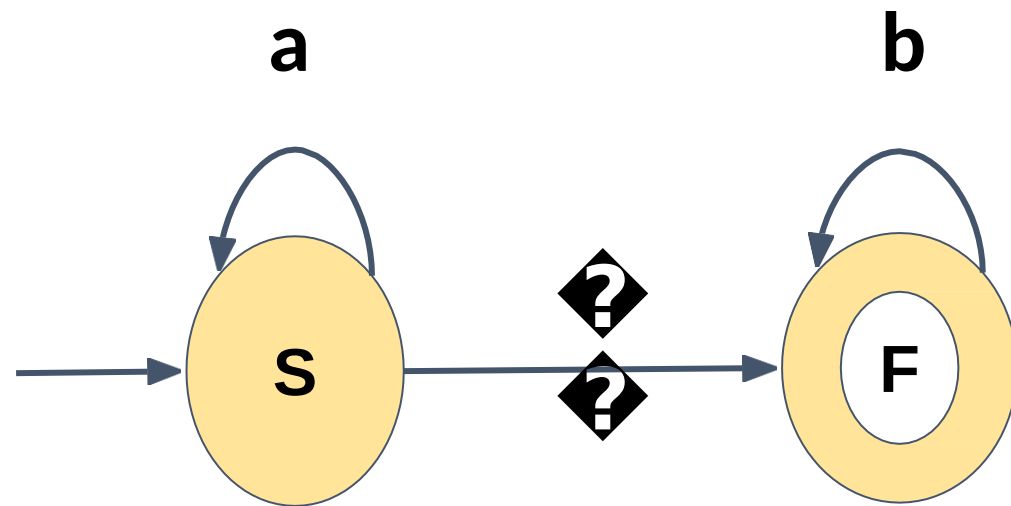


a^*b^*

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

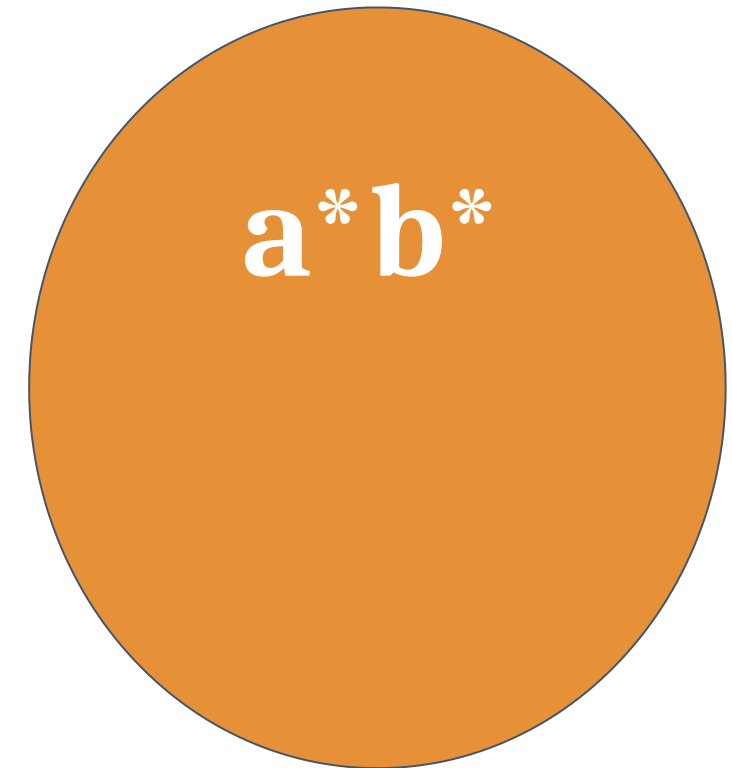
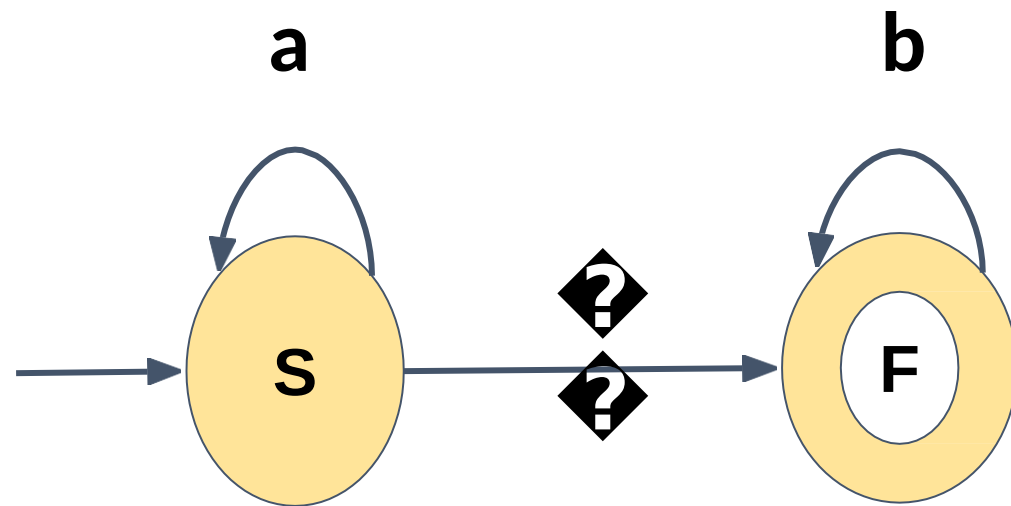
Let's look at an example :



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

Let's look at an example :



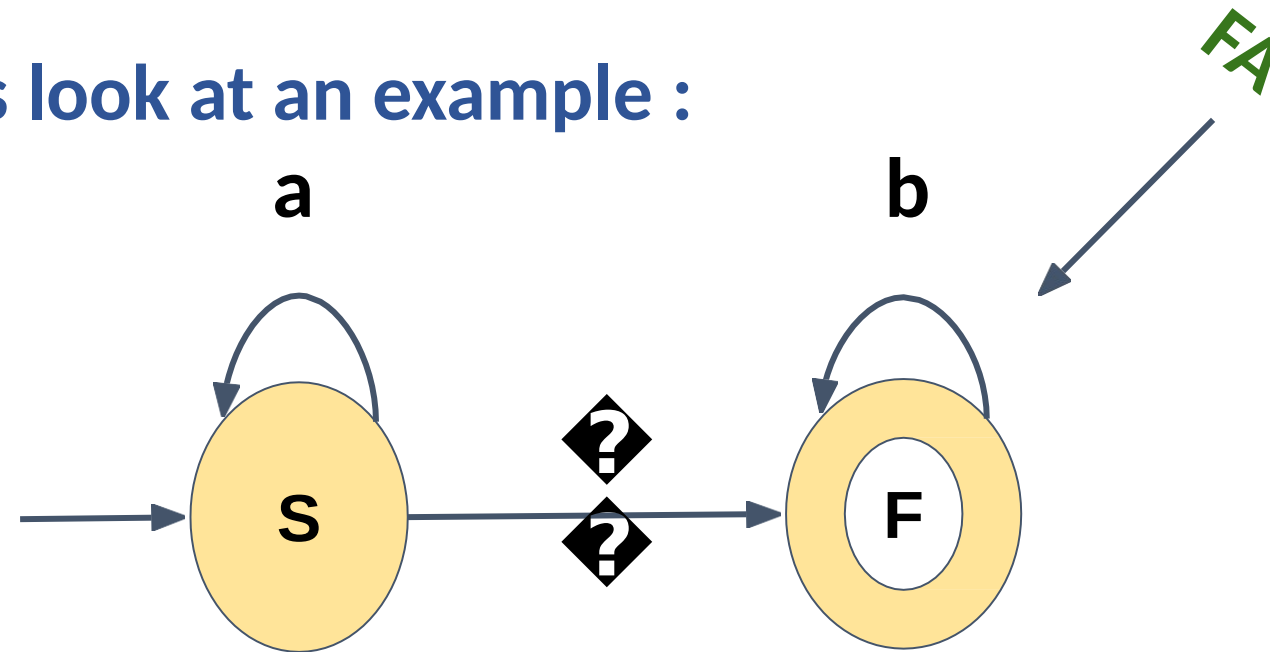
$S \rightarrow aS \mid F$

$F \rightarrow bF$
 $\mid \lambda$

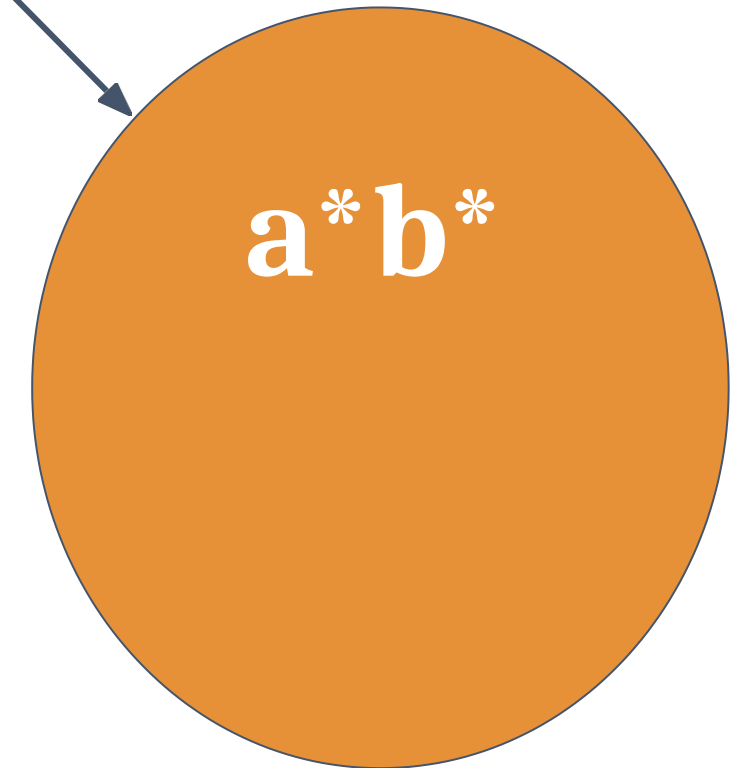
Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

Let's look at an example :



Regex



$S \rightarrow aS \mid F$

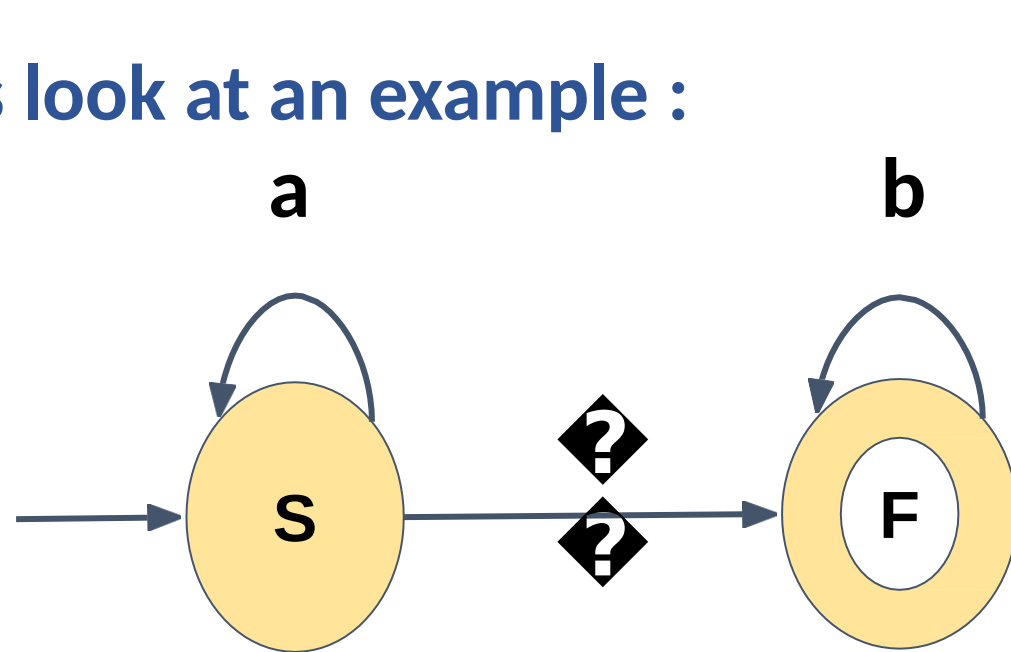
$F \rightarrow bF$
 $\mid \lambda$

Regular
Grammar

Automata Formal Languages and Logic

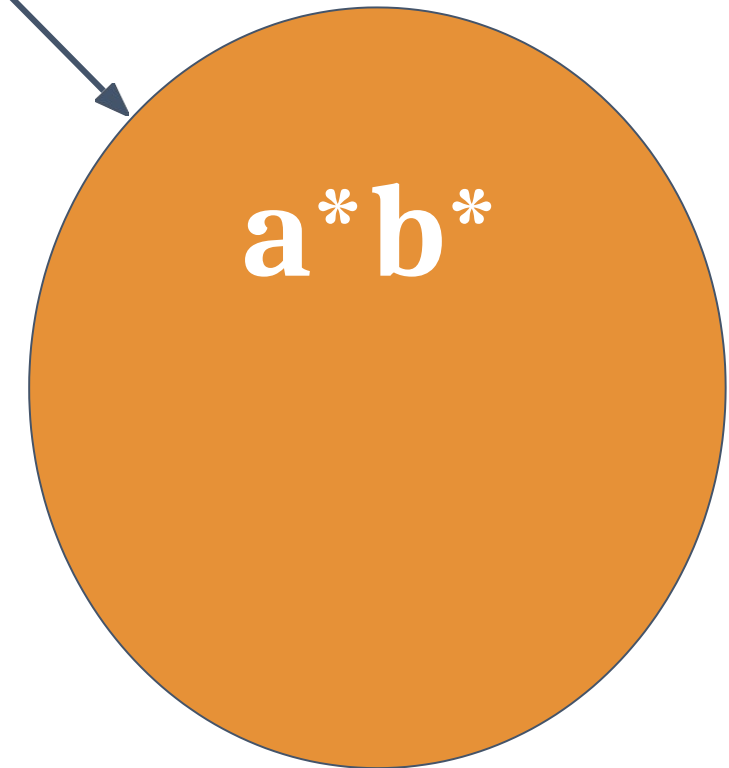
Unit 2 - Pumping Lemma for Regular Languages

Let's look at an example :



FA

Regex



Definitely Regular !

$S \rightarrow aS \mid F$

$F \rightarrow bF$
 $\mid \lambda$

Regular
Grammar

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

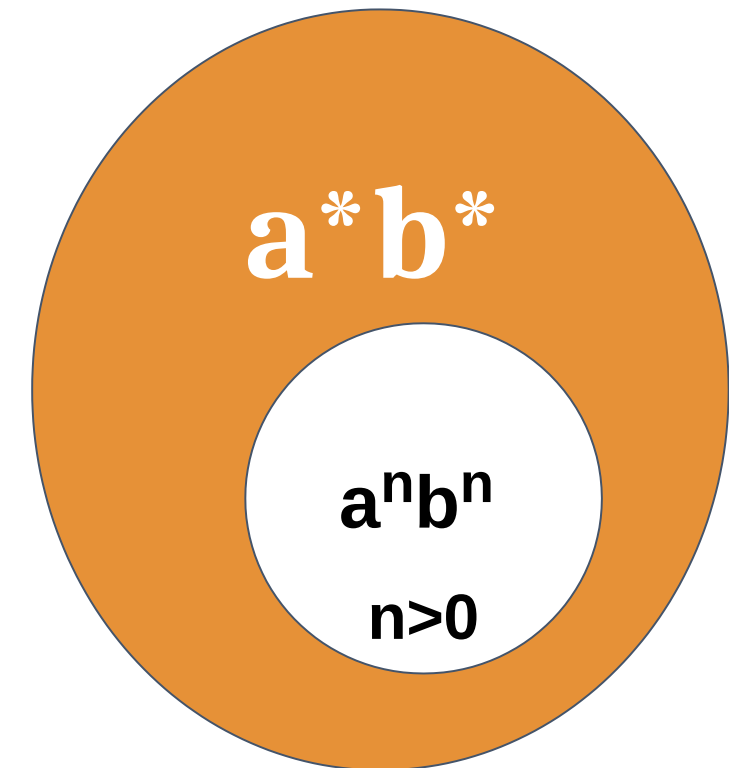
For the language $a^n b^n$ Can we construct either of :

Finite Acceptor or

Regular Grammar or

Regular Expression

????????????

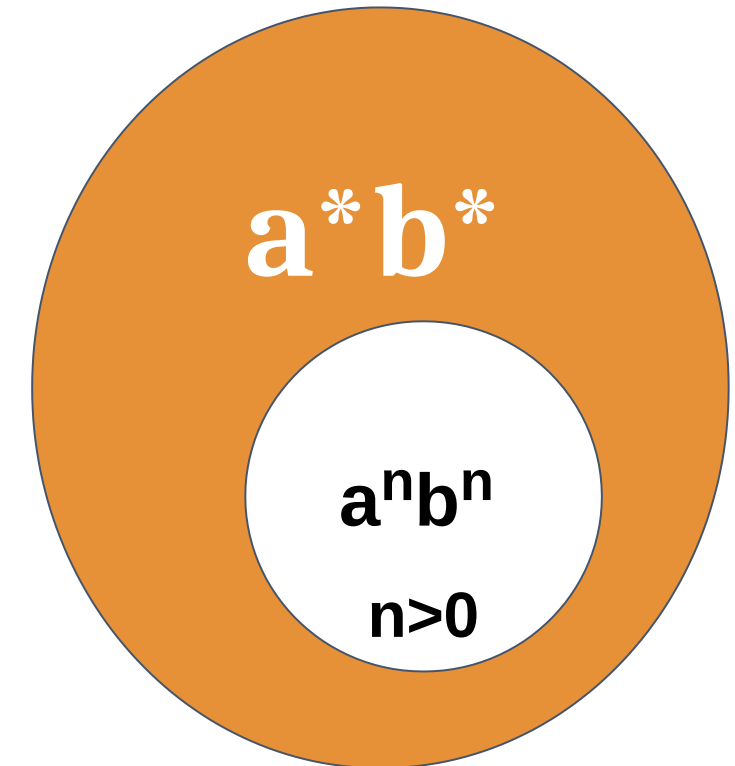


Unit 2 - Pumping Lemma for Regular Languages

aaaaaaaaaaaaaaaaaaaaaaaaa bbbbbbbbbbbbbbbbbbbbbbbbbb

↑

state q



Automata Formal Languages and Logic

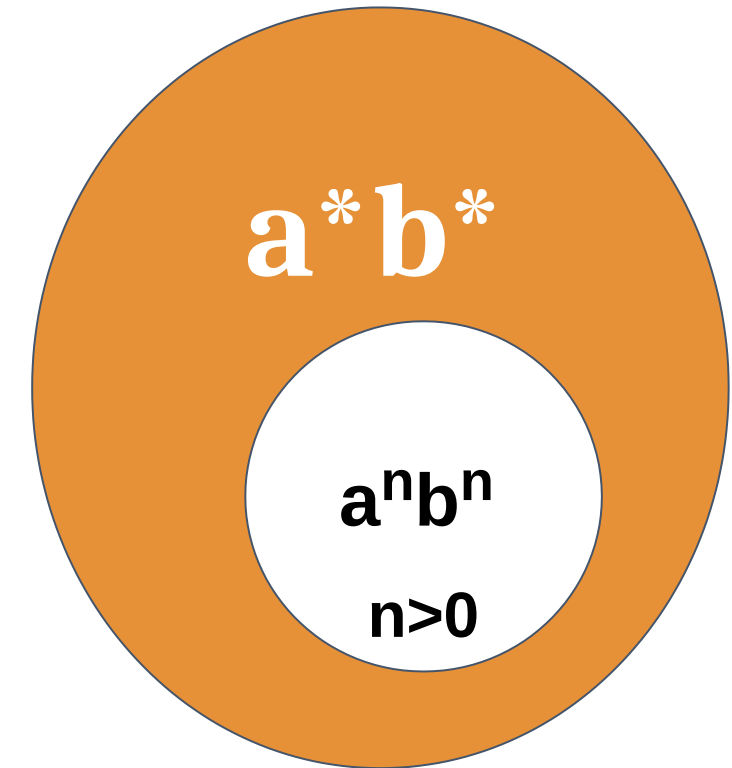
Unit 2 - Pumping Lemma for Regular Languages

Basically,

There is no way to remember how many a's you have seen to compare with the upcoming b's !

The value of n could be anything!

We cannot come up with a FA that takes care of all n!



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

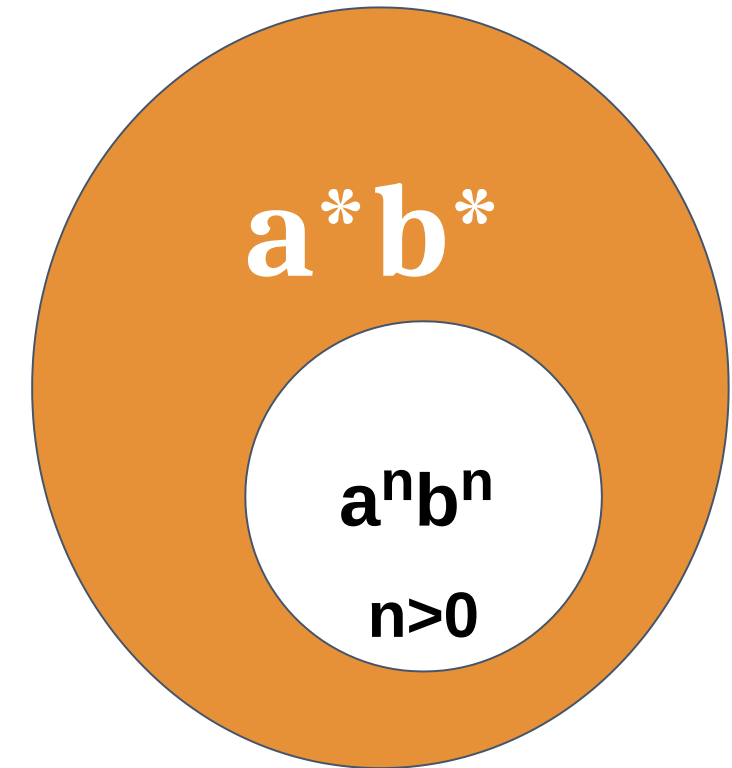
Basically,

There is no way to remember how many a 's you have seen to compare with the number of b 's.

Definitely Non-Regular!

The value of n could be anything!

We cannot come up with a FA that takes care of all n !



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

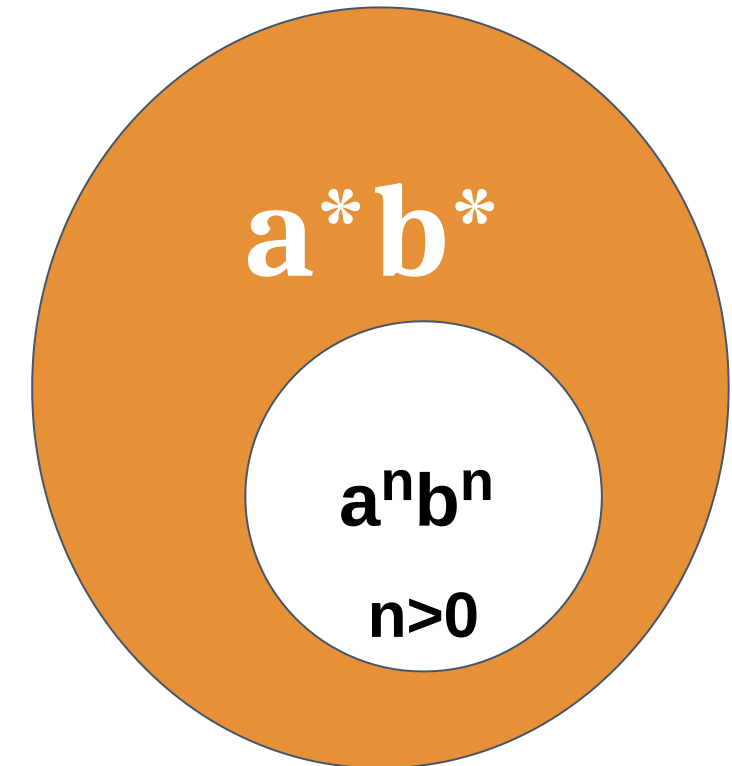
Basically,

There is no way to reach a contradiction
seen to compare the number of a's and b's you have

The value of n can be

We cannot come up with a FA that takes care of all n!

**Finite
Automata
has limits!**

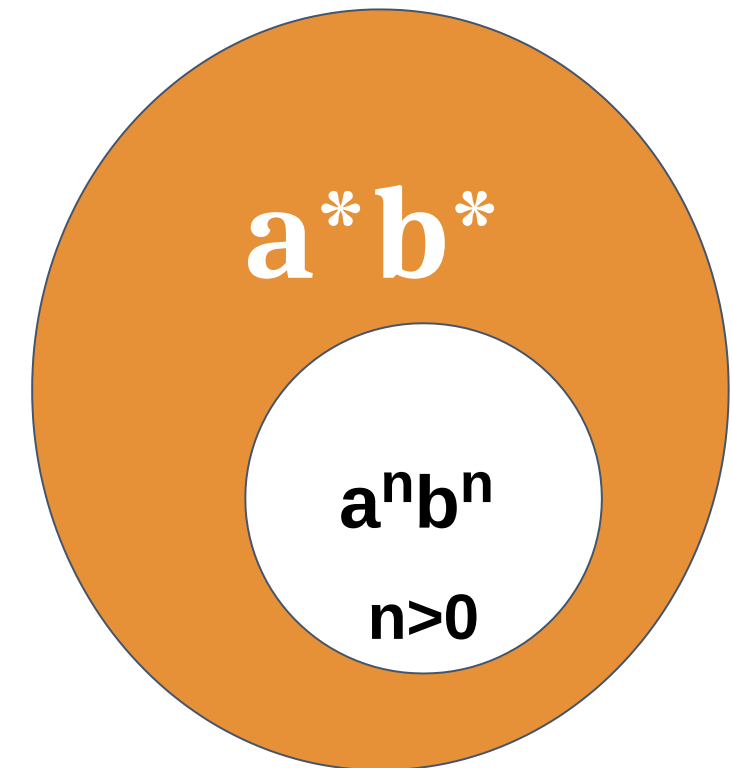


Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

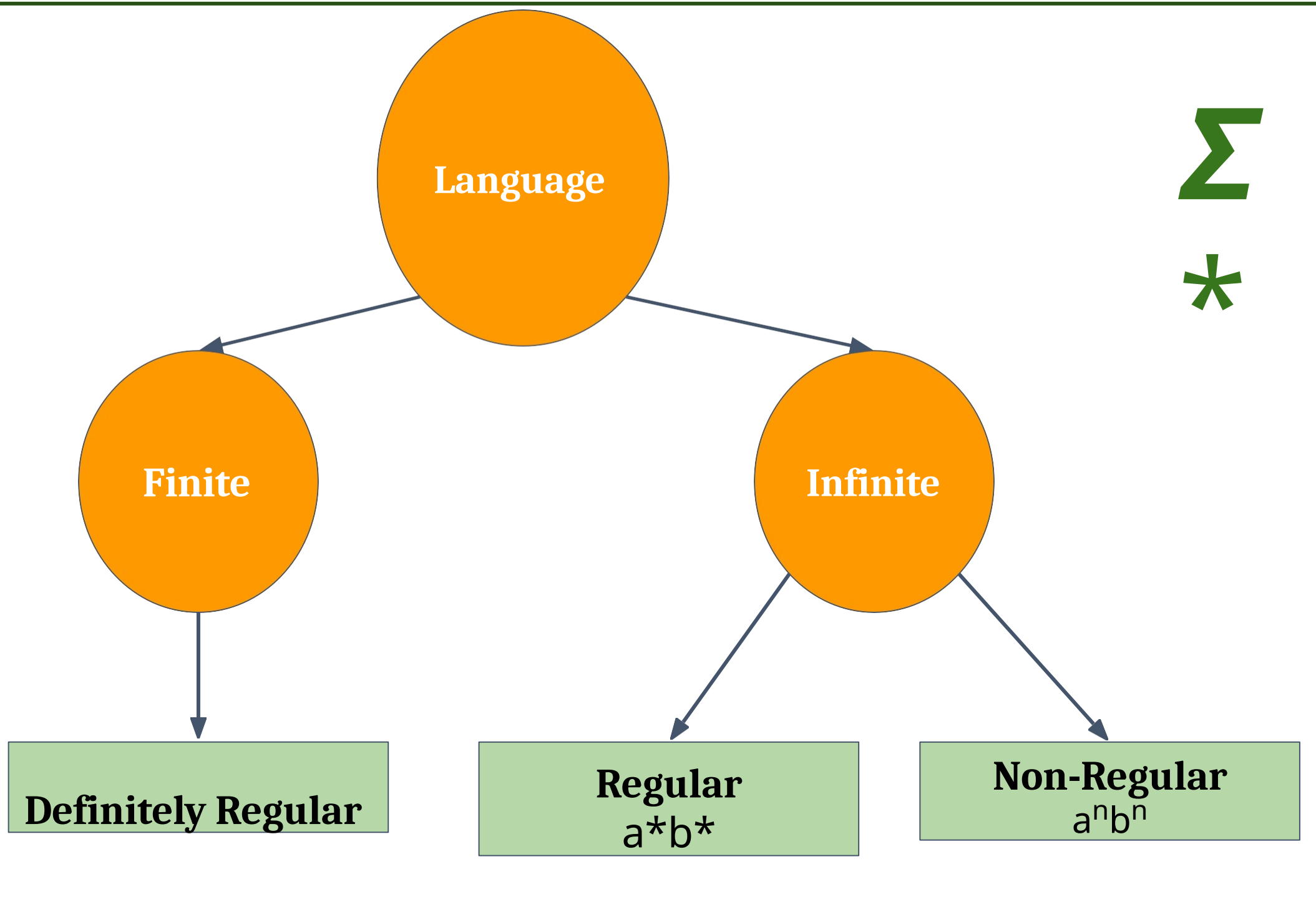
Limits of Finite Automata :

- Finite Memory !
- String comparison not possible - A finite automata can only "count" ; that is, It can maintain a counter, where different states correspond to different values of the counter but not same value.
- Linear Power :
an $n > 0$ is possible but a^{n^2} , a^{2^n} , a^i where i is prime is not

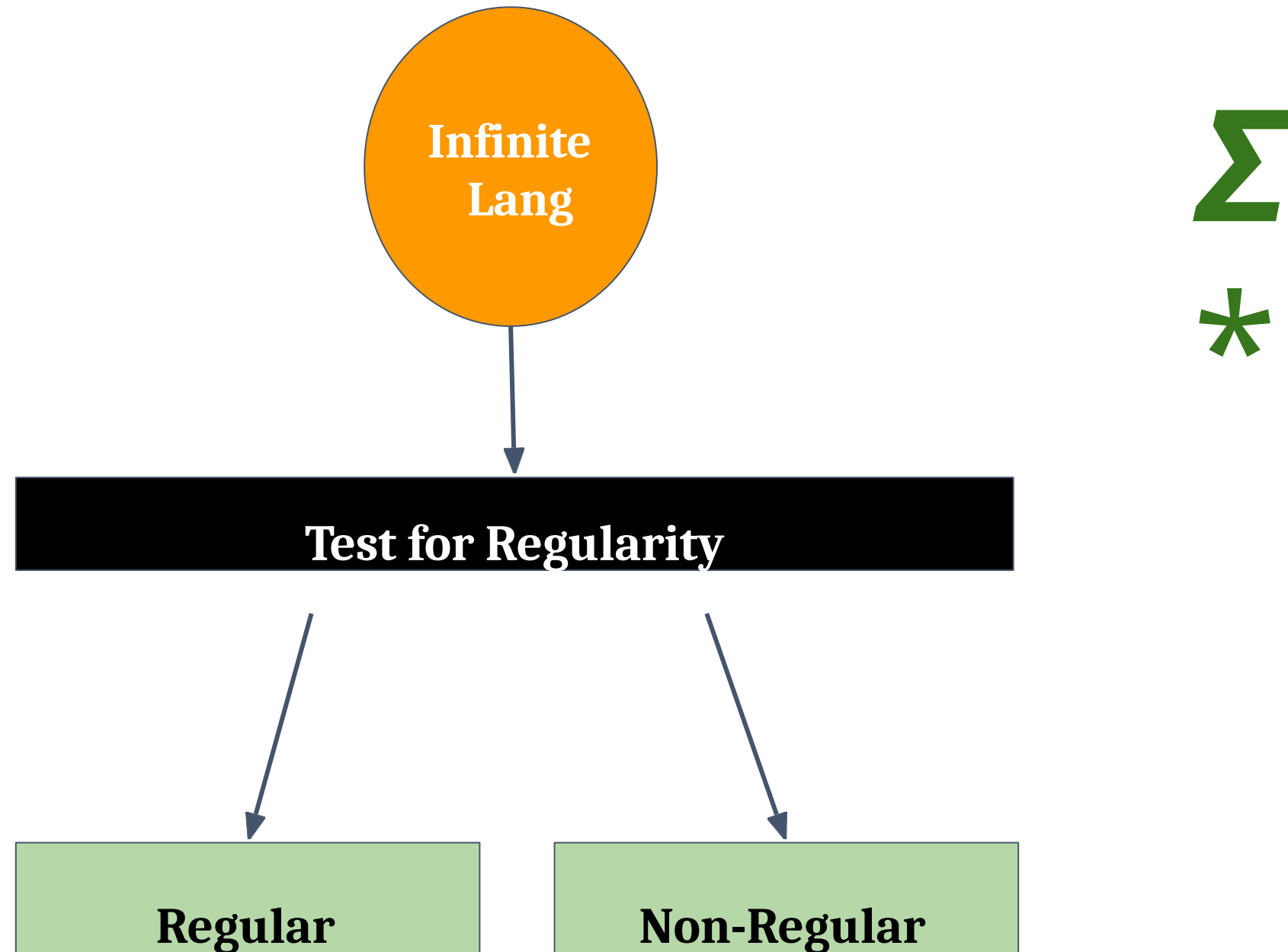


Automata Formal Languages and Logic

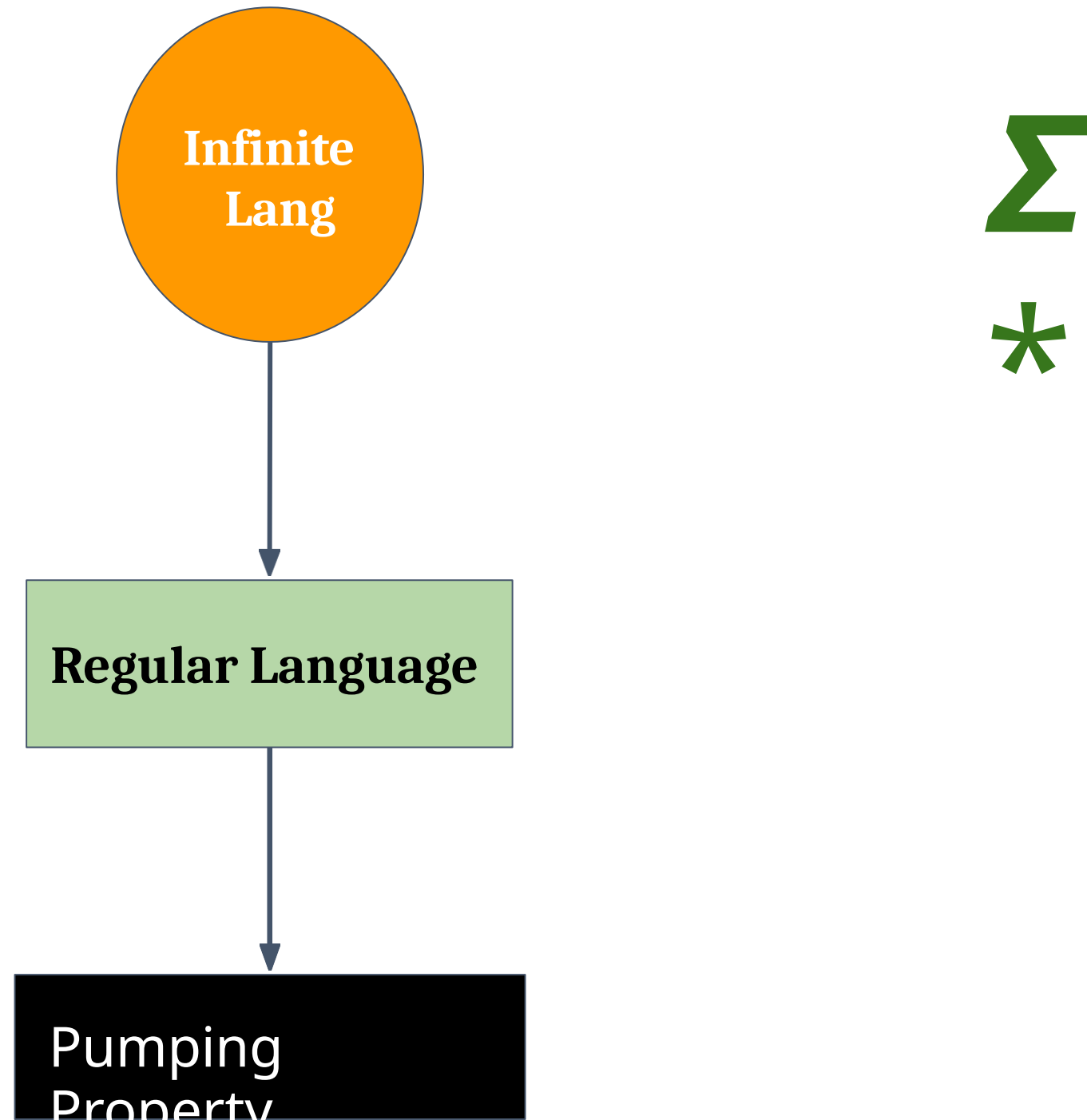
Unit 2 - Pumping Lemma for Regular Languages



Unit 2 - Pumping Lemma for Regular Languages

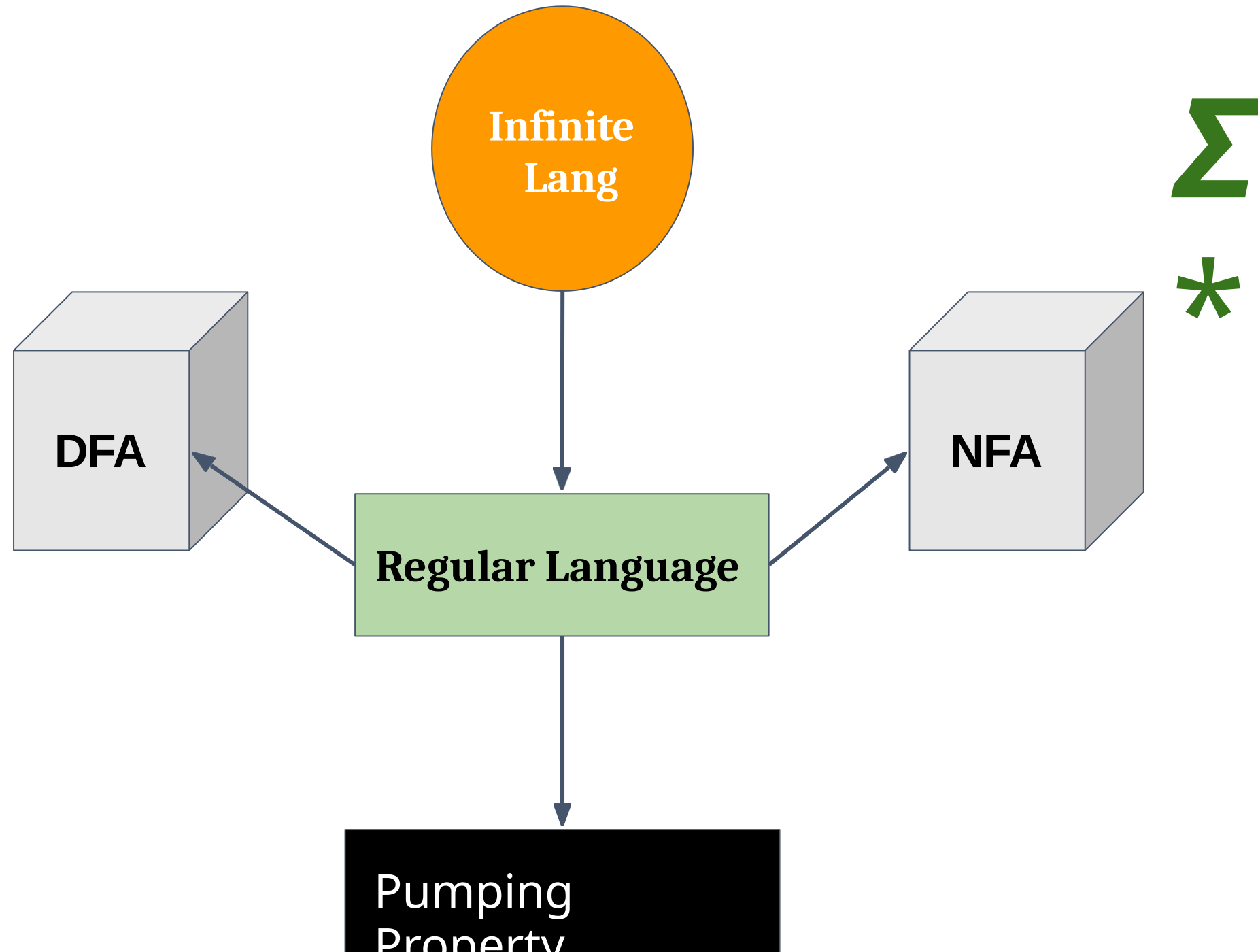


Unit 2 - Pumping Lemma for Regular Languages



Automata Formal Languages and Logic

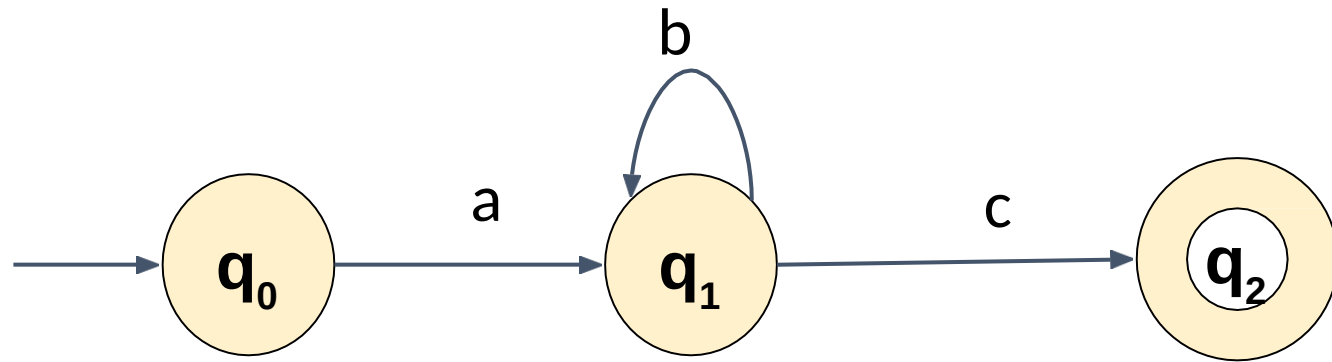
Unit 2 - Pumping Lemma for Regular Languages



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

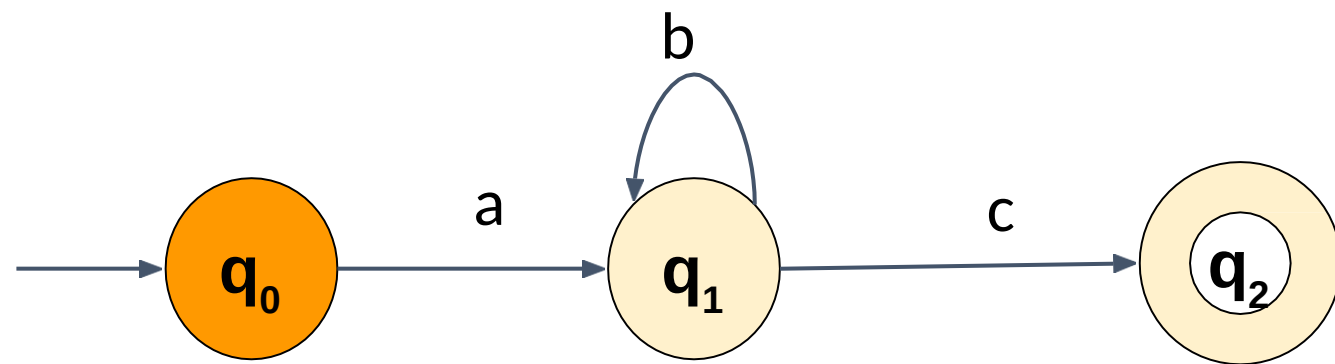
Let's take an example of infinite regular language ab^*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

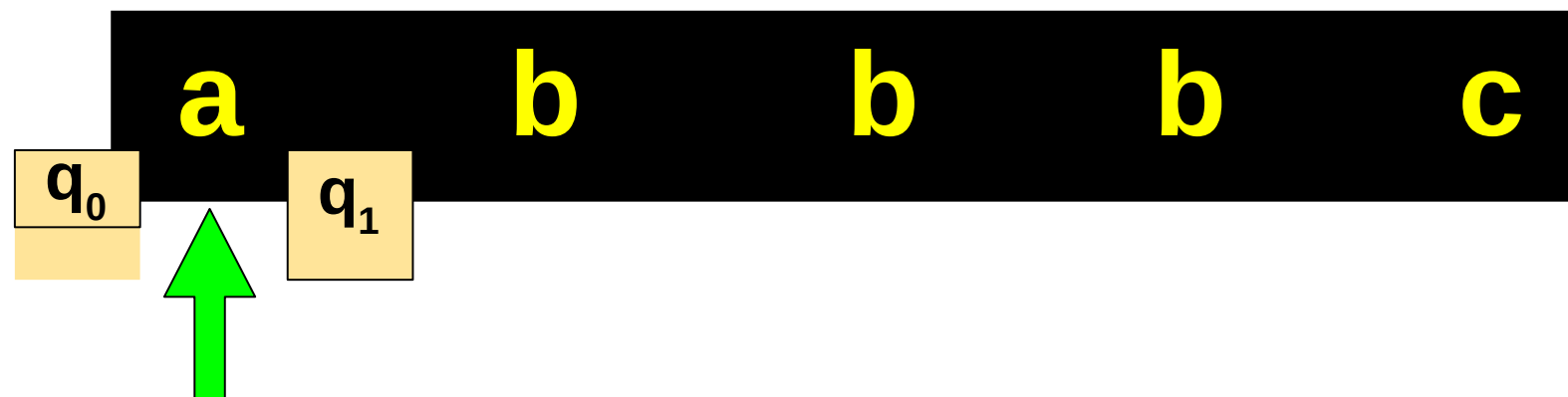
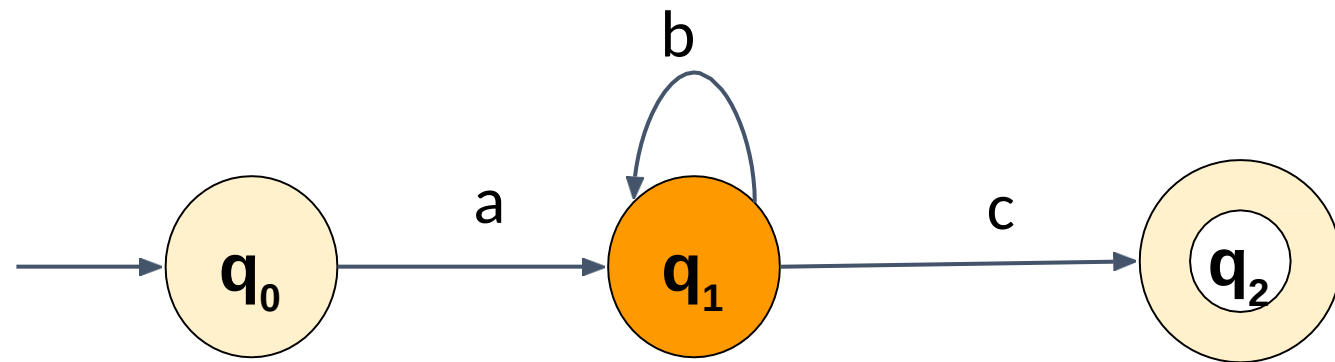
Let's take an example of infinite regular language ab^*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

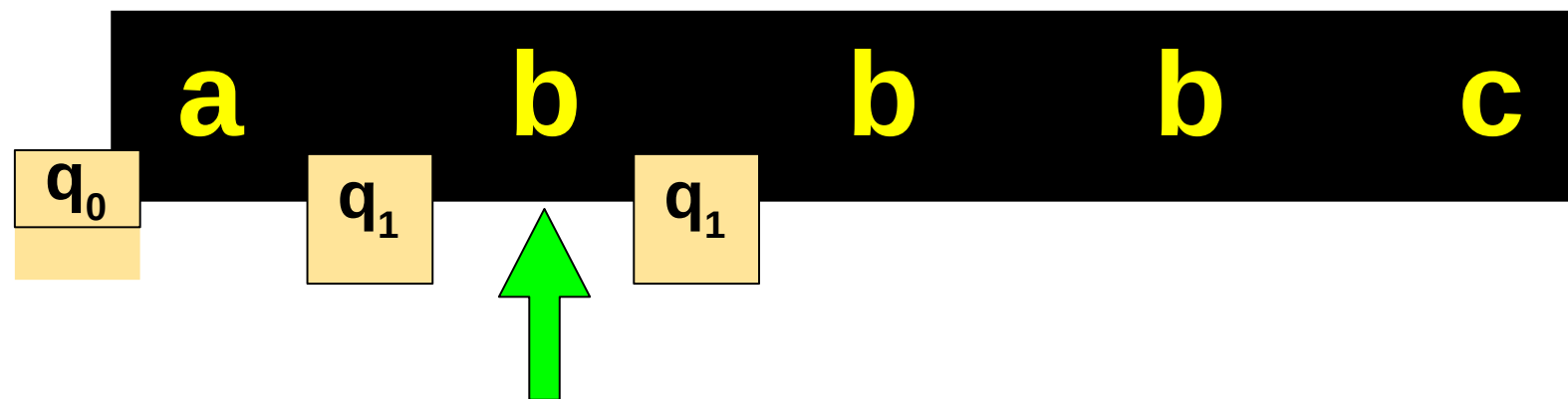
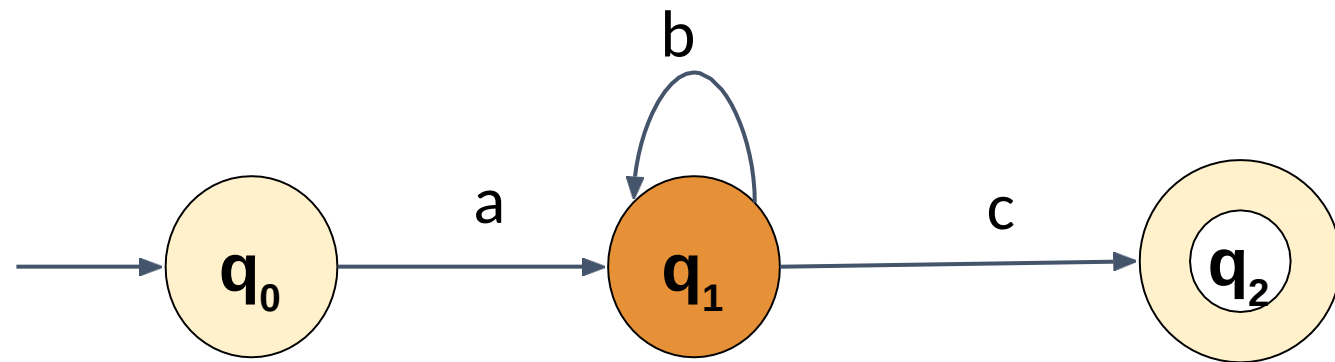
Let's take an example of infinite regular language ab^*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

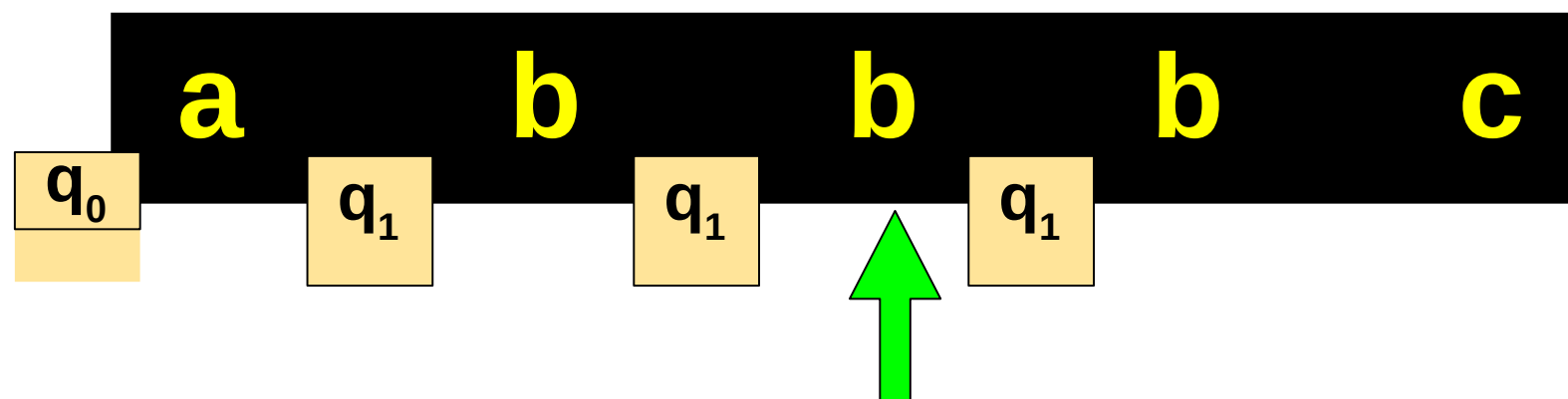
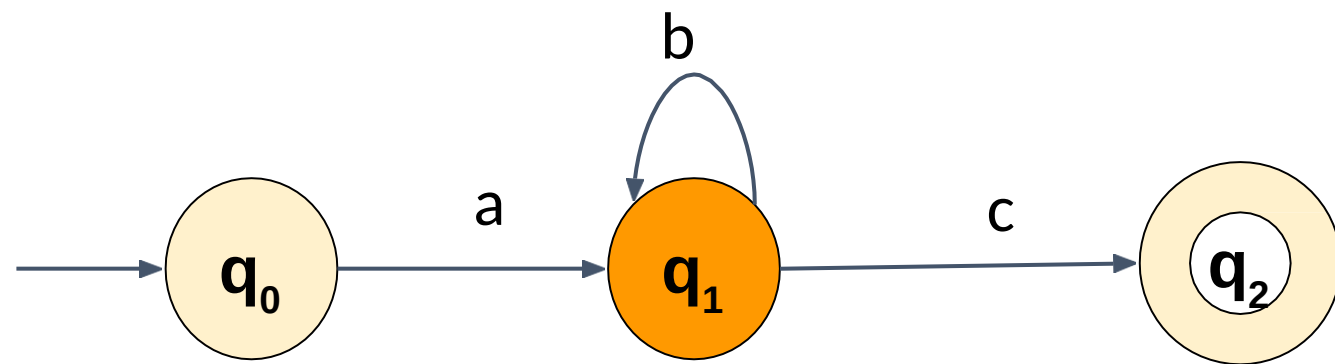
Let's take an example of infinite regular language ab^*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

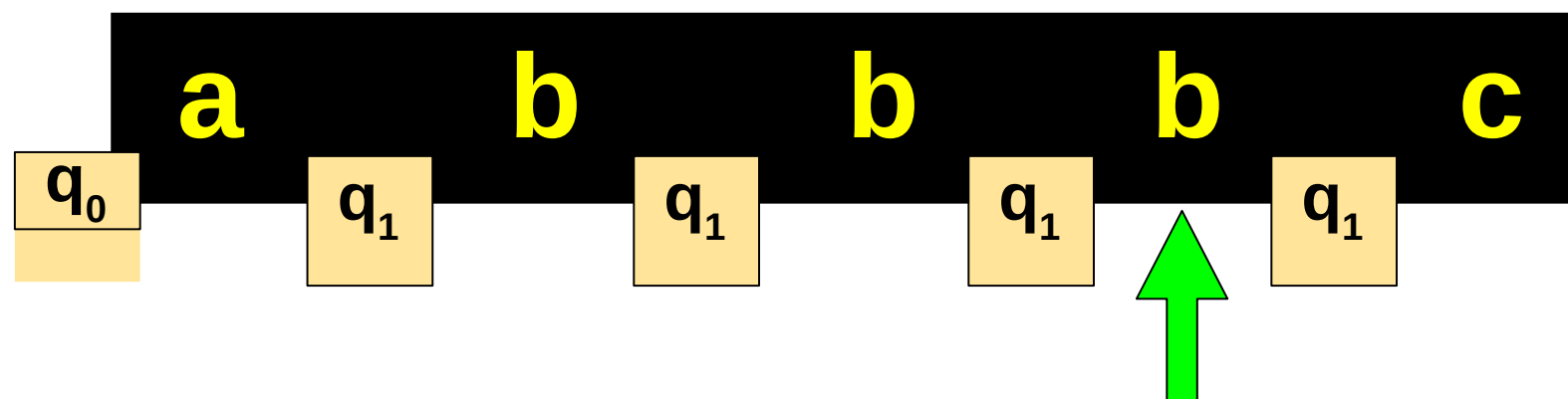
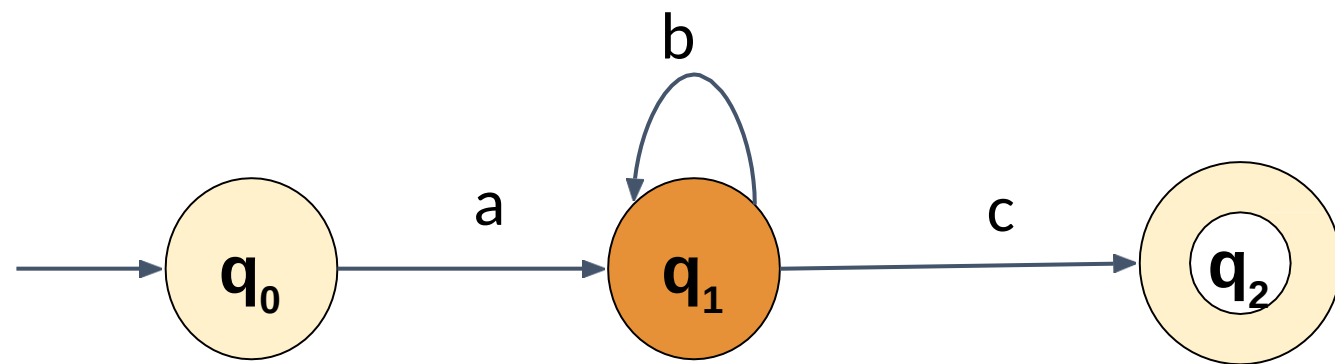
Let's take an example of infinite regular language ab^*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

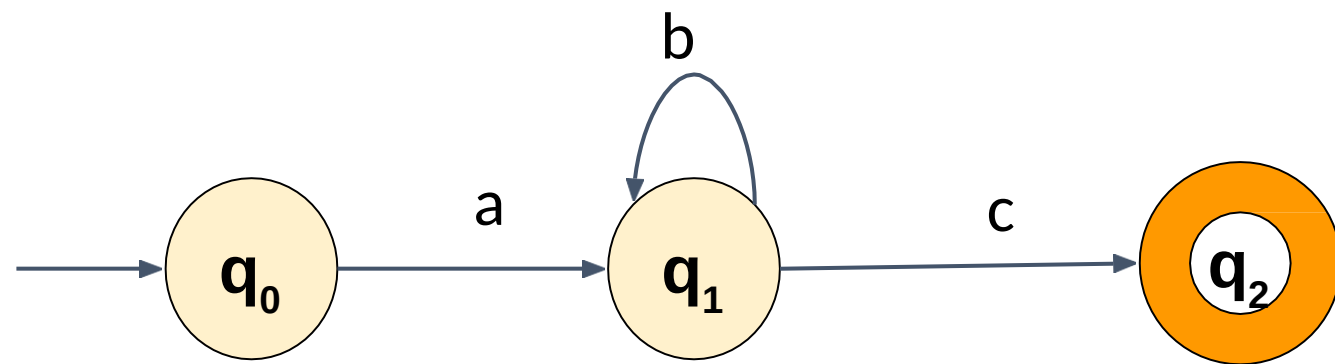
Let's take an example of infinite regular language ab^*c



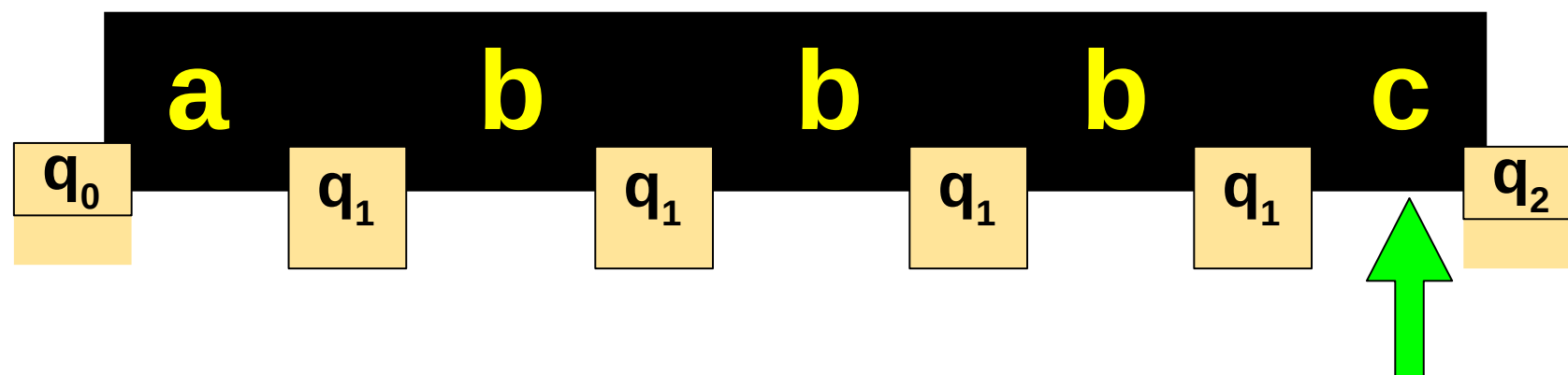
Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

Let's take an example of infinite regular language ab^*c

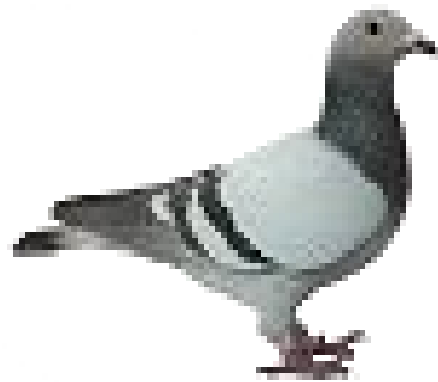


if $|w| \geq n$
we visit a set of states more than once



if $|w| \geq n$

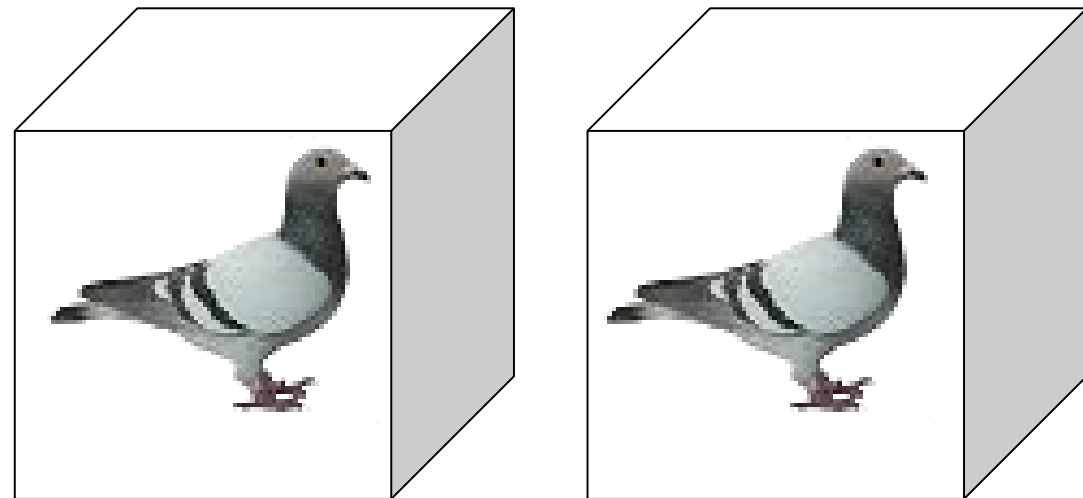
we visit a set of states more
than once



**The Pigeonhole
Principle**

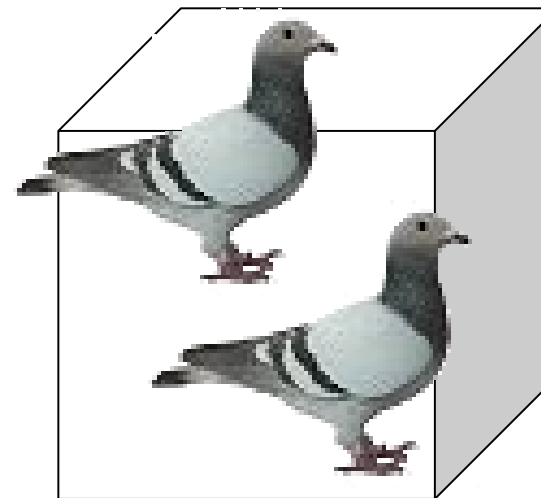
The Pigeonhole Principle

$n > m$
pigeons pigeonholes

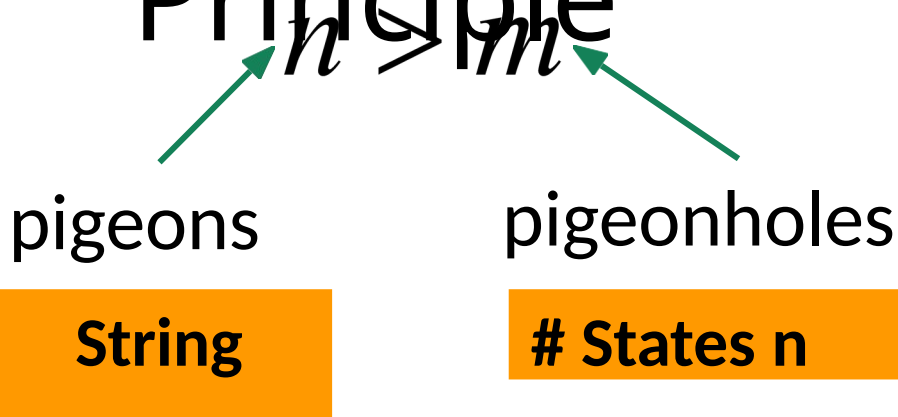


.....

There is a pigeonhole
with more than 1 pigeon

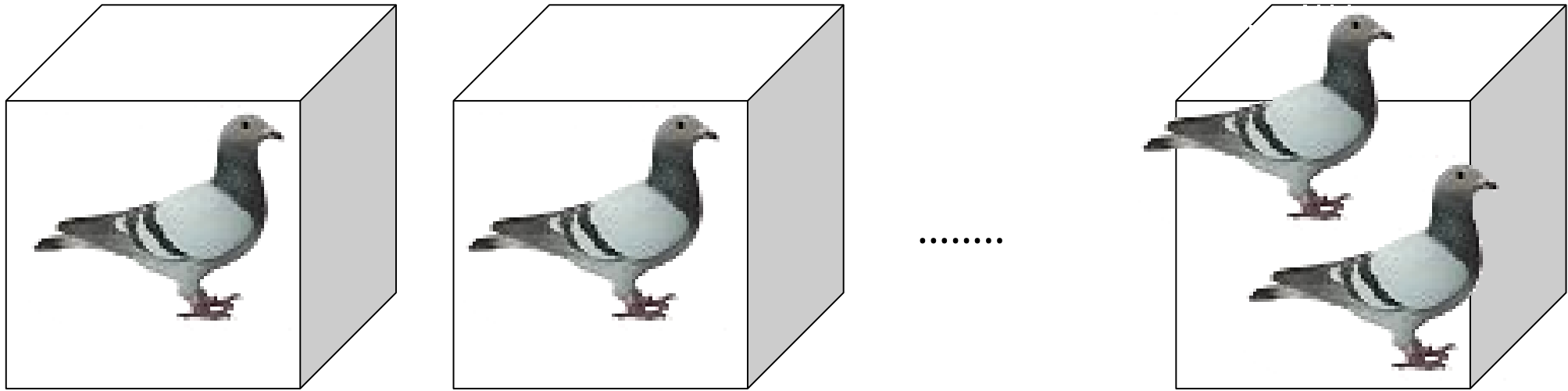


The Pigeonhole Principle



if $|w| > n$
A set of states will be visited
more than once

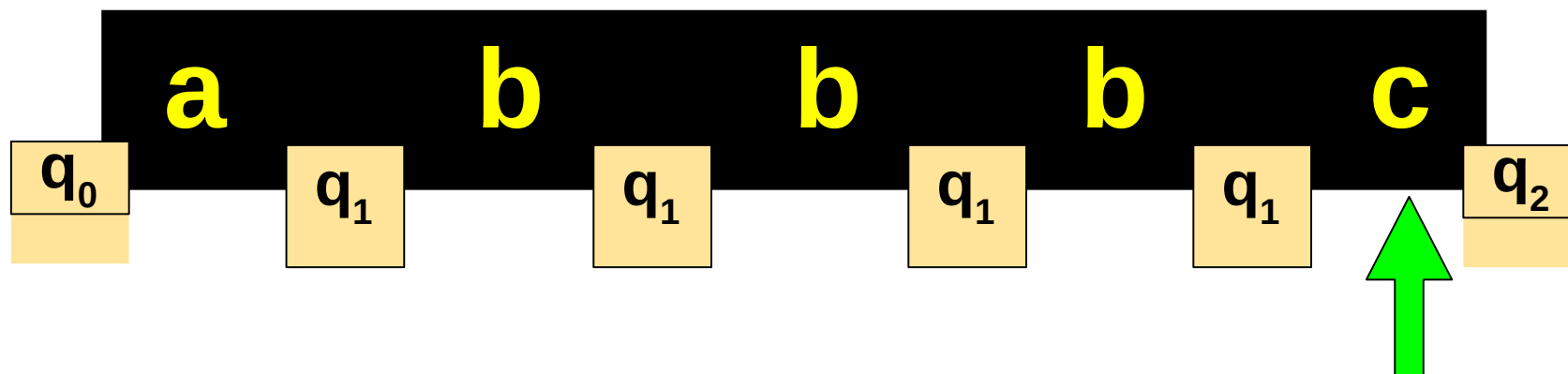
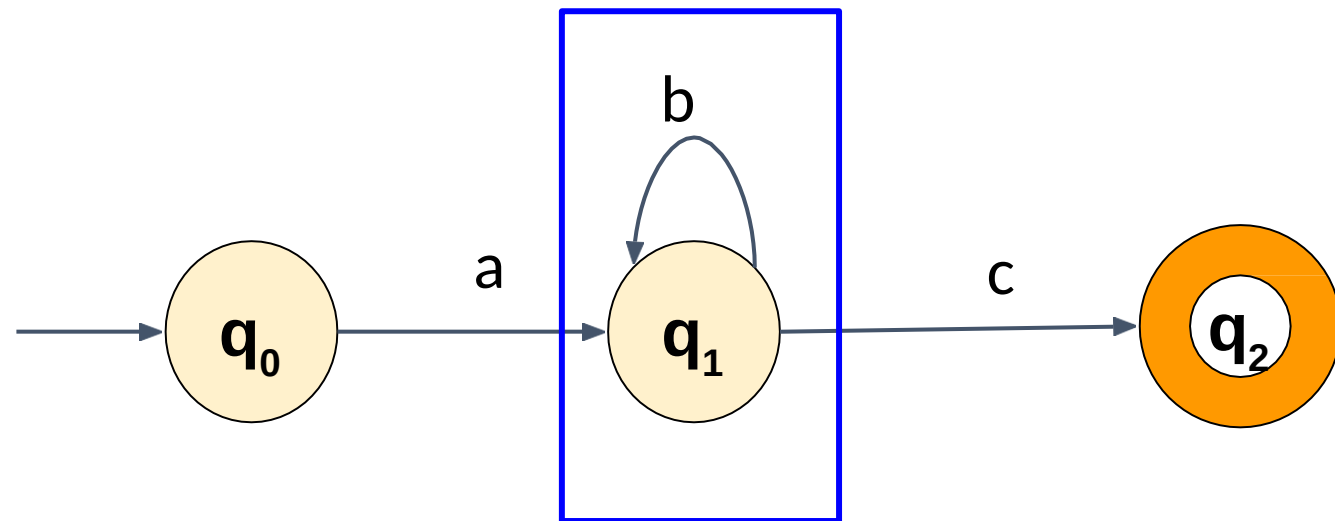
There is a pigeonhole
with more than 1 pigeon



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

Let's take an example of infinite regular language ab^*c



if $|w| \geq n$

we visit a set of states more than once which means,

there exists a loop in our Automata (within these n states)

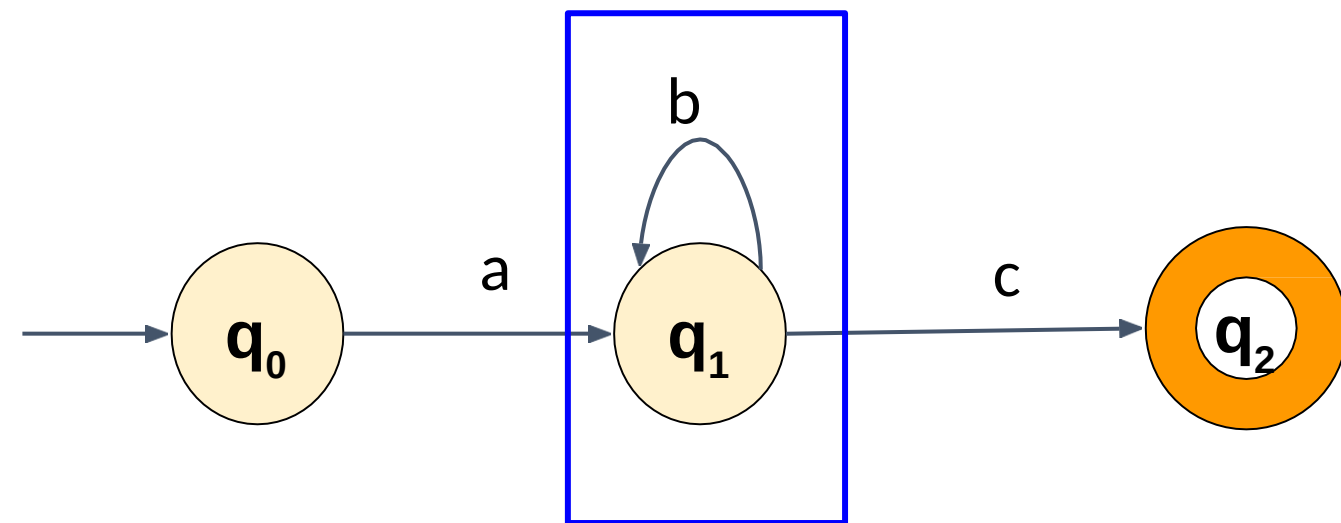
if we pump that loop 0 or more no. of times,

the resultant string will always be in the language

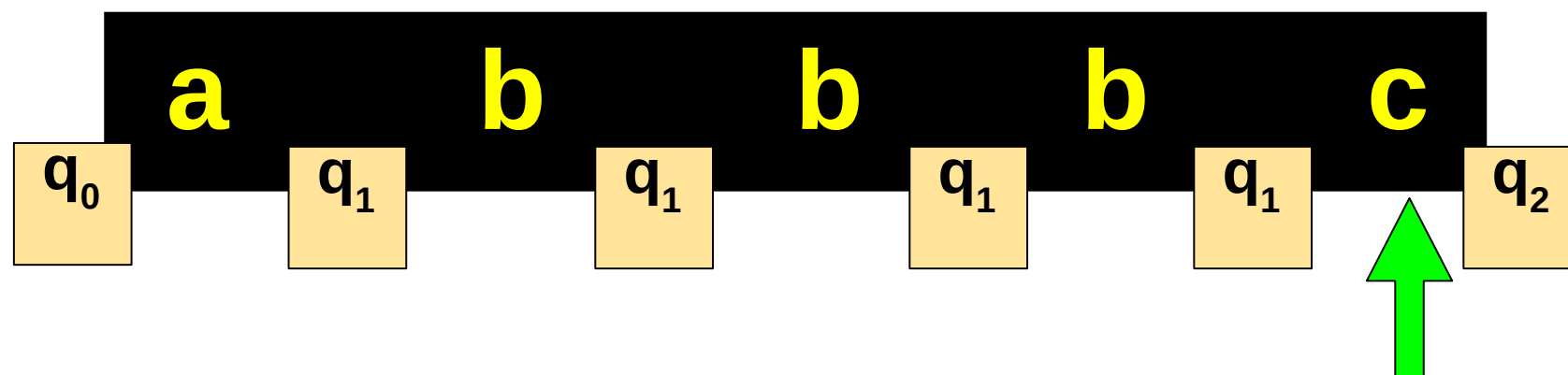
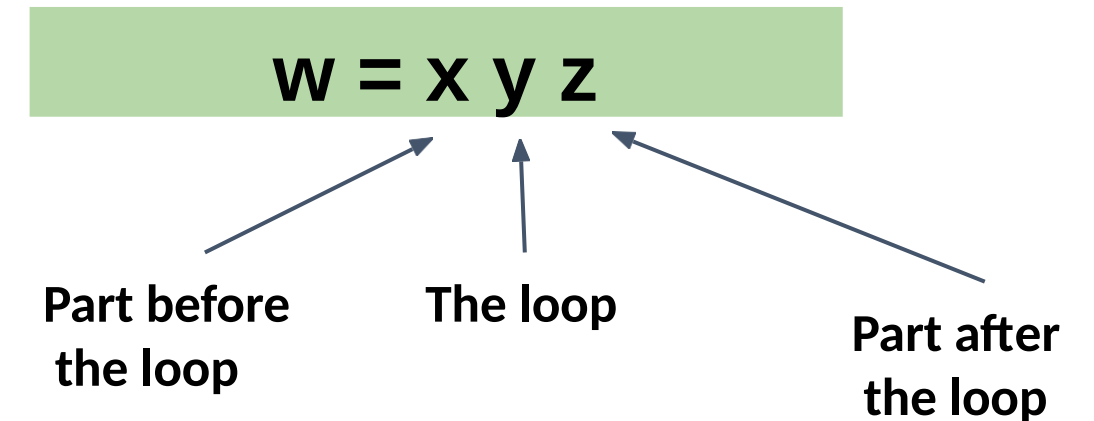
Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

Let's take an example of infinite regular language ab^*c



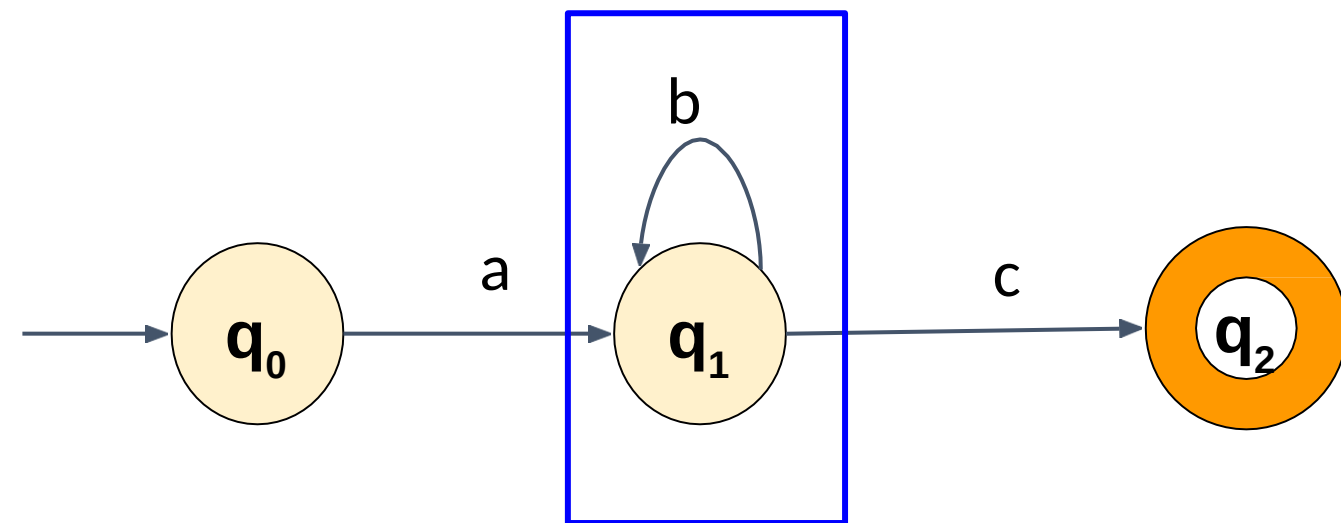
There exists 3 parts to a string w :



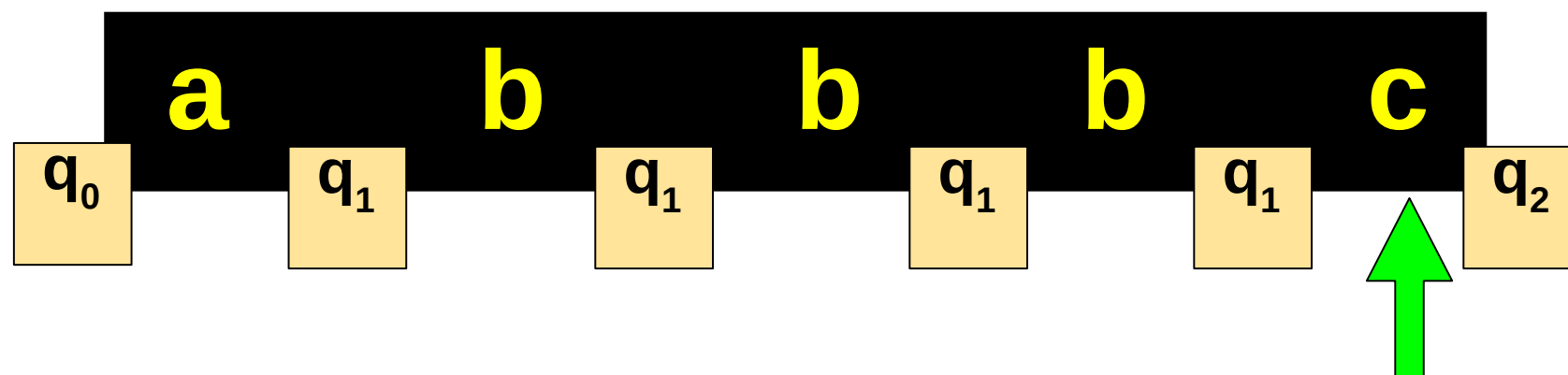
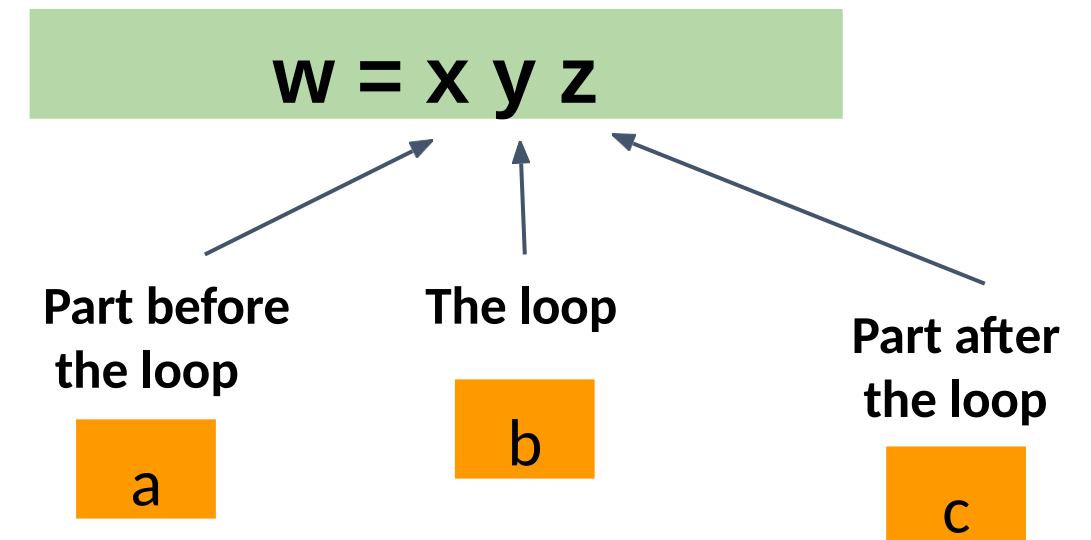
Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

Let's take an example of infinite regular language ab^*c



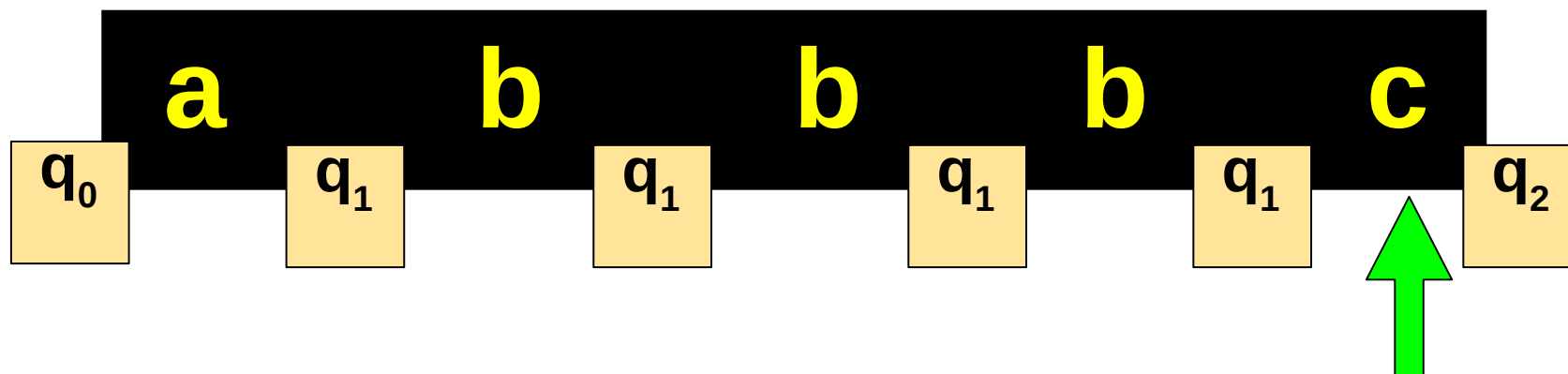
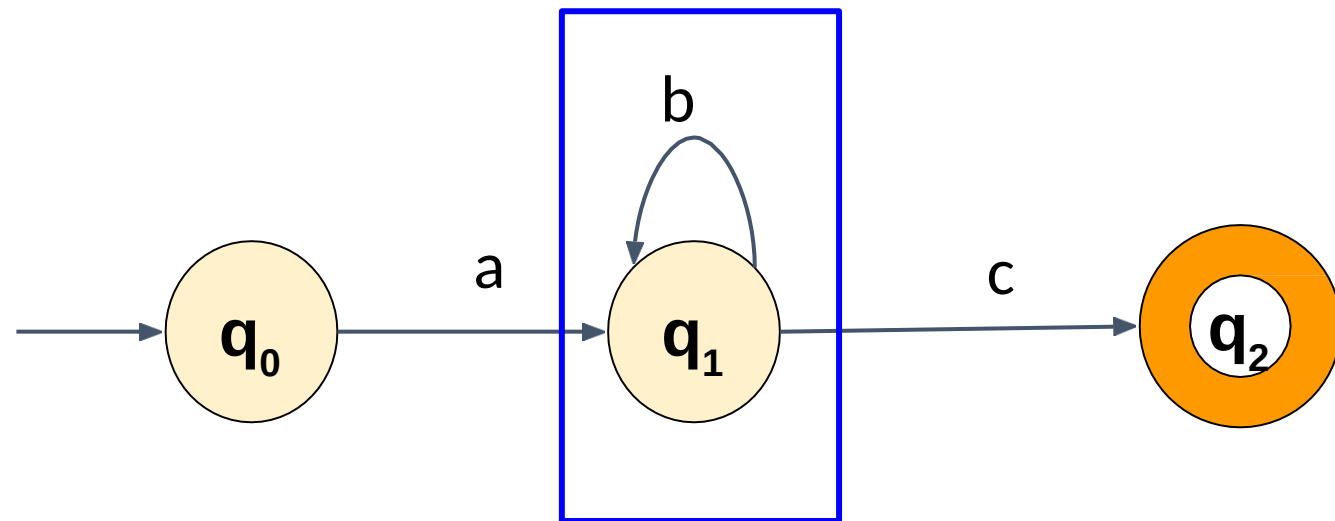
There exists 3 parts to a string w :



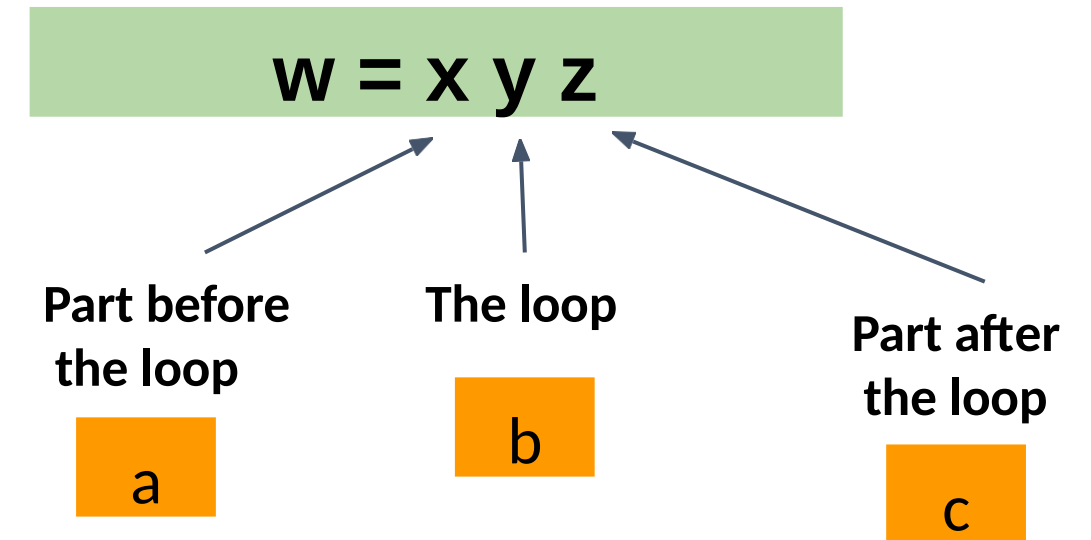
Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

Let's take an example of infinite regular language ab^*c



There exists 3 parts to a string w :



$y \neq \epsilon$ that is $|y| \geq 1$

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

The Pumping property States,

For every Regular language L, **(infinite)**

there exists n where n is the # states in Finite Automata for L

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

The Pumping property States,

For every Regular language L, **(infinite)**

there exists n where n is the # states in Finite Automata for L

In our example of infinite regular language ab^*c

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

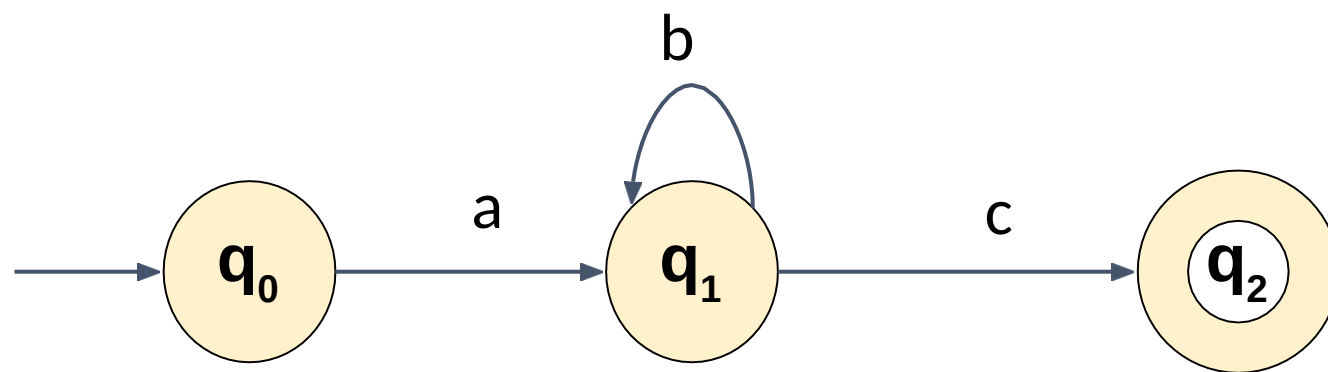
The Pumping property States,

For every Regular language L, **(infinite)**

there exists n where n is the # states in Finite Automata for L

$n = 3$

In our example of infinite regular language ab^*c



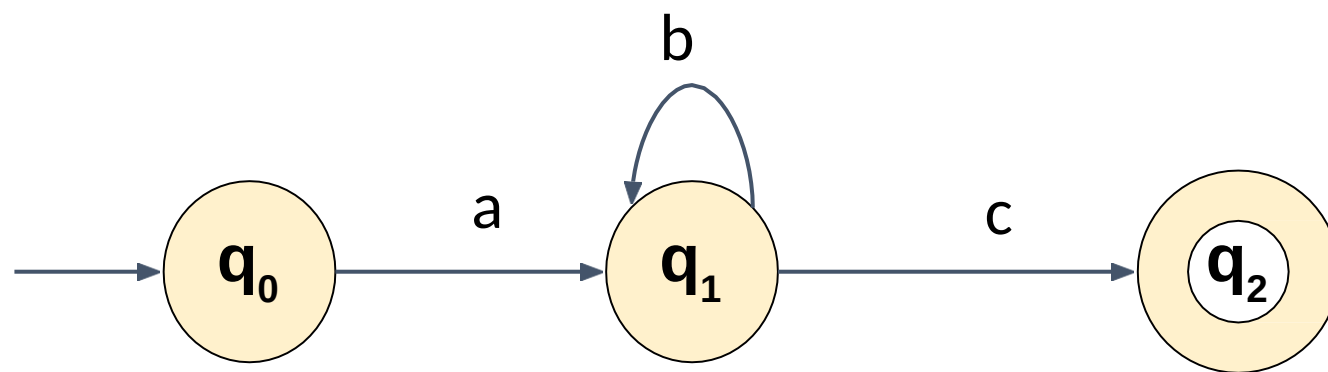
The Pumping property States,

For every Regular language L, **(infinite)**

there exists n where n is the # states in Finite Automata for L



In our example of infinite regular language ab^*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

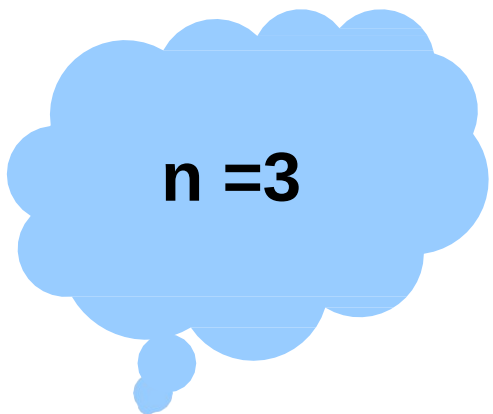
The Pumping property States,

For every Regular language L, **(infinite)**

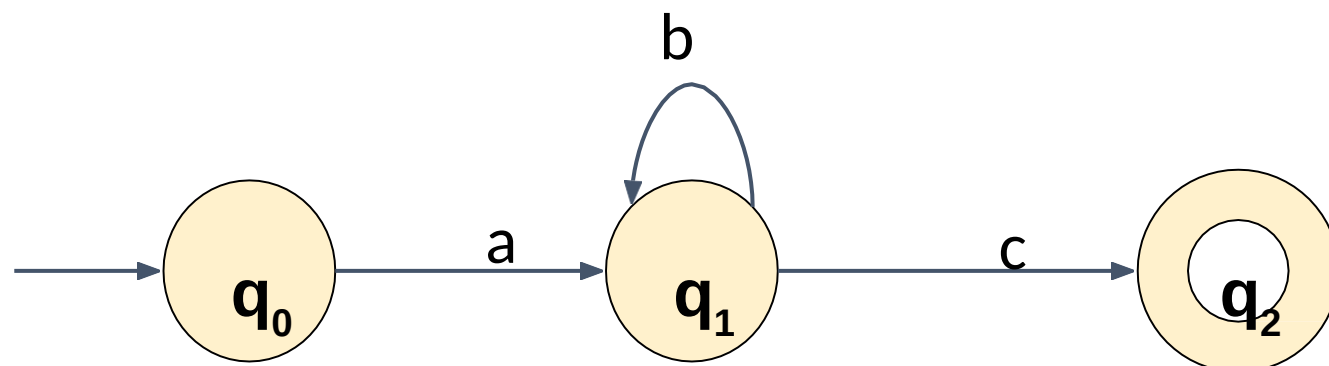
there exists n where n is the # states in Finite Automata for L

For every string w that belongs to L such that,

$$|w| \geq n$$


$$n = 3$$

In our example of infinite regular language ab^*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

The Pumping property States,

For every Regular language L, **(infinite)**

there exists n where n is the # states in Finite Automata for L

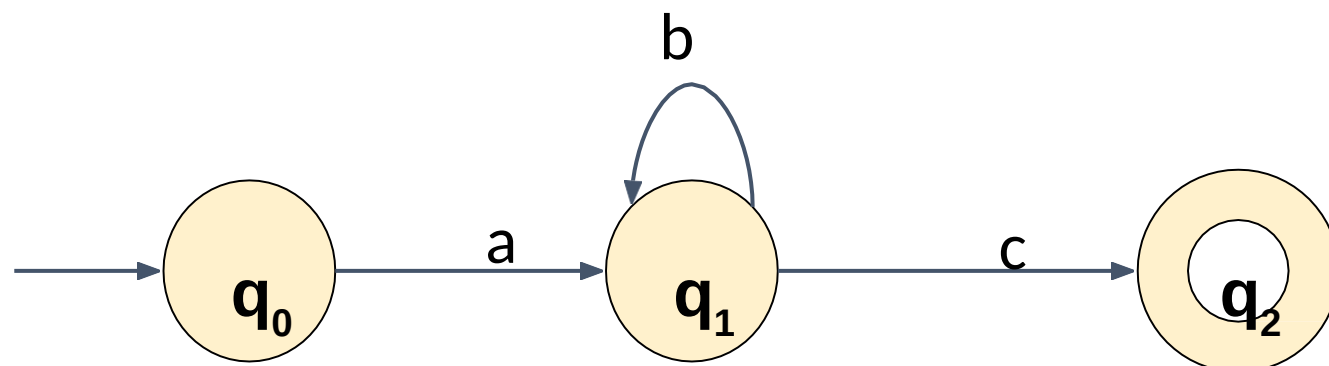
For every string w that belongs to L such that,

$$|w| \geq n$$

w = abbbc
 $|w| = 5 > n$

n = 3

In our example of infinite regular language ab^*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

The Pumping property States,

For every Regular language L, **(infinite)**

there exists n where n is the # states in Finite Automata for L

For every string **w** that belongs to L such that,

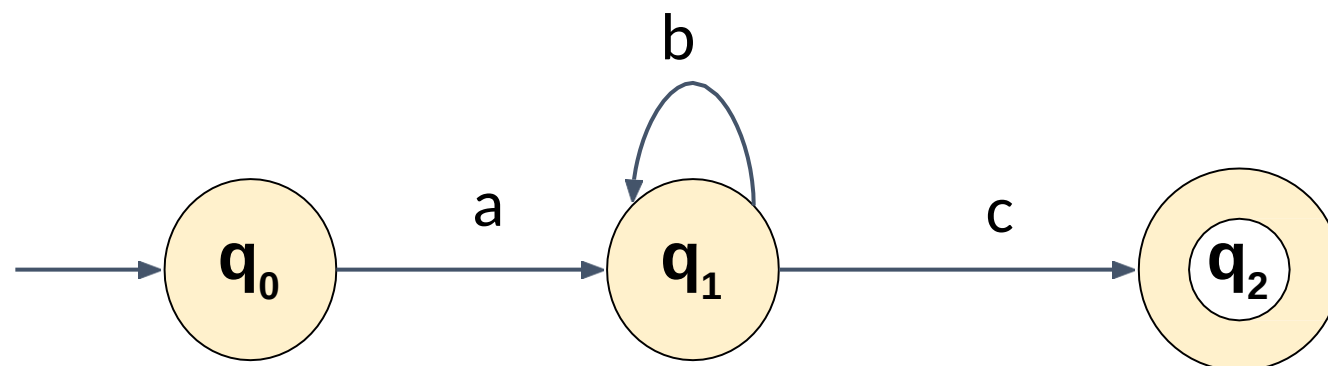
$$|w| \geq n$$

There exists a break up of the string in three parts $w = xyz$ such that $|y| \geq 1$ and $|xy| \leq n$

$$w = abbbc$$
$$|w| = 5 > n$$

$$n = 3$$

In our example of infinite regular language ab^*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

The Pumping property States,

For every Regular language L, **(infinite)**

there exists n where n is the # states in Finite Automata for L

For every string **w** that belongs to L such that,

$$|w| \geq n$$

There exists a break up of the string in three parts $w = xyz$ such that $|y| \geq 1$ and $|xy| \leq n$

$$w = abbbc$$

$$|w| = 5 > n$$

$$n = 3$$

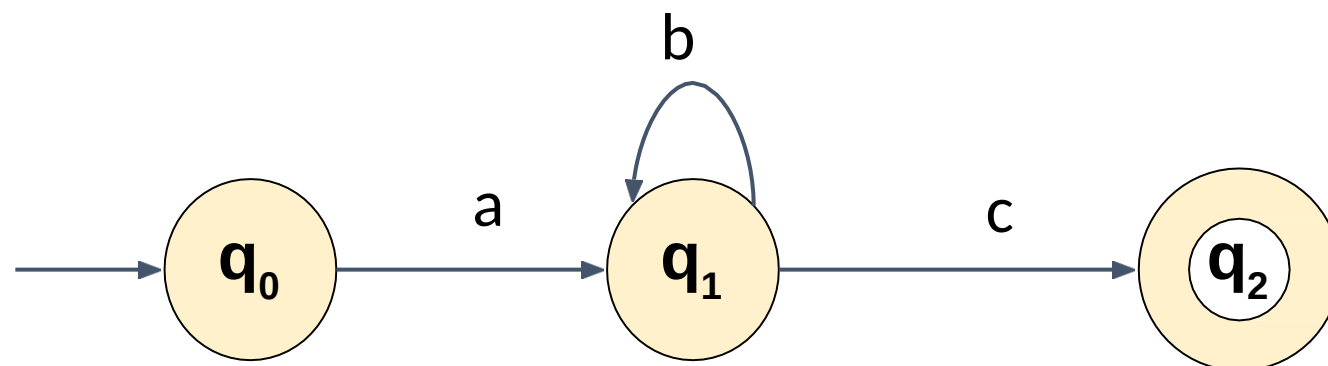
$$w = abc$$

$$x = a$$

$$y = b$$

$$z = c$$

In our example of infinite regular language ab^*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

The Pumping property States,

For every Regular language L, **(infinite)**

there exists n where n is the # states in Finite Automata for L

For every string **w** that belongs to L such that,

$$|w| \geq n$$

There exists a break up of the string in three parts **w = xyz**

such that $|y| \geq 1$ and $|xy| \leq n$,

for every $i \geq 0$,

xy^iz belongs to L

w = abbbc

$|w| = 5 > n$

n = 3

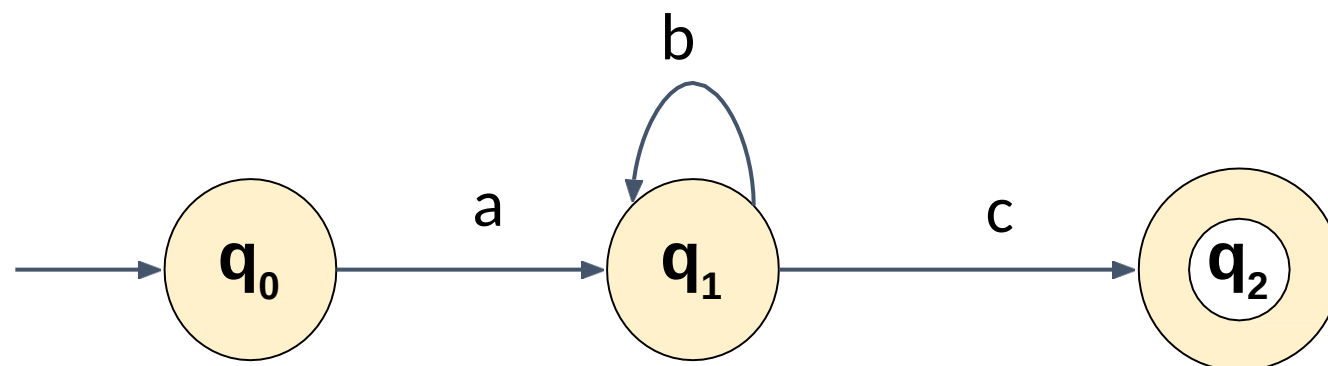
w = abc

x = a

y = b

z = c

In our example of infinite regular language ab^*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

The Pumping property States,

For every Regular language L, (**infinite**)

there exists n where n is the # states in Finite Automata for L

For every string **w** that belongs to L such that,

$$|w| \geq n$$

There exists a break up of the string in three parts **w = xyz** such that $|y| \geq 1$ and $|xy| \leq n$,
for every **i** ≥ 0 ,

xyⁱz belongs to L

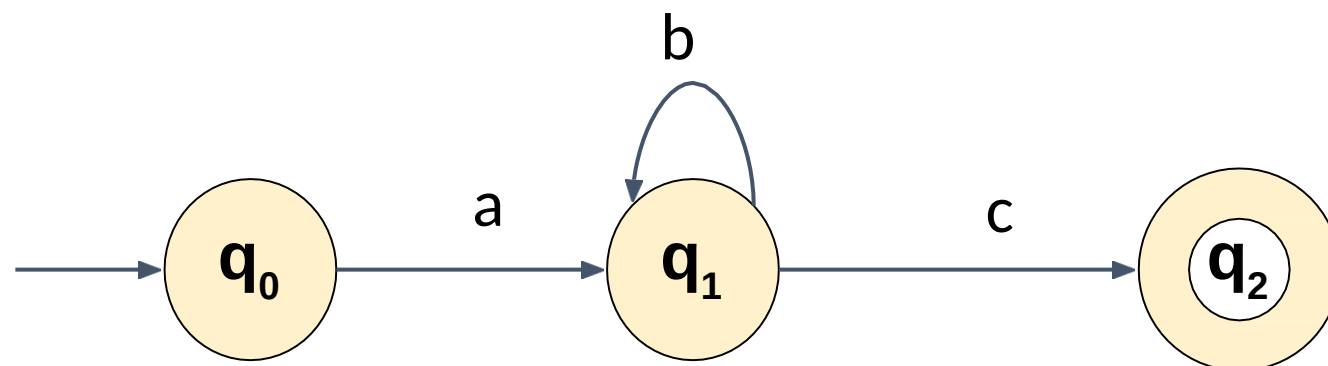
w = abbbc
|w| = 5 > n

w = abc
x = a
y = b
z = c

n = 3

for i ≥ 0,
abⁱc is in lang ab*c

In our example of infinite regular language ab^*c



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages (infinite)

Pumping Property

For every Regular language L ,

there exists n where n is the # states in Finite Automata for L

For every string w that belongs to L such that,

$$|w| \geq n$$

There exists a break up of the string in three parts $w = xyz$ such that $|y| \geq 1$ and $|xy| \leq n$,

for every $i \geq 0$,

xy^iz belongs to L

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages (infinite)

Pumping Property

For every Regular language L ,

there exists n where n is the # states in Finite Automata for L

For every string w that belongs to L such that,

$$|w| \geq n$$

There exists a break up of the string in three parts $w = xyz$ such that $|y| \geq 1$ and $|xy| \leq n$,

for every $i \geq 0$,

xy^iz belongs to L

Replace

For every $w \in L$ $\forall$

There exists $n \in \mathbb{N}$

$\exists$

belongs to L $\in$

$\in$

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages (infinite)

Pumping Property

- $\forall$ Regular language L ,
 - $\exists$ n where n is the # states in Finite Automata for L
 - $\forall$ string $w \in L$ such that,
 - $|w| \geq n$
 - $\exists w = xyz$ such that $|y| \geq 1$ and $|xy| \leq n$,
 - $\forall i \geq 0$,
 - $xy^iz \in L$

Replace

For every $\rightarrow \forall$

There exists $\rightarrow \exists$

belongs to $\rightarrow \in$

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages (infinite)

Pumping Property

- ∀ Regular language L,
- ∃ n where n is the # states in Finite Automata for L
- ∀ string $w \in L$ such that,
 $|w| \geq n$
- ∃ $w = xyz$ such that $|y| \geq 1$ and $|xy| \leq n$,

∀ $i \geq 0$,

$xy^iz \in L$

To Prove a lang is Non-Regular

~Pumping Property

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages (infinite)

Pumping Property

- Regular language L ,
- $\exists n$ where n is the # states in Finite Automata for L
- $\forall$ string $w \in L$ such that,
 - $|w| \geq n$
 - $\exists w = xyz$ such that $|y| \geq 1$ and $|xy| \leq n$,
 - $\forall i \geq 0$,
 - $xy^iz \in L$

$\sim \forall = \exists$
 $\sim \exists = \forall$

To Prove a lang is Non-Regular

$\sim$ Pumping Property

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages (infinite)

Pumping Property

- Regular language L,
- $\exists n$ where n is the # states in Finite Automata for L
- $\forall$ string $w \in L$ such that,
 - $|w| \geq n$
 - $\exists w = xyz$ such that $|y| \geq 1$ and $|xy| \leq n$,
 - $\forall i \geq 0$,
 - $xy^iz \in L$

To Prove a lang is Non-Regular

~Pumping Property

- $\exists$ a language L which is claimed to be regular,
- $\forall n$ where n is the # states in Finite Automata for L
- $\exists$ string $w \in L$ such that,
 - $|w| \geq n$
 - $\forall w = xyz$ such that $|y| \geq 1$ and $|xy| \leq n$,
 - $\exists i \geq 0$,
 - $xy^iz \notin L$

~
=
~
=

This **contradicts** the claim made, hence proving that the language is not regular

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages (infinite)

Pumping Property

- Regular language L ,
- $\exists n$ where n is the # states in Finite Automata for L
- $\forall$ string $w \in L$ such that,
 - $|w| \geq n$
 - $\exists w = xyz$ such that $|y| \geq 1$ and $|xy| \leq n$,
 - $\forall i \geq 0$,
 - $xy^iz \in L$

To Prove a lang is Non-Regular

$\sim$ Pumping Property

- $\exists$ a language L that is not regular,
- $\forall n$ where n is the # states in Finite Automata for L
- $\exists$ string $w \in L$ such that,
 - $|w| \geq n$
 - $\forall w = xyz$ such that $|y| \geq 1$ and $|xy| \leq n$,
 - $\exists i \geq 1$ such that $xy^iz \notin L$

Proof by

Contradiction

and it is not in L

$xy^iz \notin L$

This **contradicts** the claim made, hence proving that the language is not regular

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages (infinite)

Pumping Property

- $\forall$ Regular language L ,
 $\exists n$ where n is the # states in L
 $\forall$ string $w \in L$ such that $|w| \geq n$
 $\exists w = xyz$ such that $|y| \geq 1$ and $|xy| \leq n$,
 $\forall i \geq 0$,
 $xy^iz \in L$

**Pumping
Lemma**

To Prove a lang is Non-Regular

$\sim$ Pumping Property

- $\exists$ a language L that is not regular,
 $\forall n$ where n is the # states in L
 $\exists$ string $w \in L$ such that $|w| \geq n$
 $\forall w = xyz$ such that $|y| \geq 1$ and $|xy| \leq n$,
 $\exists i \geq 1$ such that $xy^iz \notin L$

Proof by

Contradiction

This **contradicts** the claim made, hence proving that the language is not regular

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

Procedure to prove a language is Not regular :

1. Assume the opposite: L is regular
2. Use Pumping Lemma to obtain a contradiction

It suffices to show that only one string gives a contradiction

3. Thereby proving L is not regular

Procedure to prove a language is Not regular :

1. Assume the opposite: L is regular
2. Use Pumping Lemma to obtain a contradiction

! String must be appropriately

It suffices to show that only one **string** gives a contradiction

3. Thereby proving L is not regular

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages



Pumping Property

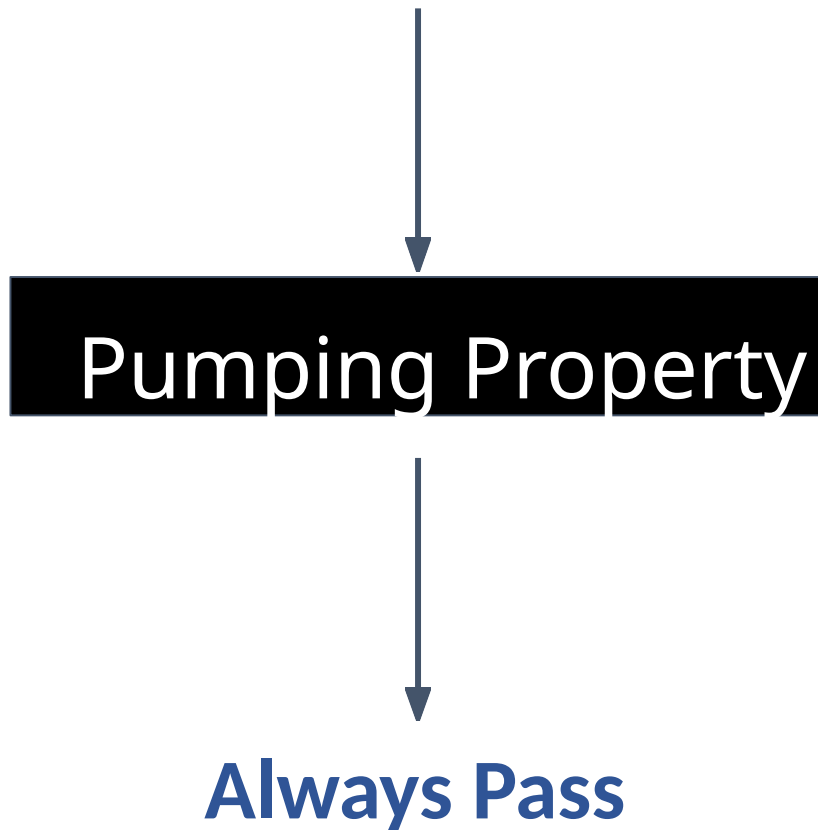


Always Pass

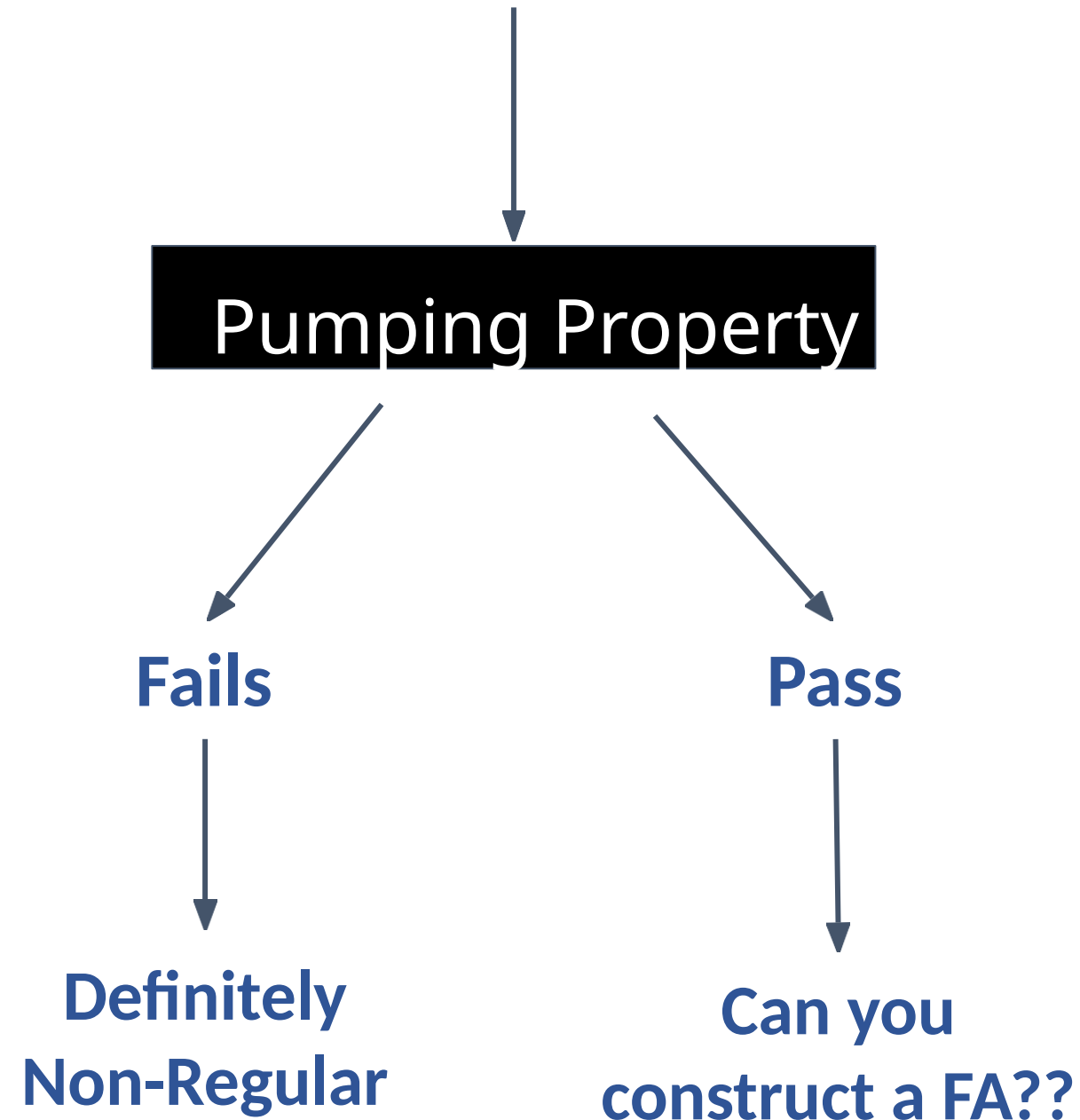
Automata Formal Languages and Logic

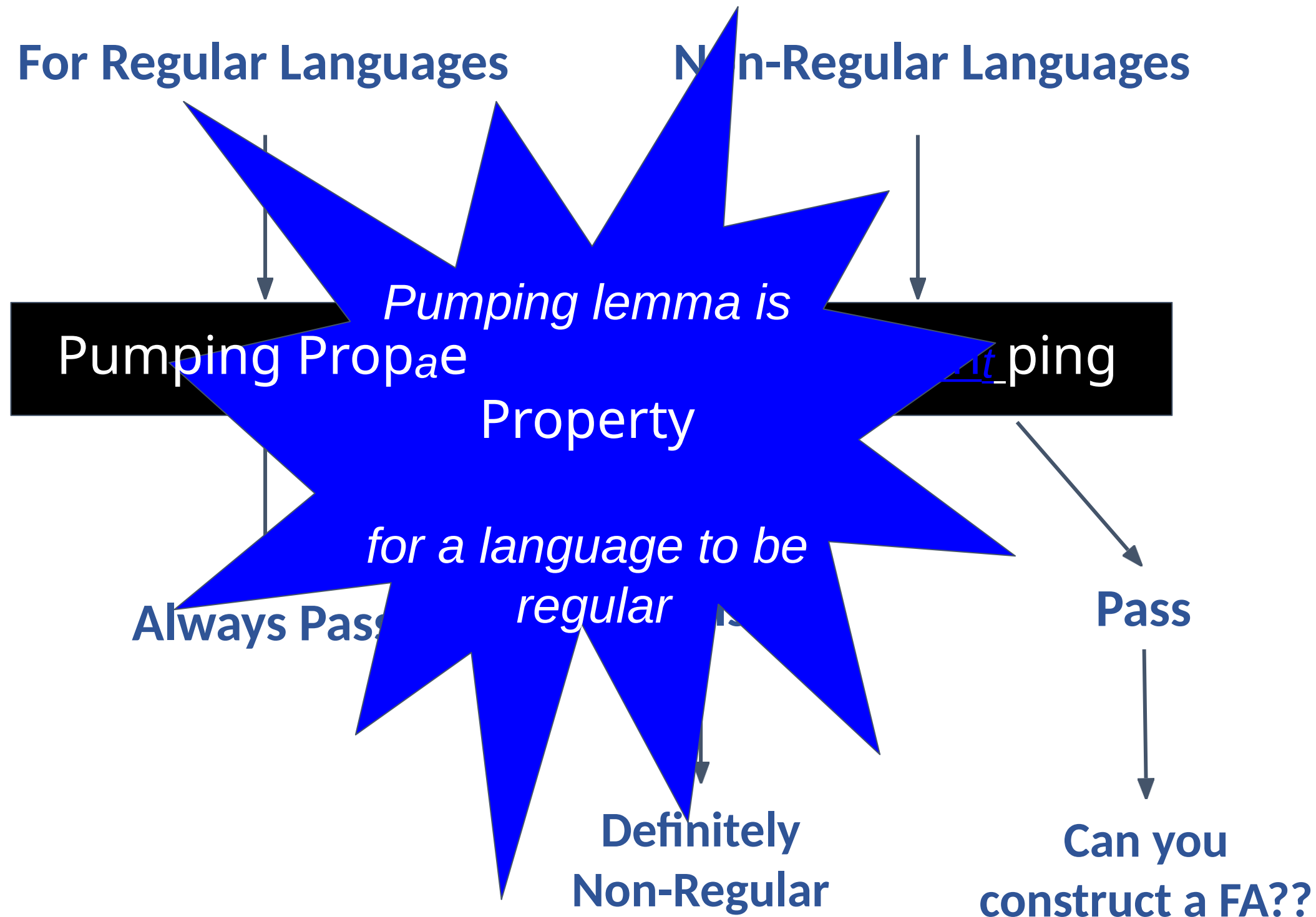
Unit 2 - Pumping Lemma for Regular Languages

For Regular Languages



Non-Regular Languages





Pumping lemma as a game



***Pumping lemma is a
game between***



You

Adversar

You
u

The
role



Adversary
y

Claims L is regular

Unit 2 - Pumping Lemma for Regular Languages

You

Okay! Give me the no. of
states in your machine for L

*The
role*



Adversar

y

Claims L is regular

Unit 2 - Pumping Lemma for Regular Languages

You

Okay! Gimme the no. of
states in your machine for L

***The
role***



Adversar

y

Claims L is regular
**There are n states in my
automata for L**

Unit 2 - Pumping Lemma for Regular Languages

You

Okay! Gimme the no. of states in your machine for L

Okay! here is the string w from L such that $|w| \geq n$

Could you tell me where is the loop in your machine?

**The
role**



Adversary

y

Claims L is regular.
There are n states in my automata for L

Unit 2 - Pumping Lemma for Regular Languages

You

Okay! Gimme the no. of states in your machine for L

Okay! here is the string w from L such that
 $|w| \geq n$

Could you tell me where is the loop in your machine?

**The
role**



Adversary

y

Claims L is regular.
There are n states in my automata for L

The loop is xy^iz

Unit 2 - Pumping Lemma for Regular Languages

You

Okay! Gimme the no. of states in your machine for L

Okay! here is the string w from L such that

$$|w| \geq n$$

Could you tell me where is the loop in your machine?

Find some i, so that the resultant string is not in L

**The
role**



Adversar

y

**Claims L is regular.
The are n states in my automata for L**

The loop is xy^iz

Unit 2 - Pumping Lemma for Regular Languages

You

Okay! Gimme the no. of states in your machine for L

Okay! here is the string w from L such that

$$|w| \geq n$$

Could you tell me where is the loop in your machine?

Find some i, so that the resultant string is not in L

**The
role**



Adversar

y

Claims L is regular.
The are n states in my automata for L

The loop is xy^iz

Oh no! I lost!!!!
L is not regular



Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

You

*The
role*

Adversar

Okay! Gimme the n states in your machine

Okay! here is the string w from L such that $|w| \geq n$

Could you tell me what is the loop in your machine?

Okay! but for some i , the resultant string is not in L

YOU WON!
GAME OVER

claims L is regular
there are n states in my automata for L

The loop is xy^iz

Oh no! I lost!!!!
 L is not regular



*Using Pumping lemma prove
that the language $a^n b^n$ is
not reg*



You
u

The
role



Adversary

Claims $L = a^n b^n$ is regular

Unit 2 - Pumping Lemma for Regular Languages

You

Okay! Gimme the no. of states in your machine for L

*The
role*



Adversary

Claims $L = a^n b^n$ is regular

Unit 2 - Pumping Lemma for Regular Languages

You

Okay! Gimme the no. of states in your machine for L

*The
role*



Adversary

Claims $L = a^n b^n$ is regular

There are 10 states in my automata for L

You

Okay! Gimme the no. of states in your machine for L

okay! I'll choose the string

a^6b^6

$|w| \geq 10$

Now tell me where is the loop in your automata?

*The
role*



Adversary

Claims $L = a^n b^n$ is regular

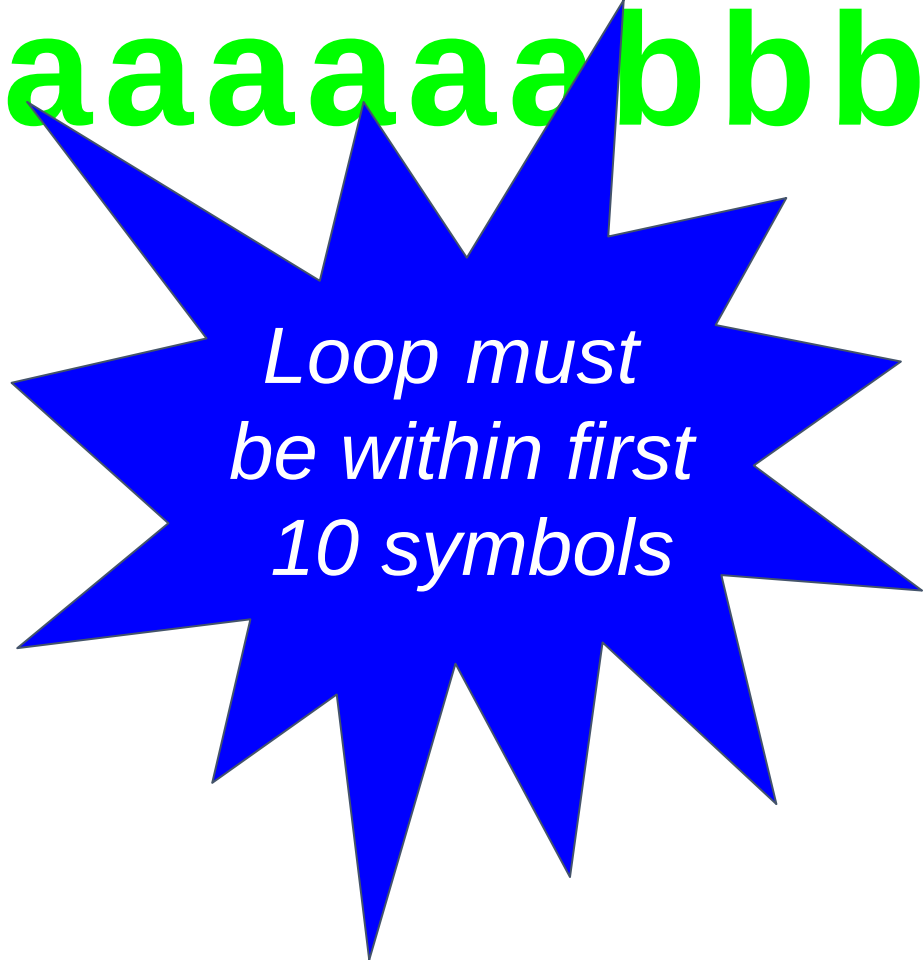
There are 10 states in my automata for L

*Where is the
bop ?*

aaaaaabbbbbbb

*Where is the
loop ?*

aaaaaaaaabbbbbbb

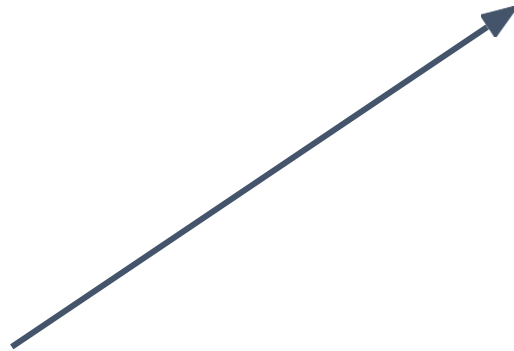


*Loop must
be within first
10 symbols*

Where is the loop ?

aaaaabbbbbbb

$a^5(a)^ib^6$



Where is the bop ?

aaaaabbbb

Pump down, $i=0 \rightarrow a^5b^6 \notin L$

$a^5(a)^0b^6$

*Where is the
loop ?*

aaaaabbbb

$a^5(a)^2b^6$

Pump up, $i=2 \rightarrow a^7b^6 \notin L$
Pump down, $i=0 \rightarrow a^3b^6 \notin L$

*Where is the
loop?*

aaaaabbbb

$i = 1$ does not help!

$a^5(a)^i b^6$

Pump down, $i=0 \rightarrow a^5b^6 \notin L$

*Where is the
loop ?*

Pump up, $i=2 \rightarrow a^5ababb^5 \notin L$

*aaaaa**ab**bbbb**b*

$a^5(ab)^ib^5$

*Where is the
loop ?*

aaaaa**ab**bbbb

$i = 0$ or $i = 1$

Pump up, $i=2 \rightarrow a^5ababb^5 \in L$

$$a^5(ab)^ib^5$$

Where is the bop ?

aaaaaabbb**b**bb

Pump down, $i=0 \rightarrow a^6b^5 \notin L$

Pump up, $i=2 \rightarrow a^6b^7 \notin L$

$a^6(b)^ib^5$



Where is the bop ?

aaaaaabbb**b**bb

Pump down, $i=0 \rightarrow a^6b^5 \notin L$

Pump up, $i=2 \rightarrow a^6b^7 \notin L$

$i=1$ does not help !

$a^6(b)^ib^5$

Where is the bop ?

aaaaaabbbbbbb

For every break up
possible we got
some i that will result
in a string $\notin L$

You

Okay! Gimme the no. of states in your machine for L

okay! I'll choose the string

a^6b^6

$|w| \geq 10$

Now tell me where is the loop in your automata?

The role



Adversary

Claims $L = a^n b^n$ is regular

There are 10 states in my automata for L

We saw and explored different possibilities where the loop could be

Unit 2 - Pumping Lemma for Regular Languages

You

Okay! Gimme the no. of states in your machine for L

okay! I'll choose the string

a^6b^6

$|w| \geq 10$

Now tell me where is the loop in your automata?

Okay! but for some i , nothing worked out!

*The
role*



Adversary

Claims $L = a^n b^n$ is regular

There are 10 states in my automata for L

We saw and explored different possibilities where the loop could be

You

Okay! Gimme the no. of states in your machine for L

okay! I'll choose the string

a^6b^6

$|w| \geq 10$

Now tell me where is the loop in your automata?

Okay! but for some i , nothing worked out!

The role



Adversary

Claims $L = a^n b^n$ is regular

There are 10 states in my automata for L

We saw and explored different possibilities where the loop could be

Oh no! I lost!!!!
L is not regular



Unit 2 - Pumping Lemma for Regular Languages

You *The* *Adversary*

Okay! Gimme the number of states in your machine

okay! I'll choose the string

$$a^6b^6$$

$$|w| \geq 10$$

Now tell me where is the loop in your automata?

Okay! but for some i , nothing worked out!



...ns $L = a^n b^n$ is regular

...e are 10 states in my automata for L

...e saw and explored different possibilities where the loop could be

Oh no! I lost!!!!
 L is not regular



*Using Pumping lemma prove that the language of the form **www** over $\{a,b\}^*$ is not regular*



You
u

The
role



Adversary

Claims $L = ww$ is regular

You

**Okay! Gimme the no. of
states in your machine for L**

***The
role***



Adversary

Claims $L = ww$ is regular

You

Okay! Gimme the no. of states in your machine for L

***The
role***



Adversary

Claims $L = ww$ is regular

There are n states in my automata for L

You

Okay! Gimme the no. of states in your machine for L

okay! I'll choose the string

$a^n a^n$

$|w| \geq n$

Now tell me where is the loop in your automata?

**The
role**



Adversary

Claims $L = ww$ is regular

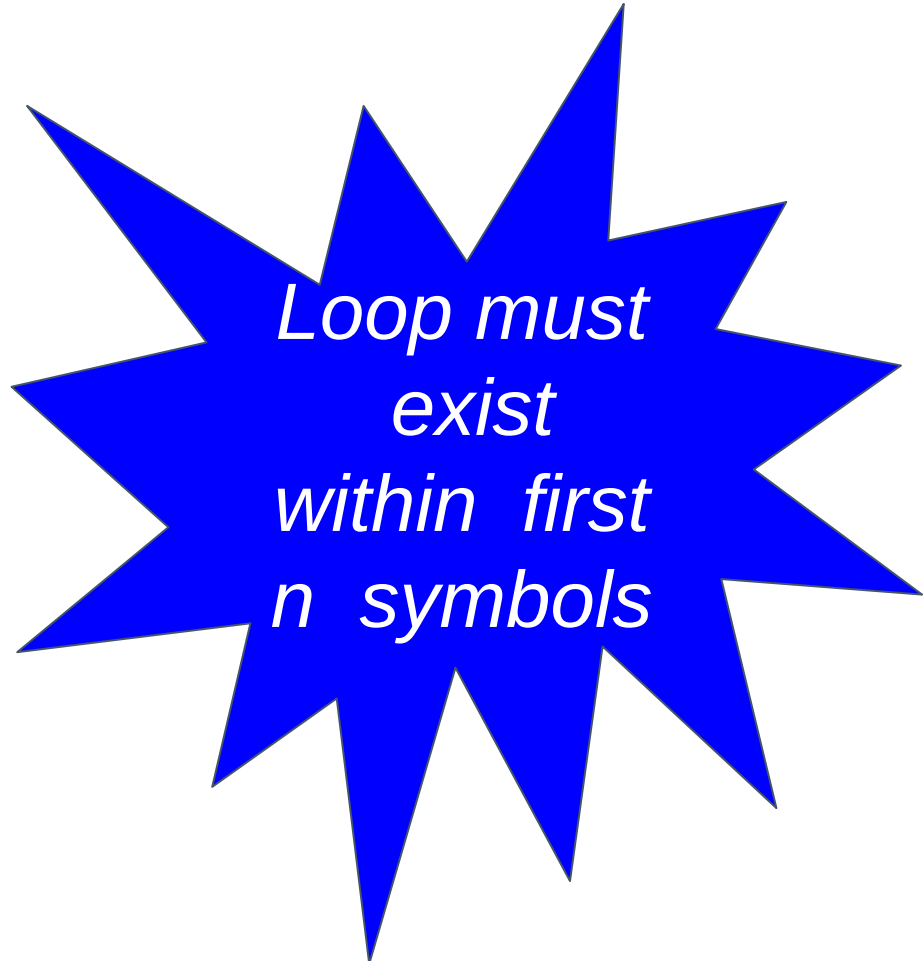
There are n states in my automata for L

Where is the bop ?

a...a..aa..aa...a..aa..a

Where is the bop ?

a...a.aa..aa...a.aa..a

A large, multi-pointed blue starburst graphic with a jagged, sunburst-like edge.

*Loop must
exist
within first
n symbols*

Where is the loop ?

a...a..aa..aa...

a..aa..a

$a^{n-2}(aa)^ia^n$

Where is the bop ?

a...a..aa..aa...a..aa..a

$$a^{n-2}(aa)^i a^n$$

Let's Pump down, $i=0$

$$= a^{n-2}a^n$$

$$= a^{n-2}a^2a^{n-2}$$

$$= a^{n-1}a^{n-1}$$

Where is the loop ?

a...a..aa..aa...a..aa..a

$$a^{n-2}(aa)^i a^n$$

Let's Pump up, $i=3$

$$= a^{n-2}(aa)^3 a^n$$

$$= a^{n-2}(a^2)^3 a^n$$

$$= a^{n-2}a^6 a^n$$

$$= a^{n-2}a^4 a^2 a^n$$

$$= a^{n-2+4}a^{n+2}$$

Where is the loop ?

a...a..aa..aa...a..aa..a

$a^{n-2}(aa)^i a^n$

Pump up or Pump down,
resultant string will always belong to
L

Where is the bop ?

$a^{n-2}(aa)^i a^n$ **YOU LOSE!** $a \dots a$

Pump up or Pump down,
resultant string will always belong to
L

Where is the

ba

a...

YOU LOSE!

la..a

aⁿ

2/ You chose a wrong string
Pump up or Pump down,
resultant string will always belong to L

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

Multiple Choice Questions

1. If for any decomposition of a string w from a regular language L (with $|w| \geq n$), there always exists an $i \geq 0$ such that $xy^iz \notin L$, what can be concluded?
 - a) L is regular
 - b) L must be infinite
 - c) L cannot be regular
 - d) L is finite

2. In the context of the **Pumping Lemma**, what does the **Pumping Constant (n)** represent?
 - a) The length of the shortest string in the language
 - b) The number of accept states in the automaton
 - c) The number of states in the finite automaton for the language
 - d) The minimum number of transitions in any DFA

Automata Formal Languages and Logic

Unit 2 - Pumping Lemma for Regular Languages

Answers

1. C
2. C



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 3

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages and Logic

Unit 3 - Context Free Grammar Definition

Grammar

V -
Variables

expr, var, S, A, B..., w,x,y

T -
Terminals

a, b, c ..., 0, 1, ... id, num

P - Production
Rules

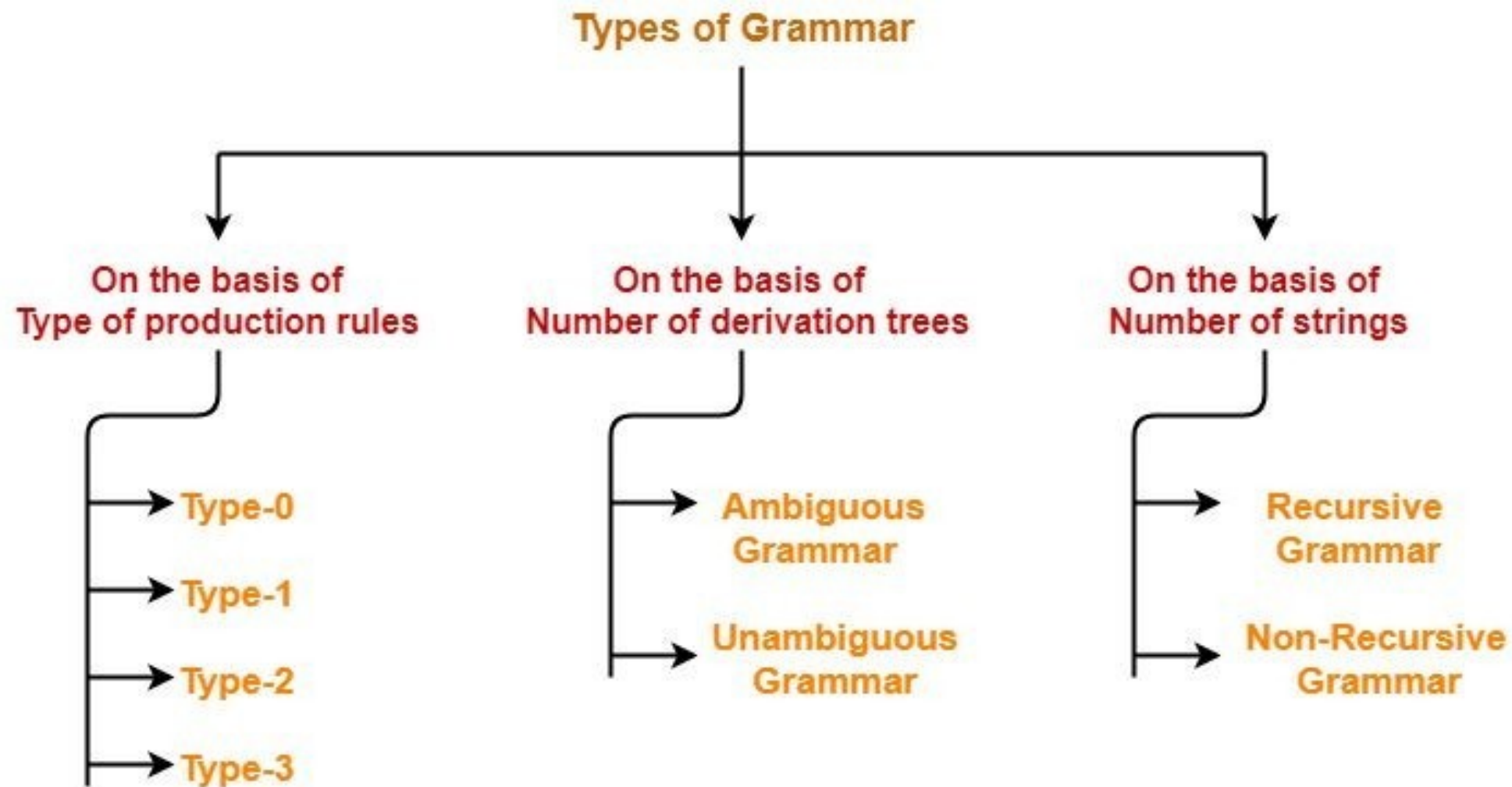
$\alpha = (V \cup T)^*$
 $A \rightarrow \alpha$

S - Start
Symbol

$L(G) =$

$\{w \mid S \Rightarrow w, \text{ where } w \in \Sigma^*\}$

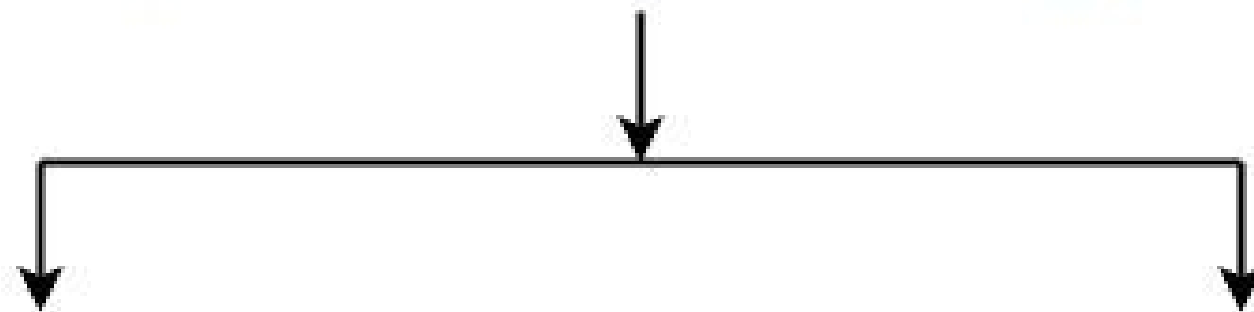
Unit 3 - Context Free Grammars



(Chomsky Hierarchy)

Types of Grammar

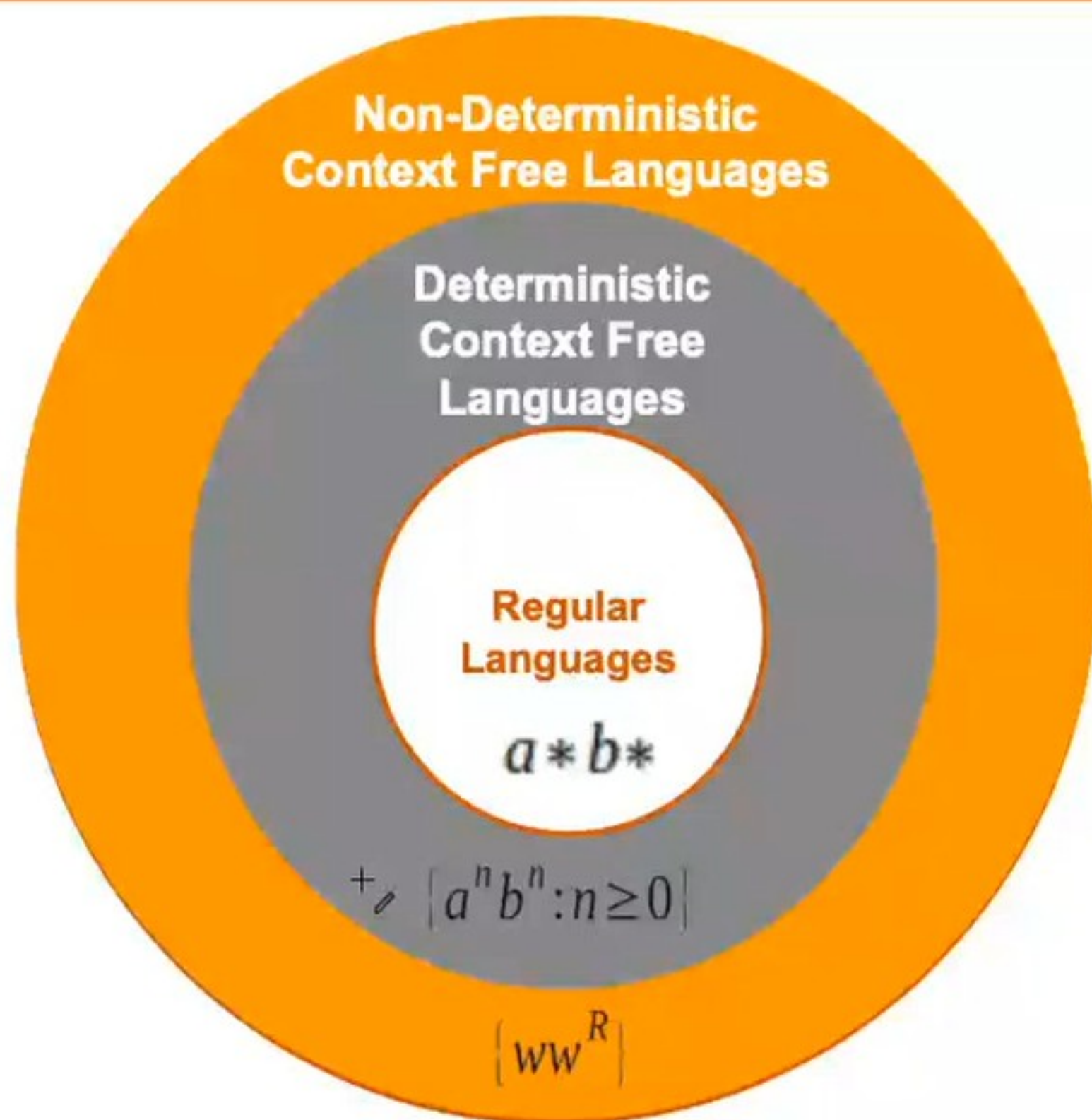
(On the basis of Number of Strings)



Recursive Grammar

Non-Recursive Grammar





Unit 3 - Context Free Grammars

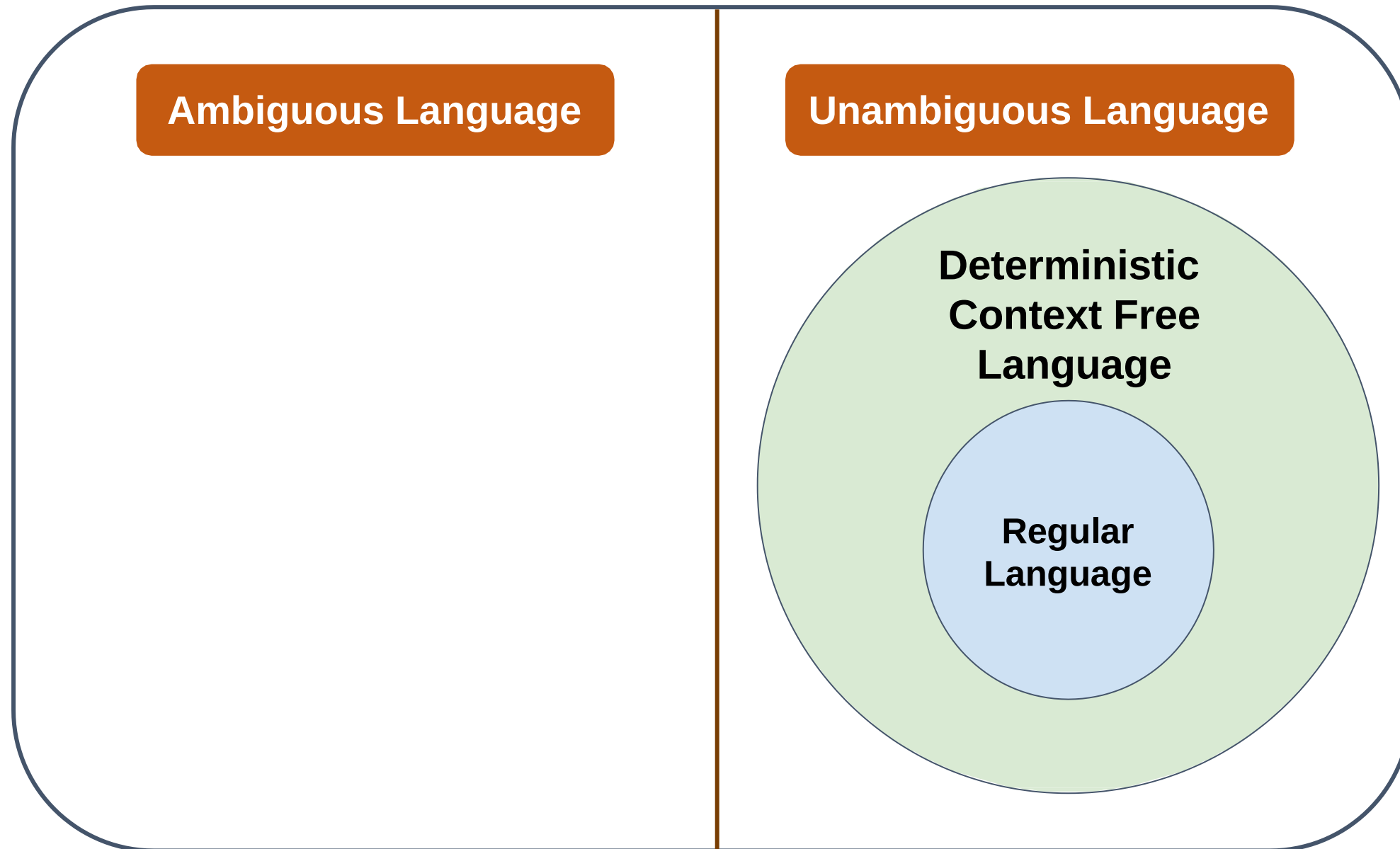
Context Free Language

Ambiguous Language

Unambiguous Language

**Deterministic
Context Free
Language**

**Regular
Language**



Example 1:

Construct linear grammar for the even palindromes. $L = \{ww^R, w \in \{a,b\}^*\}$

Solution :

$S \rightarrow aSa \mid bSb \mid \lambda$

Example 2:

Construct linear grammar for the even palindromes. $L = \{wCw^R, w \in \{a,b\}^*\}$

Solution :

$S \rightarrow aSa \mid bSb \mid C$

Example 3:

Construct linear grammar for $L = \{ww^R, w \in \{ab\}^* \mid (ba)^*\}$

Solution :

$S \rightarrow abSba \mid baSab \mid \lambda$

Example 4:

Construct linear grammar for $L = \{a^n w w^R b^n \mid w \in \{a, b\}^*\}$

Solution :

$S \rightarrow aSb \mid A$

$A \rightarrow aAa \mid bAb \mid \lambda$

Change the conditions to w belongs to $\{a, b\}^+$ and $n \geq 1$

Example 5:

Construct linear grammar for $L = \{a^n b^{n+1}, n \geq 0\}$

Solution :

$S \rightarrow aSb \mid b$

Example 6:

Construct linear grammar for $L=\{a^{n+2}b^n, n \geq 1\}$

Solution :

$S \rightarrow aSb \mid aaab$

$S \rightarrow aSb \mid aa \quad n \geq 0$

Example 7:

Construct linear grammar for $L=\{a^n b^{2n}, n \geq 0\}$

Solution :

$S \rightarrow aSbb \mid \lambda$

Example 8:

Construct linear grammar for $L=\{a^n b^{n-3}, n \geq 3\}$

Solution :

$S \rightarrow aSb \mid aaa$

Example 9:

Construct linear grammar for $L=\{a^n b^m, n>m\}$

Solution :

$S \rightarrow aSb \mid aS \mid a$

Example 10:

Construct linear grammar for $L = \{a^n b^m, n \neq m\}$

Solution :

$S \rightarrow A \mid B$

$A \rightarrow aAb \mid aA \mid a$

$B \rightarrow aBb \mid \textcolor{red}{Bb} \mid b$

Example 11:

Construct linear grammar for $L = \{a^n b^m, n = 2 + (m \bmod 3)\}$

Solution :

$S \rightarrow aaA \mid aaabA \mid aaaabbA$

$A \rightarrow bbbA \mid \lambda$

Example 12:

Construct linear grammar for $L = \{a^n b^m, n \neq 2m\}$

Solution :

$S \rightarrow aaSb \mid A \mid B \mid aC$

$A \rightarrow aA \mid a$

$B \rightarrow Bb \mid b$

$C \rightarrow Cb \mid$

λ

Example 13:

Construct linear grammar for $L = \{a^{n+2}b^m, m > n, n \geq 0\}$

Solution :

$S \rightarrow aSb \mid aab \mid Sb$

$S \rightarrow a S b \mid aabA$

$A \rightarrow Ab \mid \lambda$

Example 14:

Construct linear grammar for $L = \{a^n b^m c^m d^n, n, m \geq 1\}$

Solution :

$S \rightarrow aSd \mid aAd$

$A \rightarrow bAc \mid bc$

Example 15:

Construct linear grammar for $L = \{a^n b^m c^k, k = n + m, n, m, k \geq 0\}$

Solution :

$S \rightarrow aSc \mid A$

$A \rightarrow bAc \mid ?$
 $?$

Example 16:

Construct linear grammar for $L = \{a^n b^m c^k, m = 2n, k = 2, n \geq 0\}$

Solution :

$S \rightarrow AB$

$A \rightarrow aAbb \mid \diamond ?$

$B \rightarrow cc \mid \diamond ?$

Hence non linear

Example 17:

Construct linear grammar for $L = \{a^n b^m c^k, m, n \geq 0, k = n + 2m\}$

Solution :

$S \rightarrow aSc \mid A$

$A \rightarrow bAcc \mid ?$
 $?$

Example 18:

Construct linear grammar for $L = \{ |w| \bmod 3 \neq |w| \bmod 2, w \in \{a\}^* \}$

Solution :

$S \rightarrow aaaaaaS \mid aa \mid aaa \mid aaaa \mid aaaaa$



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 2

Preet Kanwal

Department of Computer Science & Engineering

Unit 2 - Context Free Grammars

Example 1:

Construct CFG for $L = \{uvw^R, |u| = |w| = 2, |v| \geq 1, w \in \{a,b\}^*\}$

Solution :

$S \rightarrow A B$

$A \rightarrow aa | bb | ab | ba$

$B \rightarrow aBa | bBb | aAa | bAb$

Unit 2 - Context Free Grammars

Example 2:

Construct a CFG for $L = \{w \mid n_a(w) = n_b(w), w \in \{a,b\}^*\}$

Solution :

$S \rightarrow aSb \mid bSa \mid \lambda \mid SS$

Unit 2 - Context Free Grammars

Example 3:

Construct a CFG for $L = \{n_a(w) = n_b(w) + 1, w \in \{a,b\}^*\}$

Solution :

$S \rightarrow AaA$

$A \rightarrow aAb \mid bAa \mid AA \mid \lambda$

or

$S \rightarrow aSb \mid bSa \mid abS \mid baS \mid a$

Automata Formal Languages and Logic

Unit 2 - Context Free Grammars

Example 4:

Construct a CFG for $L = \{w \mid n_a(w) = 2 \times n_b(w), w \in \{a,b\}^*\}$

Solution :

$S \rightarrow aSaSb \mid bSaSa \mid aSbSa \mid SS \mid \lambda$

Automata Formal Languages and Logic

Unit 2 - Context Free Grammars

Example 5:

Construct a CFG for $L = \{n_a(w) > n_b(w), w \in \{a,b\}^*\}$

Solution :

$S \rightarrow AaA$

$A \rightarrow aAb \mid bAa \mid AA \mid aA \mid Aa \mid \lambda$

Automata Formal Languages and Logic

Unit 2 - Context Free Grammars

Example 6:

Construct a CFG for $L = \{n_a(w) \neq n_b(w), w \in \{a,b\}^*\}$

Solution :

$S \rightarrow AaA \mid BbB$

$A \rightarrow aAb \mid bAa \mid Aa \mid aA \mid \lambda$

$B \rightarrow aBb \mid bBa \mid Bb \mid bB \mid \lambda$

Unit 2 - Context Free Grammars

Example 7:

Construct a CFG for $L = \{a^n b^n \cup a^n b^{2n}\}$.

Solution :

$S \rightarrow S1 \quad |$

$S2$

$S1 \rightarrow aS1b \mid \lambda$

$S2 \rightarrow aS2bb$

$\mid \lambda$

Unit 2 - Context Free Grammars

Example 8:

**Construct /a CFG for Language of proper nesting(parenthesis matching)
where $\Sigma = \{ (,) \}$.**

Solution :

$S \rightarrow (S) \mid$

SS

$S \rightarrow \lambda$

Unit 2 - Context Free Grammars

Example 9:

**Construct /a CFG for Language of proper nesting(parenthesis matching)
where $\Sigma = \{(\{,[\ ,\},\},)\}$.**

Solution : $S \rightarrow (S) \mid$

$\{S\} \mid [S] \mid SS \mid \lambda$

Automata Formal Languages and Logic

Unit 2 - Context Free Grammars

Example 10:

Construct a CFG for Language to generate arithmetic expressions.

Solution :

$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid id \mid number$

Unit 2 - Context Free Grammars

Example 11:

Construct a CFG for nested if else.

Solution :

$S \rightarrow \text{if condition then } S$

$S \rightarrow \text{if condition then } S \text{ else}$

$S \ S \rightarrow \{ \text{statement} \}$

Unit 2 - Context Free Grammars

Example 12:

Construct a CFG to take care of variable declarations in C Language.

Solution :

$D \rightarrow \text{Type List}$

$\text{List} \rightarrow \text{List}, \text{id} \mid$

id

$\text{Type} \rightarrow \text{int} \mid$

$\text{float} \mid \text{char}$

Unit 2 - Context Free Grammars

Example 13:

Construct a CFG to generate nested while loops.

Solution :

$S \rightarrow \text{while}(\text{condition})S \mid \{\text{statement}\}$

while loop:

$S \rightarrow \text{while}(\text{condition})S \mid \text{do } S \text{ while}(\text{condition}) \mid \{\text{statement}\}$



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Unit 2 – Pushdown Automata (PDA)

Prakash C O and Preet Kanwal

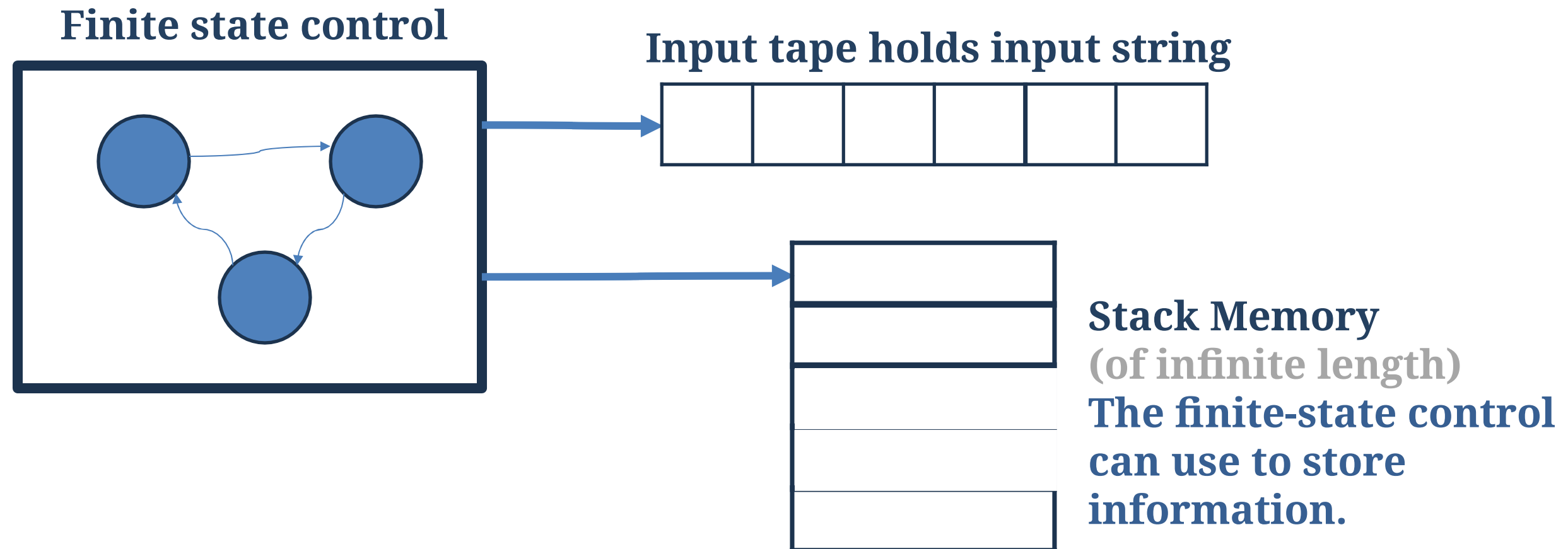
Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 2: Deterministic Pushdown Automata (DPDA)

Preet Kanwal and Prakash C O

Department of Computer Science & Engineering



PDA = Finite Automata + Stack (of infinite length)

- **A pushdown automaton (PDA) is a finite automaton equipped with a stack based memory.**
- PDAs are more capable than finite-state machines but less capable than Turing machines.
- **A pushdown automaton (PDA) differs from a finite state machine in two ways:**
 - It can use the top of the stack to decide which transition to take.
 - It can manipulate the stack as part of performing a transition.

Formal Definition of DPDA

A pushdown automaton P is a seven-tuple $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

where,

- Q is a finite set of states,
- Σ is a finite set of input symbols,
- Γ is a finite set of stack symbols,
- δ is a transition function, $Q \times (\Sigma \cup \{\}) \times \Gamma \rightarrow Q \times \Gamma^*$
- $q_0 \in Q$ is the start/initial state,
- $Z_0 \in \Gamma$, Initially, the stack holds a special symbol Z_0 that indicates the bottom of the stack.
- $F \subseteq Q$ is a set of accepting states

Note: In the above equation, Γ^* represents stack new top symbols

- **A pushdown automaton reads a given input string from left to right. In each step, it chooses a transition by considering**
 - the current state,
 - the next input symbol, and
 - the symbol at the top of the stack.

- **A pushdown automaton can also manipulate the stack, as part of performing a transition.**

The manipulation can be

- to push a particular symbol to the top of the stack, or
- to pop off the top of the stack.

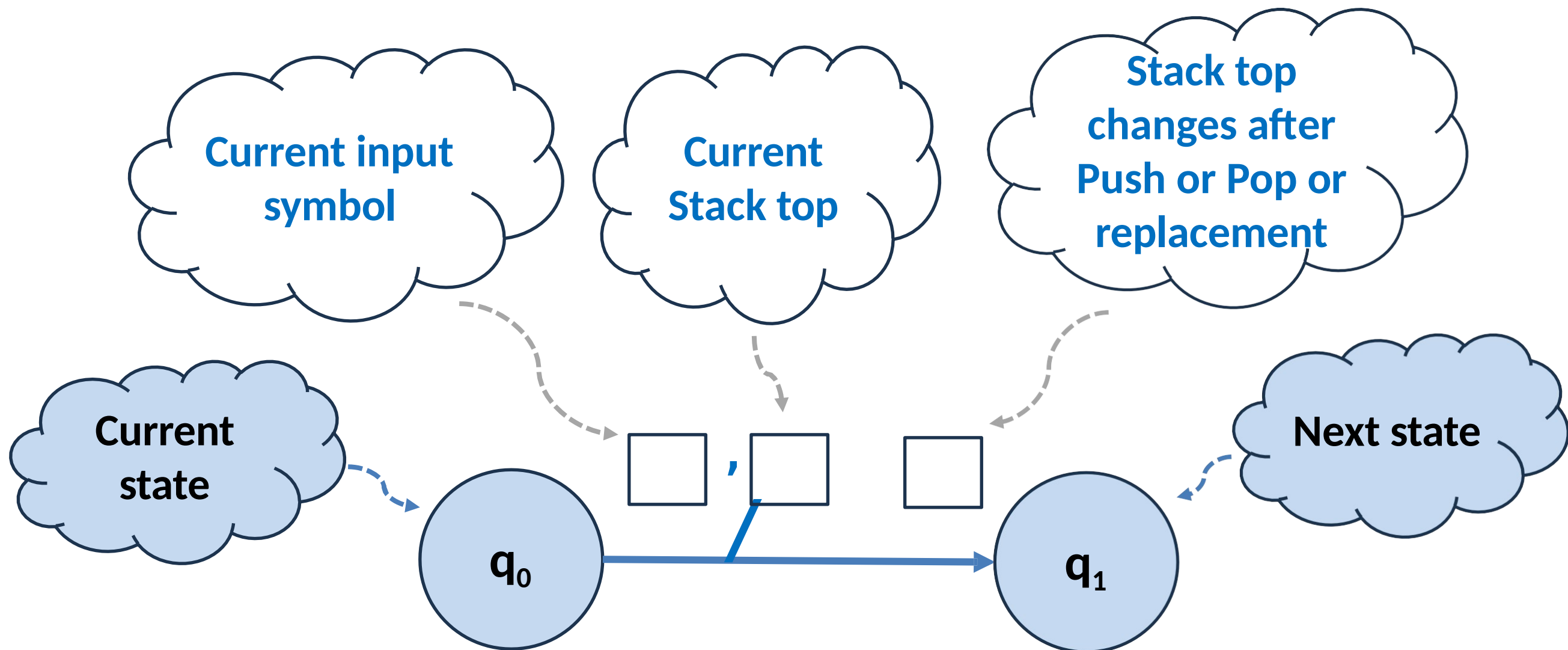
The automaton can alternatively ignore the stack, and leave it as it is.

- **Put together: Given an input symbol, current state, and stack symbol, the automaton can follow a transition to another state, and optionally manipulate (push or pop) the stack.**

- If, in every situation of the PDA, at most one transition action is possible, then the automaton is called a **deterministic pushdown automaton (DPDA)**.
- In general, if several actions are possible, then the automaton is called **a general,**
or
nondeterministic pushdown automaton (NPDA).
- **Deterministic pushdown automata(DPDA) can recognize all deterministic context-free languages (DCFLs) while nondeterministic PDAs can recognize all context-free languages,** with the former often used in parser design.
- A language that can be recognized by a DPDA is called a **deterministic context-free language.**

PDA as a state diagram

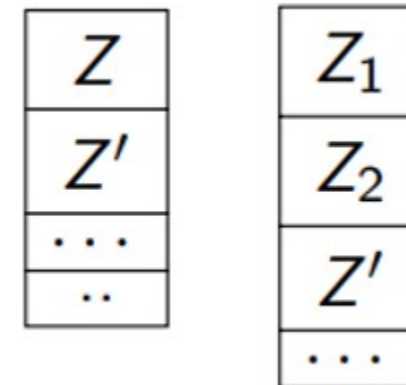
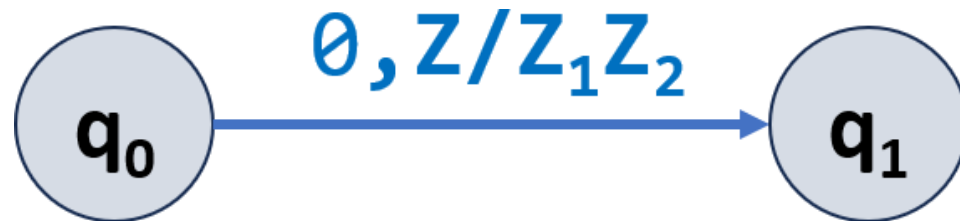
Transitions in Pushdown Automaton



Transitions in Pushdown Automaton

Let $\Sigma = \{0, 1\}$, $\Gamma = \{Z_0, Z_1, Z_2, \dots\}$ and $\delta(q_0, 0, Z) = (q_1, Z_1Z_2)$

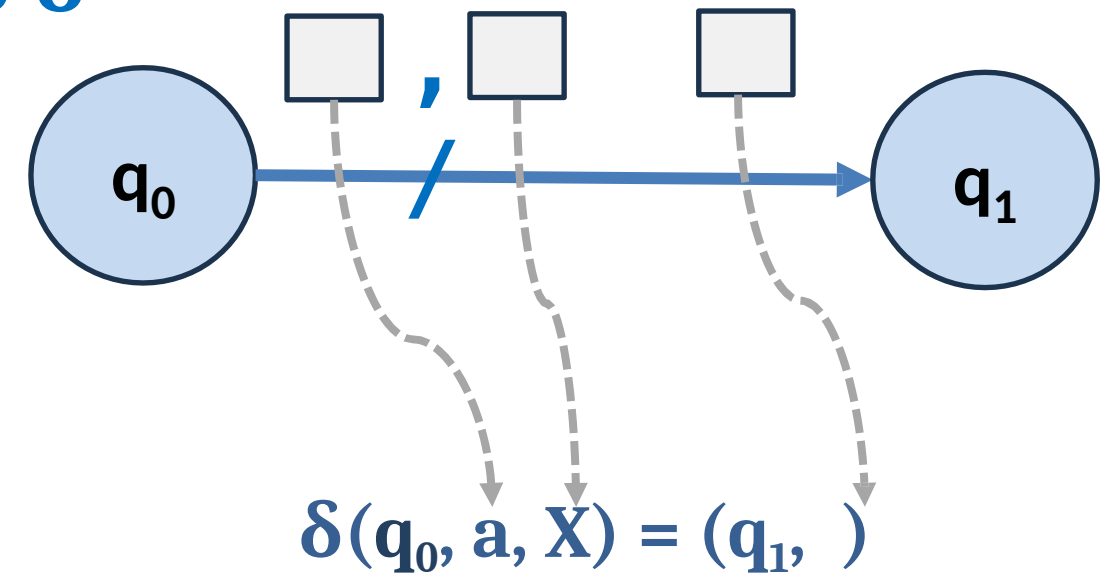
We draw the PDA transitions as follows



- To execute the transition, top symbol of the stack must be Z
- After transition Z will be popped and word Z_1Z_2 will be pushed on to the stack

Understanding stack interaction due to δ

- State transition from q_0 to q_1
- a is the current input symbol
- X is the current stack *top* symbol



- Popping X from stack:

$$\delta(q_0, a, X) = (q_1,)$$

- Stack top is unchanged:

$$\delta(q_0, a, X) = (q_1, X)$$

- Pop X and Push Y :

$$\delta(q_0, a, X) = (q_1, Y)$$

- Pushing Y in stack:

$$\delta(q_0, a, X) = (q_1, YX)$$

- Pop X and Push $Y_1Y_2...Y_k$:

$$\delta(q_0, a, X) = (q_1, Y_1Y_2...Y_k)$$

- Pushing $Y_1Y_2...Y_k$ in stack:

$$\delta(q_0, a, X) = (q_1, Y_1Y_2...Y_kX)$$

Transition Label forms

1) A transition label of the form: $\epsilon, \epsilon / \epsilon$ or $\lambda, \lambda / \lambda$

This transition means don't consume any input, don't consider stack top symbol, and don't pop anything from a stack, or don't add anything to a stack. It's the equivalent of an ϵ - transition in an NFA.

2) A transition label of the form: $a, b / c$

Means "If the current input symbol is a and the current stack symbol is b , then, pop b , and push c ."

3) A transition label of the form: $a, b / b$

Means "If the current input symbol is a and the current stack symbol is b , then, pop b , and push b (or No stack operation or No changes in stack)."

4) A transition label of the form: $a, b / \epsilon$ or $a, b / \lambda$

Means "If the current input symbol is a and the current stack symbol is b , then, pop b ."

Transition Label forms cont...

5) A transition label of the form: $a, Z / aaZ$

Means “If the current input symbol is a and the current stack symbol is Z, then, push 2 a’s.

6) A transition label of the form: $a, a / aaaa$

Means “If the current input symbol is a and the current stack symbol is a, then, push 3 a’s.

7) A transition label of the form: $a, b / aa$

Means “If the current input symbol is a and the current stack symbol is b, then, pop b and push 2 a’s.

8) A transition label of the form: $a, \epsilon / x$ or $a, \lambda / x$

Means “If the current input symbol is a (and not worried about stack top contents), it just pushes a new symbol x on to the stack.

A PDA configuration (or Instantaneous description (ID)) at any given instance is a triple: (q, w, γ)

q - current state

w - remainder of the input (i.e., unconsumed part)

γ - current stack contents as a string from top to bottom of stack

If $\delta(q, a, X) = (p, A)$ is a transition, then the following are also true:

$(q, a, X) \vdash (p, \epsilon, A)$  ID: A move from state q to p , input is empty after reading a and stack top symbol X is replaced by A .

$(q, aw, XB) \vdash (p, w, AB)$  ID: A move from state q to p , remaining input is w and stack top symbol X is replaced by A .

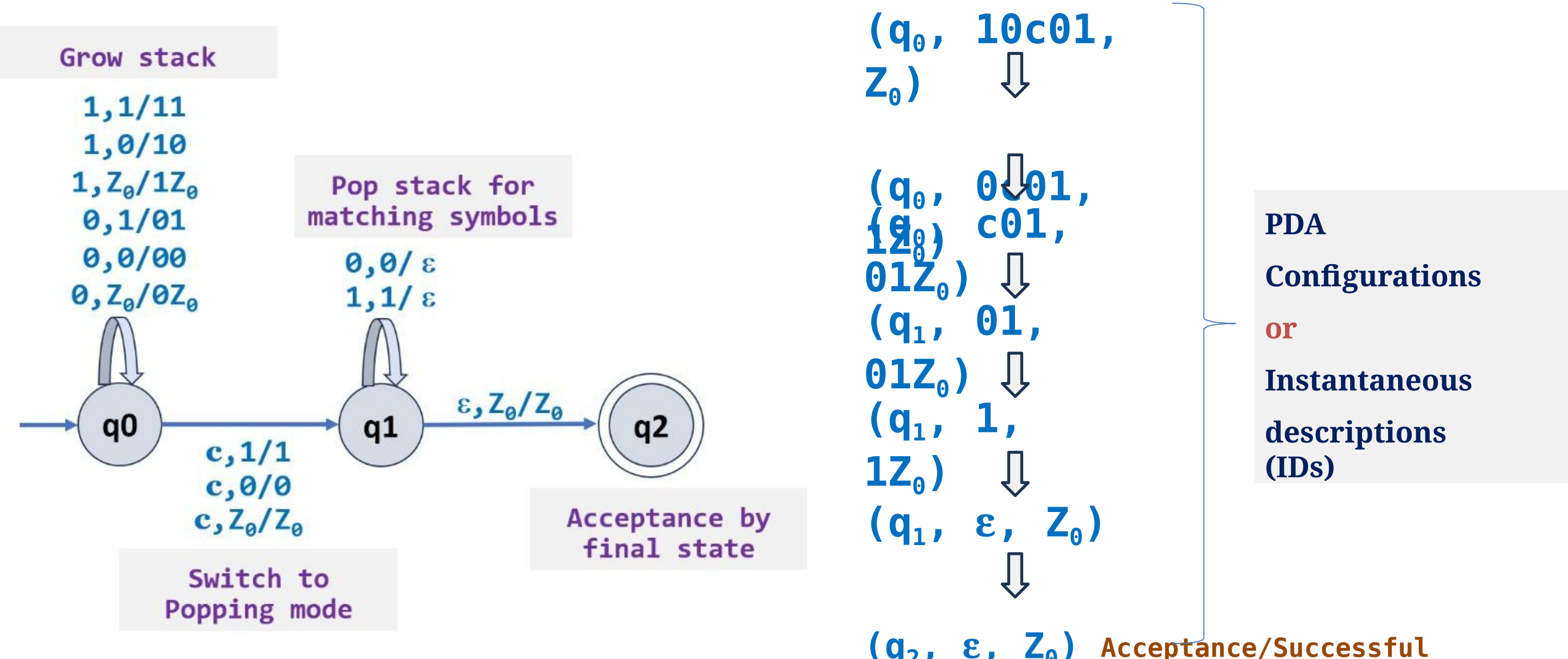
$\vdash$ sign is called a “turnstile notation” and represents one move

$\vdash^*$ sign represents a sequence of moves

1. D-PDA for $L = \{ wcw^R \mid c \text{ is some special symbol not in } w \text{ and } \Sigma=\{0, 1\} \}$

Solution :

How does the DPDA works on input 10c01 ?



Language of a PDA

- The common way to define the language of a PDA is **by final state**.
 - If P is a PDA, then $L(P)$ is the set of strings w such that $(q_0, w, Z_0) \vdash^* (f, \epsilon, Z_0)$ for final state f and any α .
- Another language defined by the same PDA is **by empty stack**.
 - If P is a PDA, then $N(P)$ is the set of strings w such that $(q_0, w, Z_0) \vdash^* (q, \epsilon, \epsilon)$ for any state q .

Final State Acceptability

A PDA is said to accept a string when it reaches its final state after completely reading the string. It is possible to transition from the initial state to a final state with a stack value.

The content of the stack doesn't matter as long as we arrive at a final state.

Empty Stack Acceptability

- The PDA accepts a string after it has emptied its stack after reading the entire string.
- The language accepted by the empty stack of a PDA P is $L(P) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \epsilon, \epsilon), q \in Q\}$

Deterministic PDA: Definition

A PDA is deterministic, if there is never a choice for a next move in any transition.

A PDA is deterministic if and only if:

- $\delta(q, a, X)$ has at most one member for any q in Q , a in $\Sigma \cup \{\epsilon\}$ and X in Γ .**
- If $\delta(q, a, X)$ is non-empty for some a in Σ , then $\delta(q, \epsilon, X)$ must be empty.**

A deterministic pushdown automaton is a PDA with the extra property that for each state in the PDA, and for any combination of a current input symbol and a current stack symbol, there is at most one transition defined.

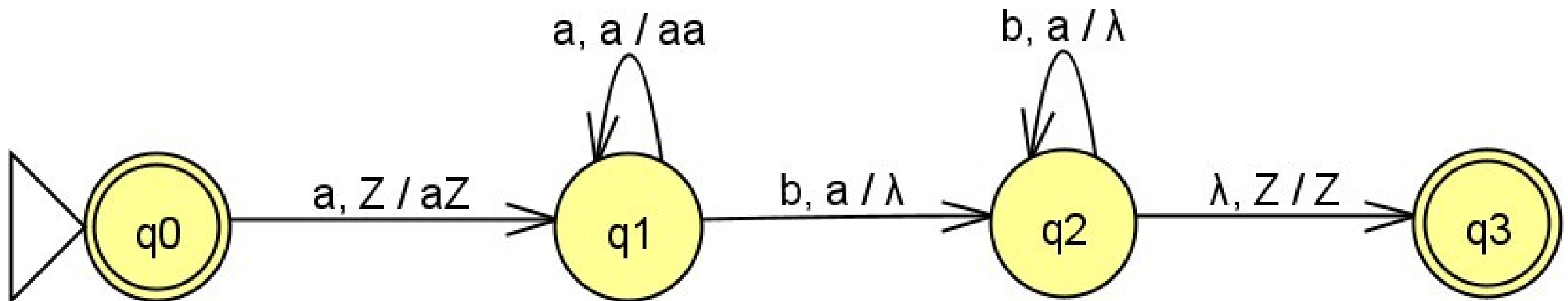
In other words, there is at most one legal sequence of transitions that can be followed for any input.

1. Construct a DPDA for the language $L = \{a^n b^n \mid n \geq 0\}$

Solution:

1. Construct a DPDA for the language $L = \{a^n b^n \mid n \geq 0\}$

Solution: $L = \{\lambda, ab, aabb, aaabbb, aaaabbbb, \dots\}$



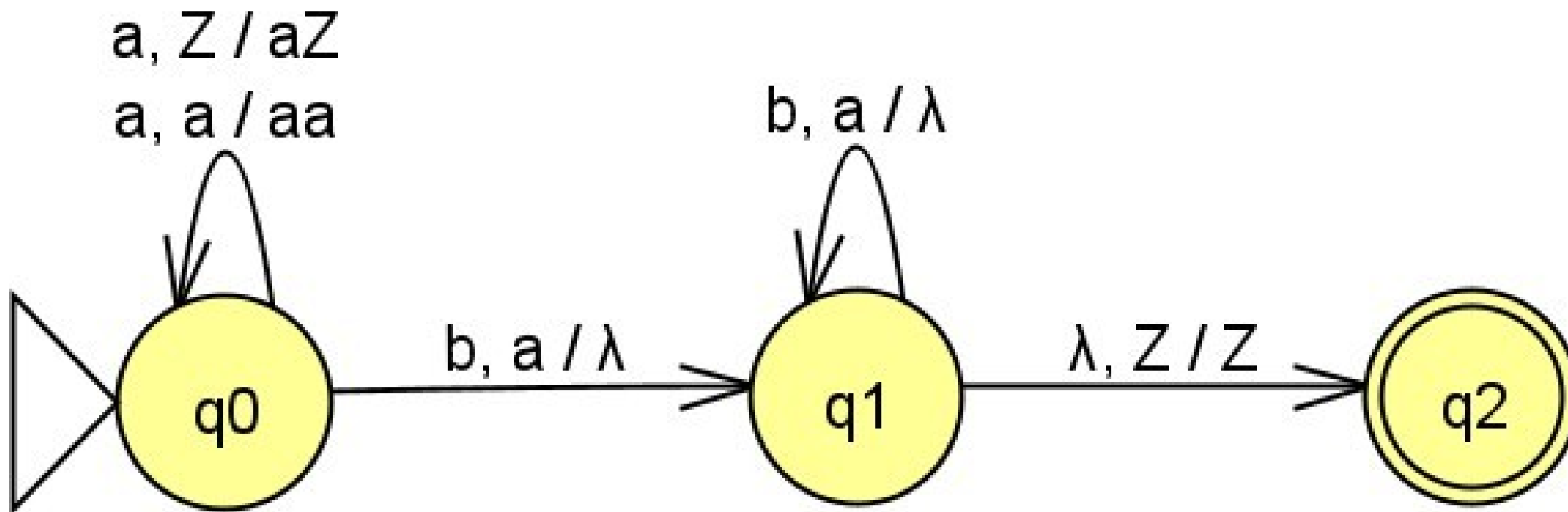
Note: Z is used instead of Z_0 to indicate the bottom of the stack.

1a. Construct a DPDA for the language $L = \{a^n b^n \mid n \geq 1\}$

Solution: $L = \{ ab, aabb, aaabbb, aaaabbbb, \dots \}$

1a. Construct a DPDA for the language $L = \{a^n b^n \mid n \geq 1\}$

Solution: $L = \{ ab, aabb, aaabbb, aaaabbbb, \dots \}$

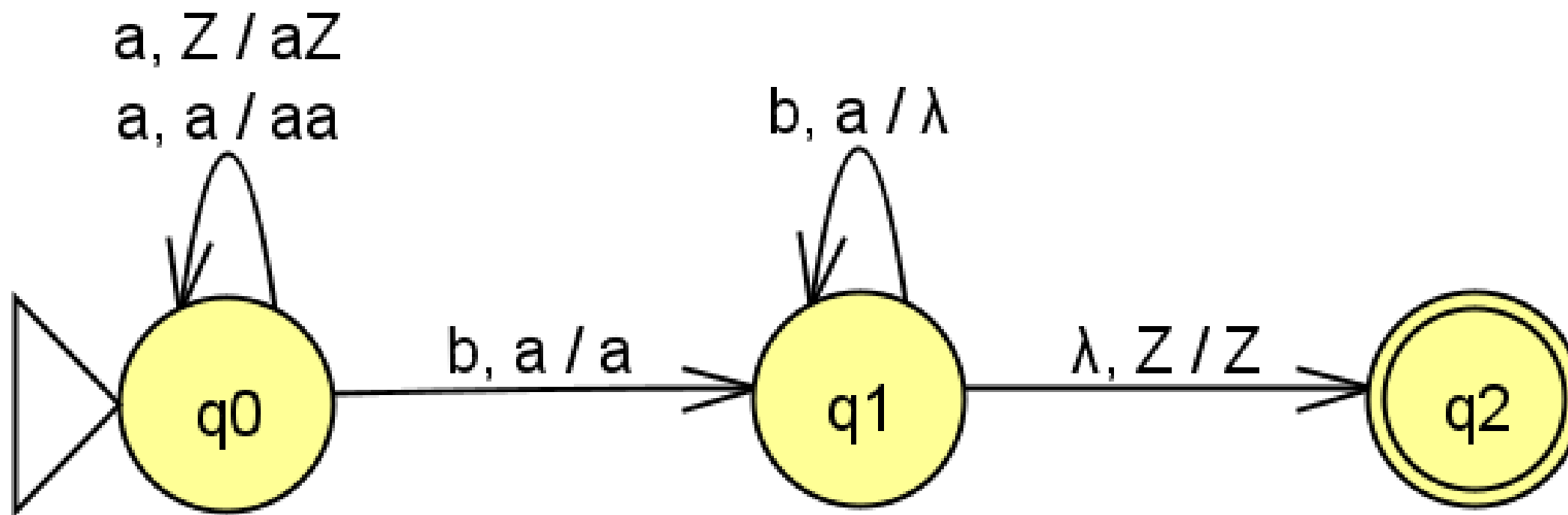


2. Construct a DPDA for the language $L = \{a^n b^{n+1} \mid n \geq 1\}$

Solution: $L = \{ abb, aabbb, aaabbbb, aaaabbbbb, \dots \}$

2. Construct a DPDA for the language $L = \{a^n b^{n+1} \mid n \geq 1\}$

Solution: $L = \{ abb, aabbb, aaabbbb, aaaabbbbb, \dots \}$

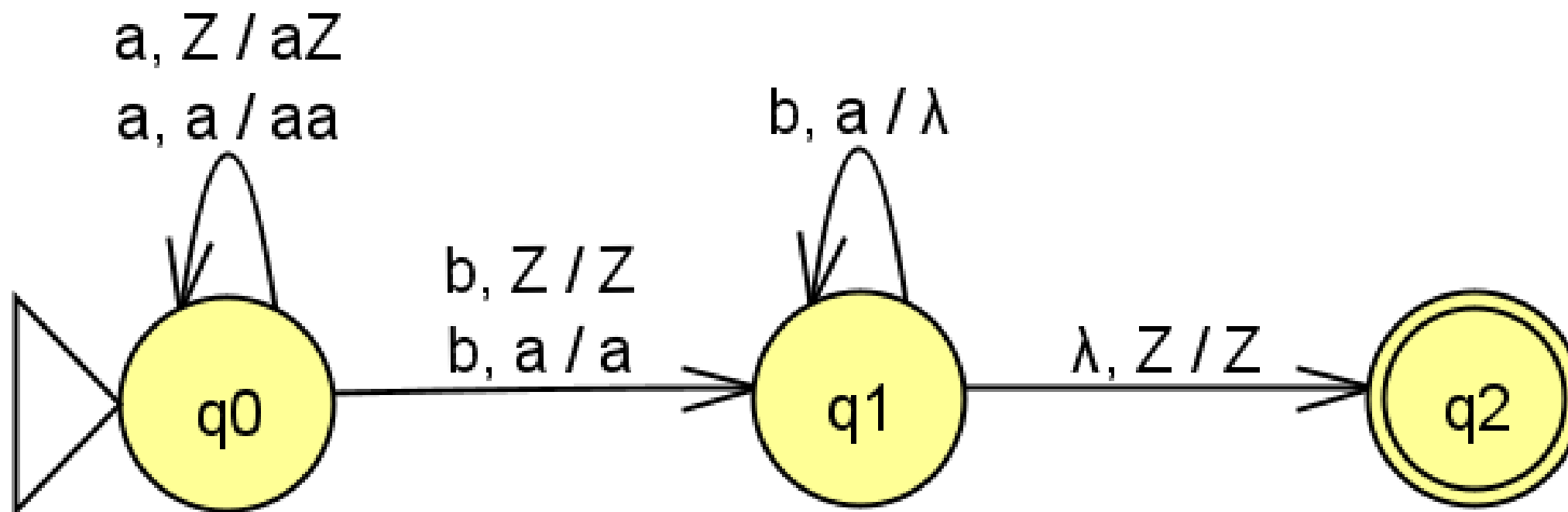


2a. Construct a DPDA for the language $L = \{a^n b^{n+1} \mid n \geq 0\}$

Solution: $L = \{ b, abb, aabbb, aaabbbb, aaaabbbbb, \dots \}$

2a. Construct a DPDA for the language $L = \{a^n b^{n+1} \mid n \geq 0\}$

Solution: $L = \{ b, abb, aabbb, aaabbbb, aaaabbbbb, \dots \}$



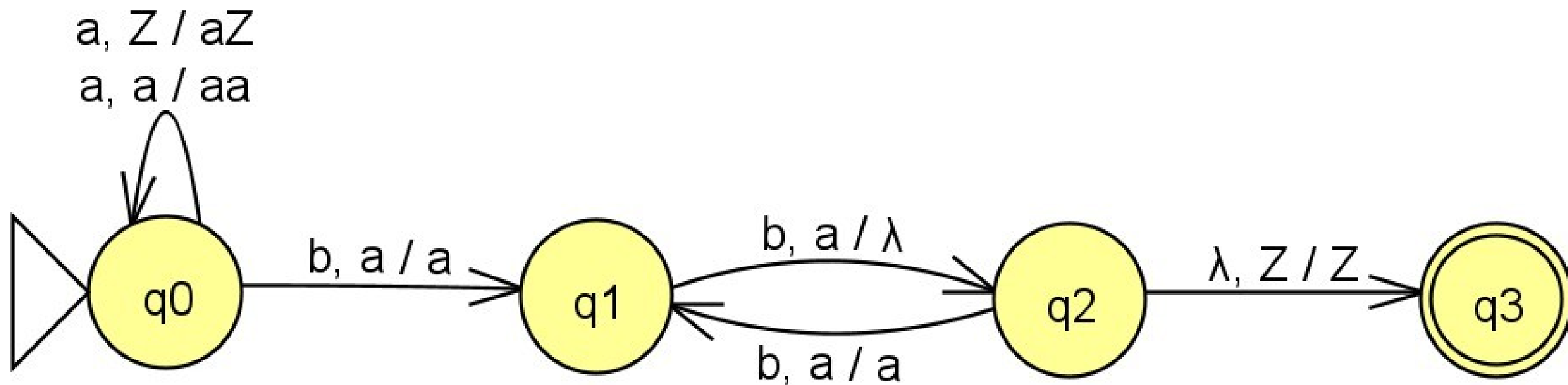
3. Construct a DPDA for the language $L = \{a^n b^{2n} \mid n \geq 1\}$

Solution: $L = \{ abb, aabbbb, aaabbbbbbb, aaaabbbbbbbbbb, \dots \}$

There are two ways to solve $L = \{a^n b^{2n} \mid n \geq 1\}$

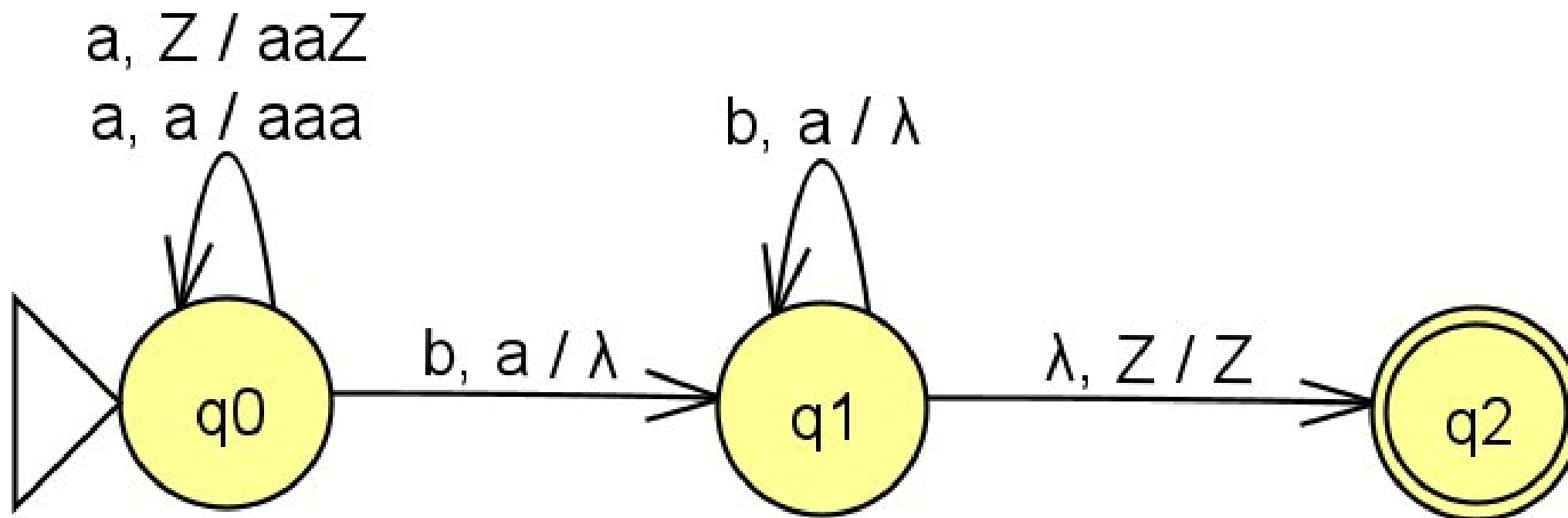
3. Construct a DPDA for the language $L = \{a^n b^{2n} \mid n \geq 1\}$

Solution-1: For every 2 b's pop one a



3. Construct a DPDA for the language $L = \{a^n b^{2n} \mid n \geq 1\}$

Solution-2: Push 2 a's by encountering one a

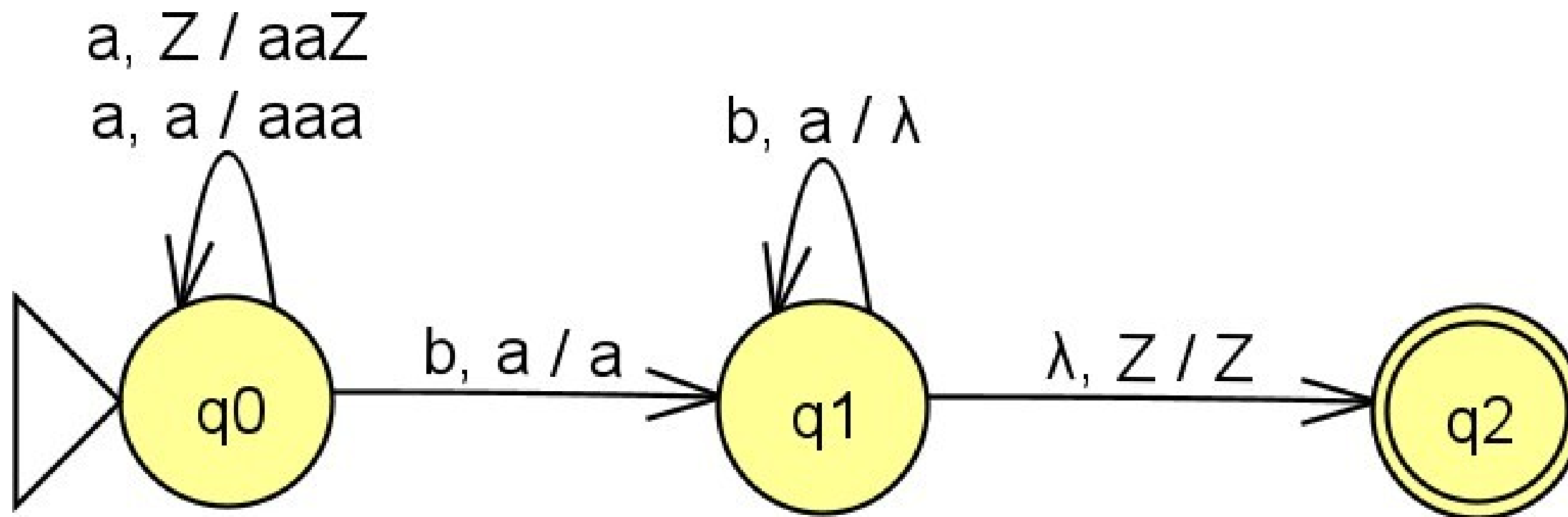


4. Construct a DPDA for the language $L = \{a^n b^{2n+1} \mid n \geq 1\}$

Solution: $L = \{ abbb, aabbbbb, aaabbbbbbb, aaaabbbbbbbbbb, \dots \}$

4. Construct a DPDA for the language $L = \{a^n b^{2n+1} \mid n \geq 1\}$

Solution: $L = \{ abbb, aabbbbbb, aaabbbbbbbb, aaaabbbbbbbbbbb, \dots \}$



5. Construct a DPDA for the language $L = \{a^n b^m c^{m+n} \mid m, n \geq 1\}$

Solution :

5. Construct a DPDA for the language $L = \{a^n b^m c^{m+n} \mid m, n \geq 1\}$

Solution :

b, b / bb

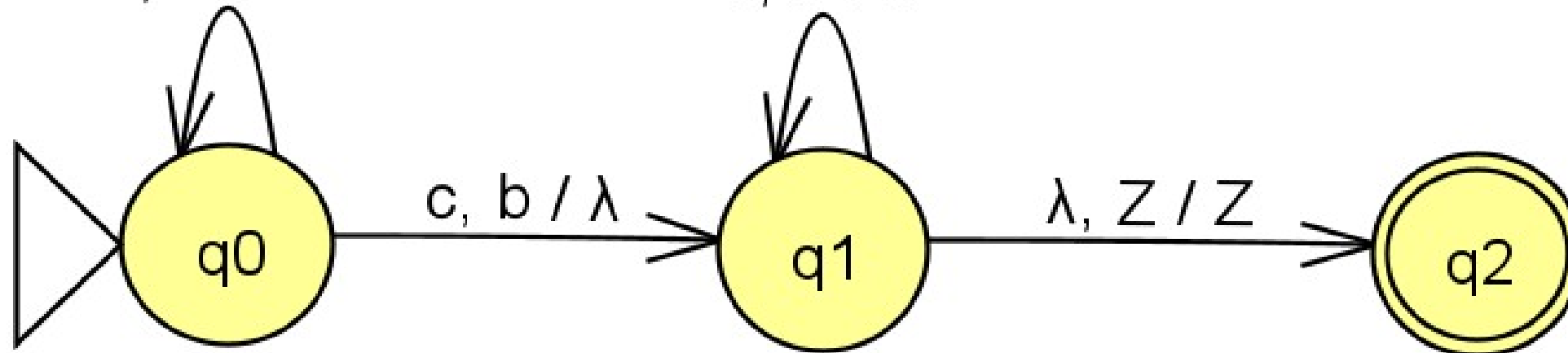
b, a / ba

a, a / aa

a, Z / aZ

c, a / λ

c, b / λ

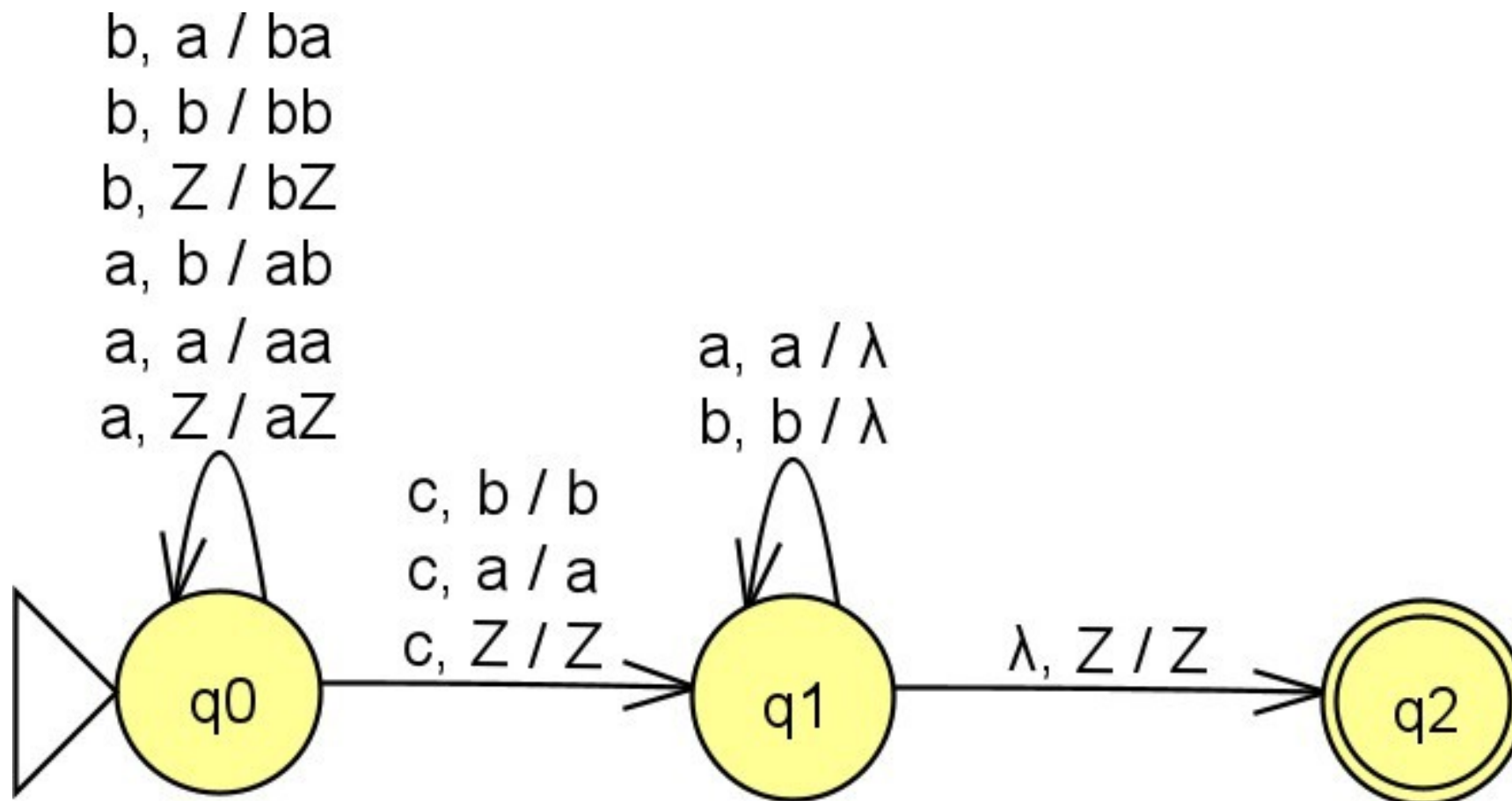


6. Construct a DPDA for the language $L = \{ wcw^R \mid w \in \{a, b\}^* \}$

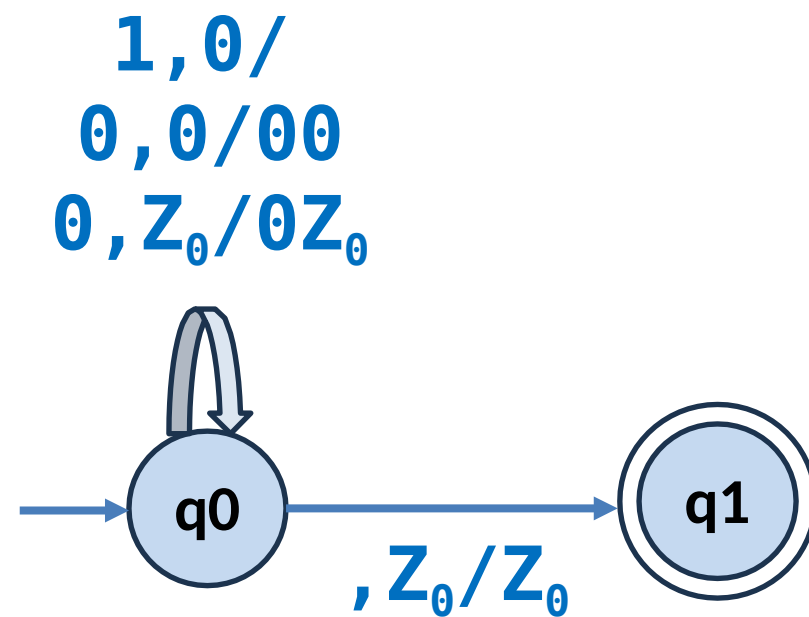
Solution :

6. Construct a DPDA for the language $L = \{ wcw^R \mid w \in \{a, b\}^* \}$

Solution :



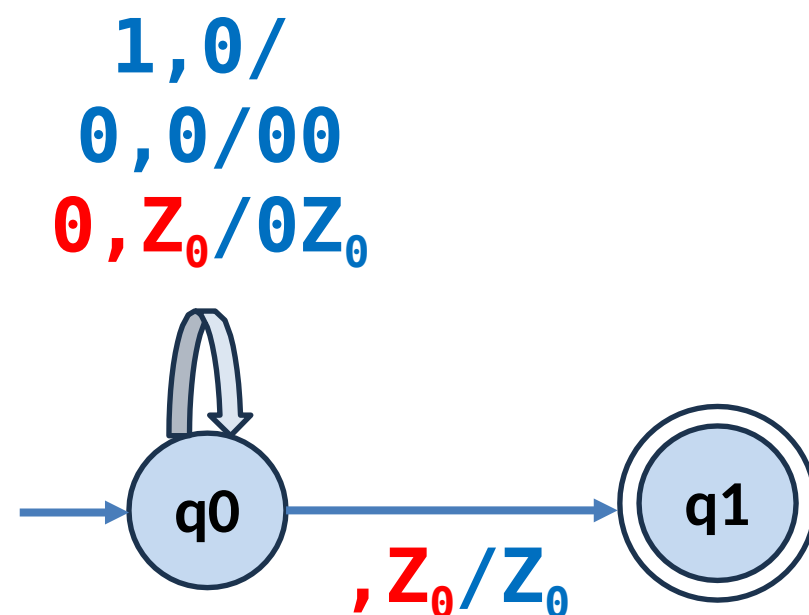
Is this a DPDA?



A PDA is deterministic if and only if:

- $\delta(q, a, X)$ has at most one member for any q in Q , a in $\Sigma \cup \{\epsilon\}$ and X in Γ .
- If $\delta(q, a, X)$ is non-empty for some a in Σ , then $\delta(q, \epsilon, X)$ must be empty.

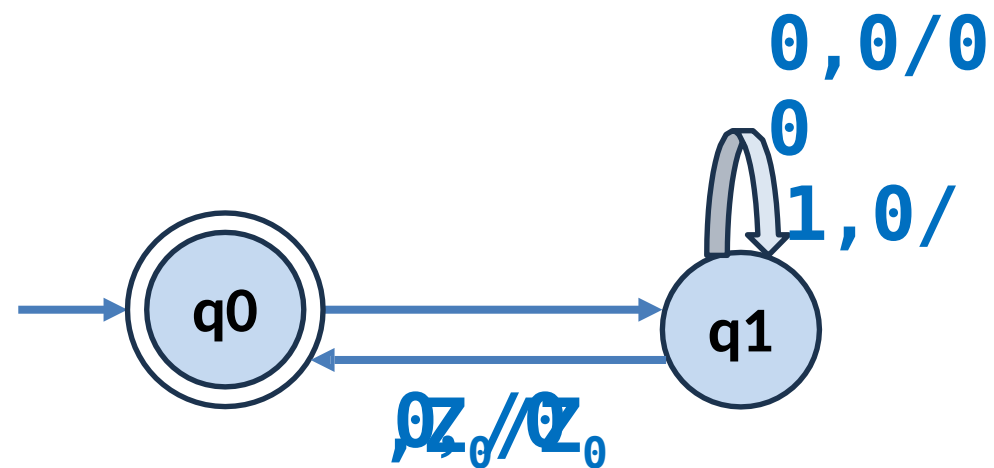
Is this a DPDA? **NO**



A PDA is deterministic if and only if:

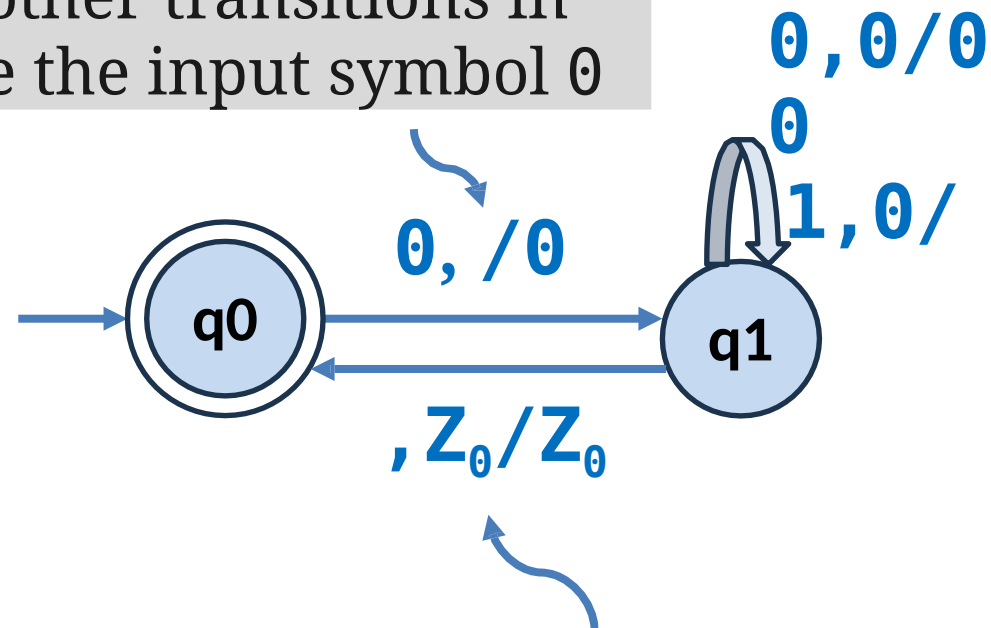
- o $\delta(q, a, X)$ has at most one member for any q in Q , a in $\Sigma \cup \{\epsilon\}$ and X in Γ .
- o If $\delta(q, a, X)$ is non-empty for some a in Σ , then $\delta(q, \epsilon, X)$ must be empty.

Is this a DPDA?



Is this a DPDA? Yes

This ϵ -transition is allowable because no other transitions in this state use the input symbol 0



This ϵ -transition is allowable because no other transitions in this state use the stack symbol Z_0

Exercise:

- 1) Construct DPDA for $a^n b^n c^m$ where $n, m \geq 1$
- 2) Construct DPDA for $a^{n+m} b^m c^n$ where $n, m \geq 1$



THANK YOU

Prakash C O and Preet Kanwal
Department of Computer Science &
Engineering



Automata Formal Languages & Logic

Unit 2 – Pushdown Automata (PDA)

Prakash C O and Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 2: Non-Deterministic Pushdown Automata (NPDA)

Prakash C O and Preet Kanwal

Department of Computer Science & Engineering

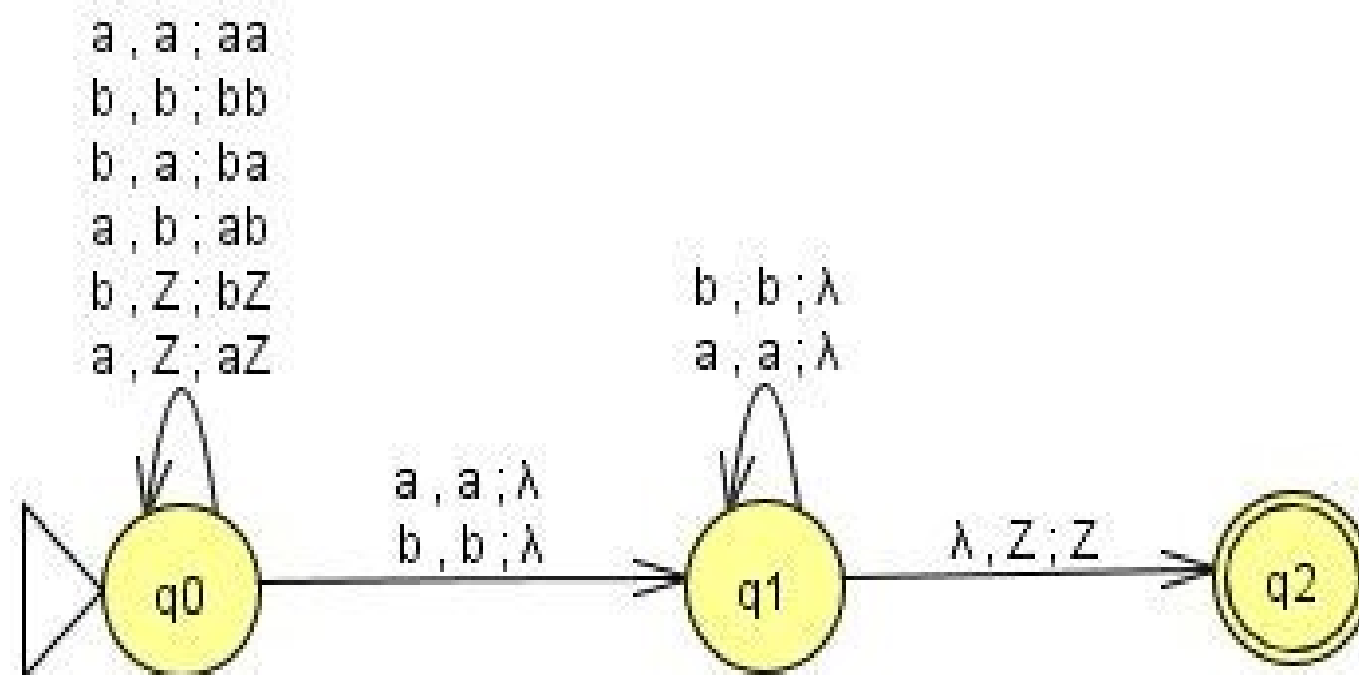
➤ Transition function of Non-Deterministic Pushdown Automaton

$$Q \times (\Sigma \cup \{ \}) \times \Gamma \rightarrow 2^Q \times \Gamma^*$$

In any situation of the PDA, if several actions are possible, then the automaton is called a general, or nondeterministic pushdown automaton (NPDA).

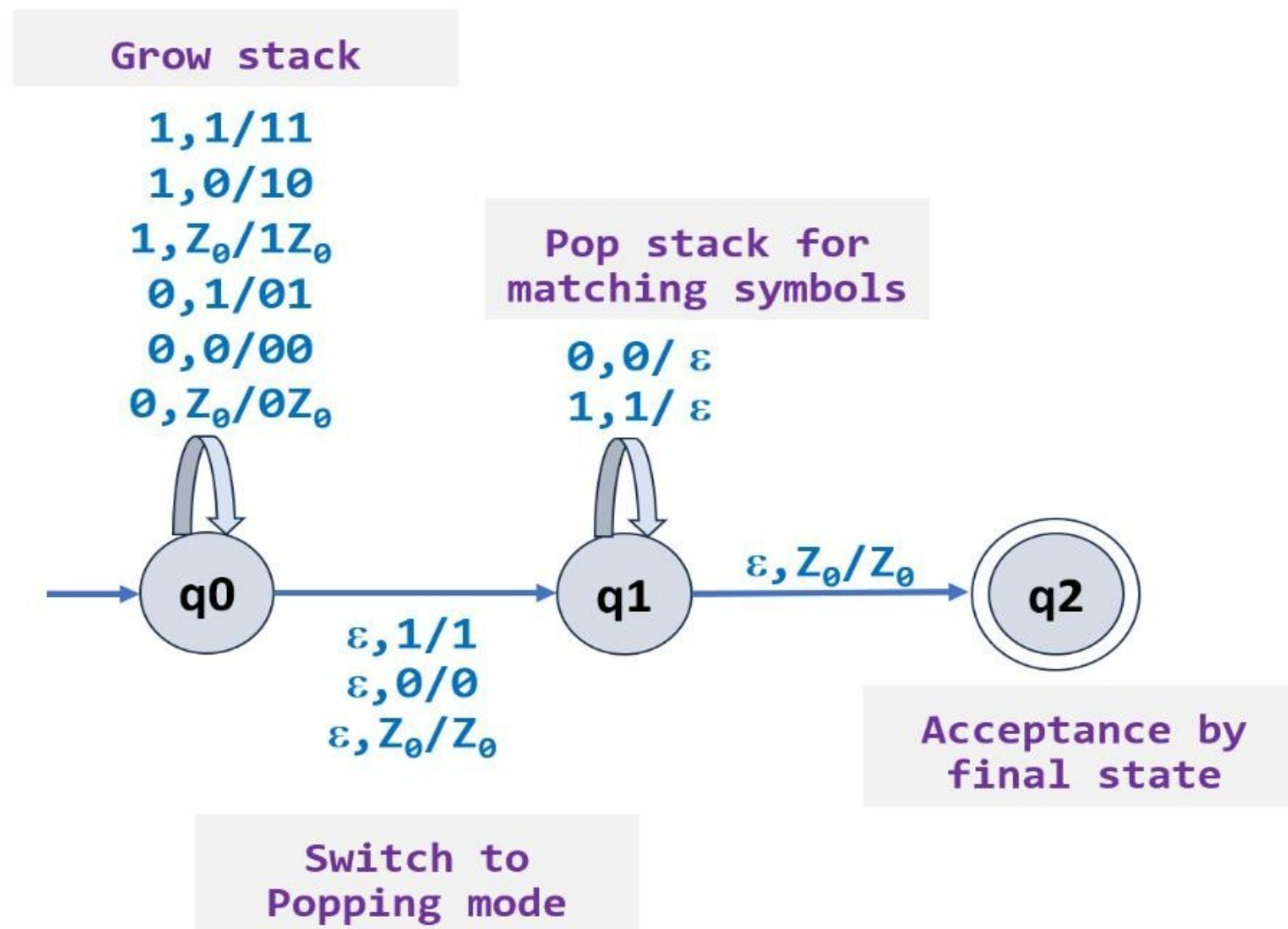
1. Construct PDA for language $L = \{ww^R \mid w \in \{a,b\}^*\}$

Solution :

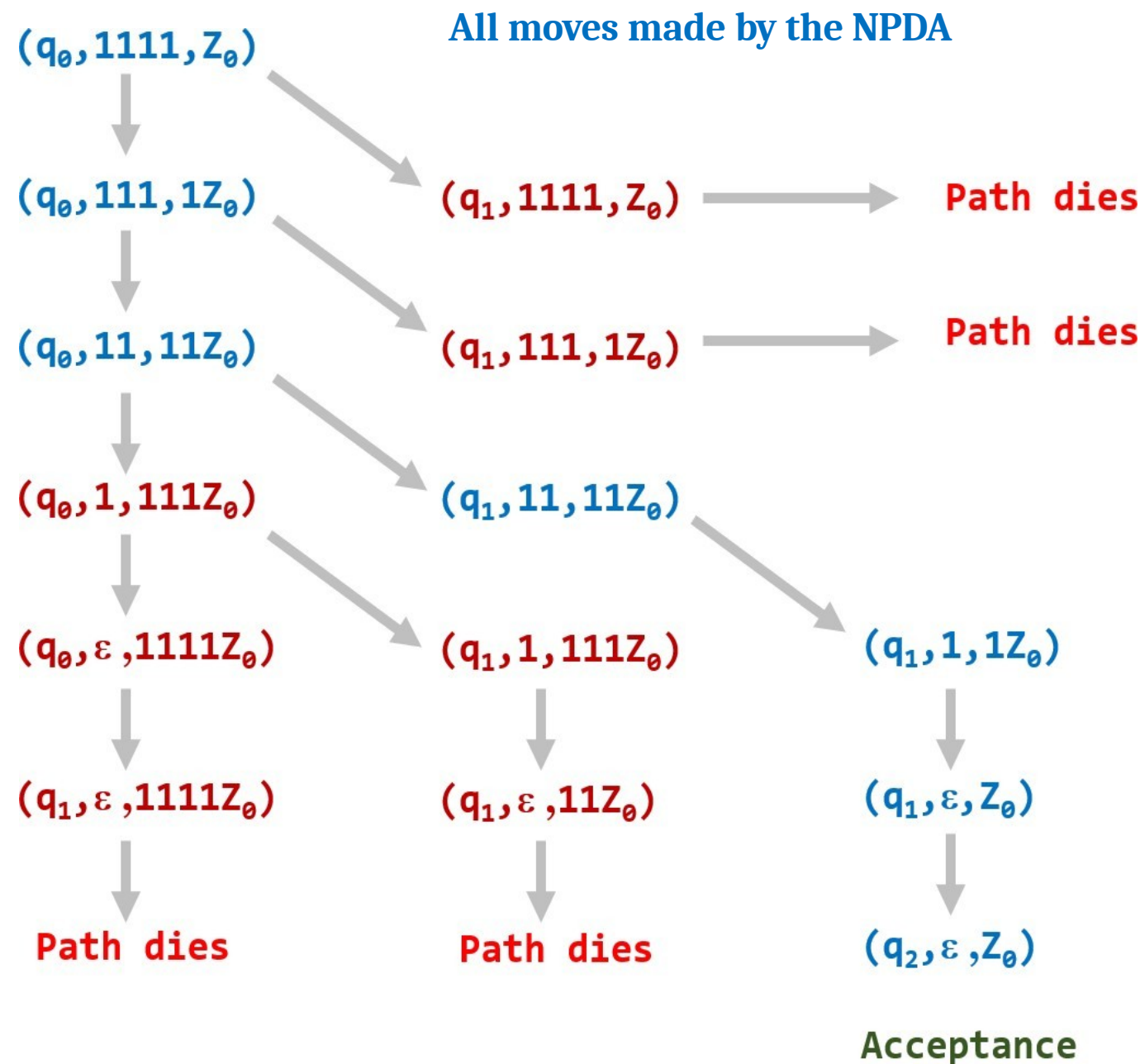


Example:

1) Consider the N-PDA recognizing $L = \{ ww^R \mid w \in \{0,1\}^* \}$



How does the N-PDA for L_{ww^R} work on input “1111”?



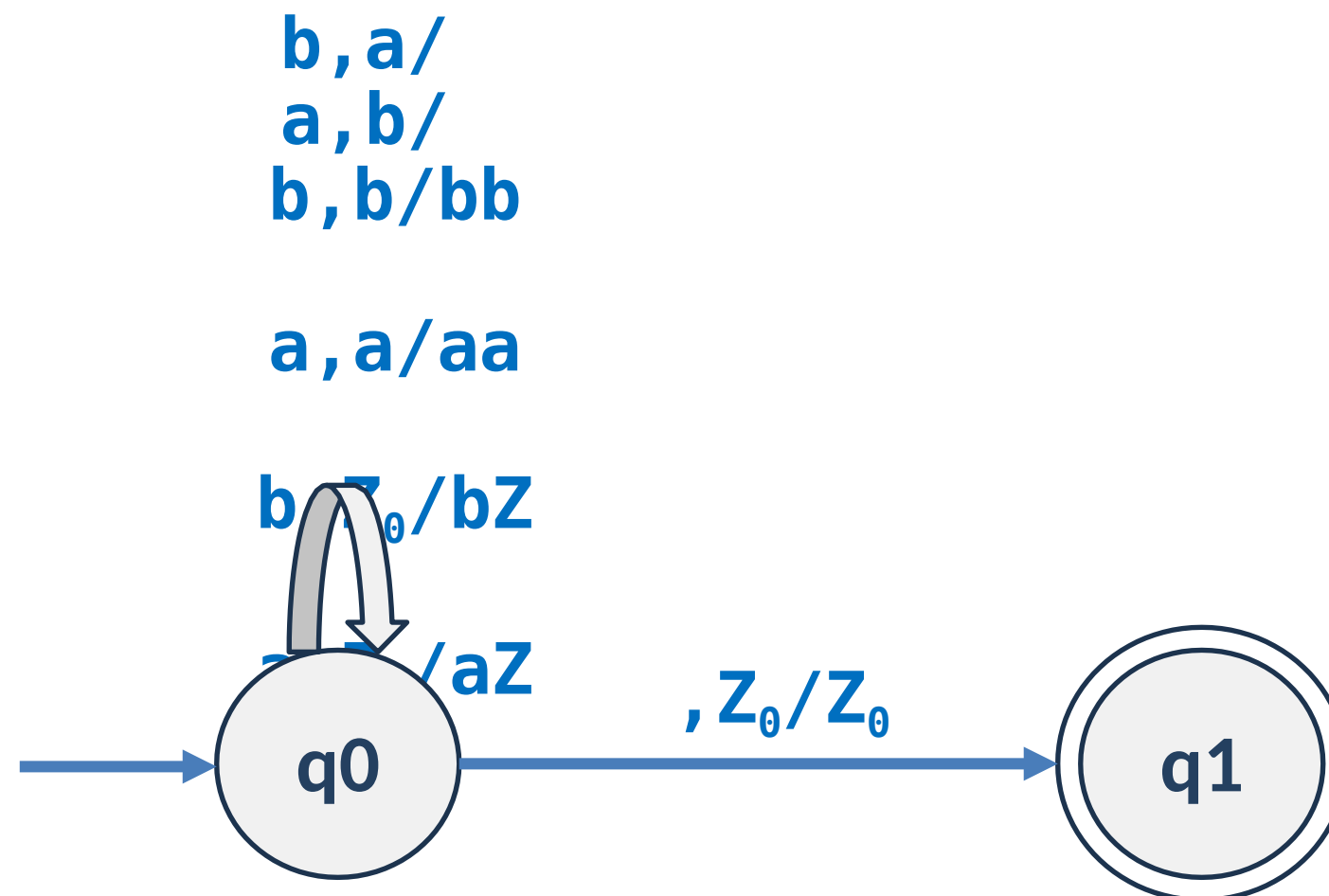
1. Construct a PDA for the language $L = \{ w \mid n_a(w) = n_b(w) \text{ and } w \in \{a, b\}^* \}$

Solution:

Unit 2 – Non-Deterministic Pushdown Automata

1. Construct a PDA for the language $L = \{ w \mid n_a(w) = n_b(w) \text{ and } w \in \{a, b\}^* \}$

Solution:



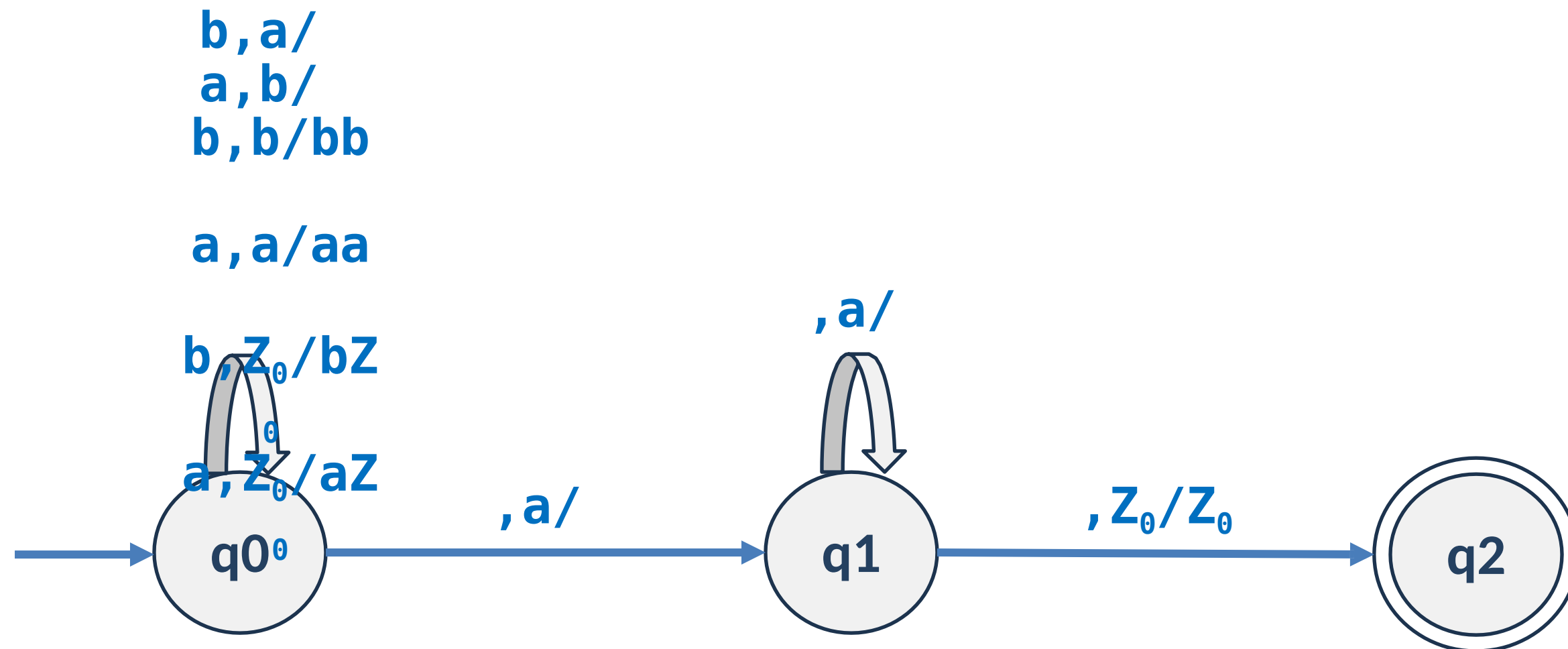
2. Construct a PDA for language $L = \{ w \mid n_a(w) > n_b(w) \text{ and } w \in \{a, b^*\} \}$

Solution :

Unit 2 – Non-Deterministic Pushdown Automata

2. Construct a PDA for language $L = \{ w \mid n_a(w) > n_b(w) \text{ and } w \in \{a, b^*\} \}$

Solution :

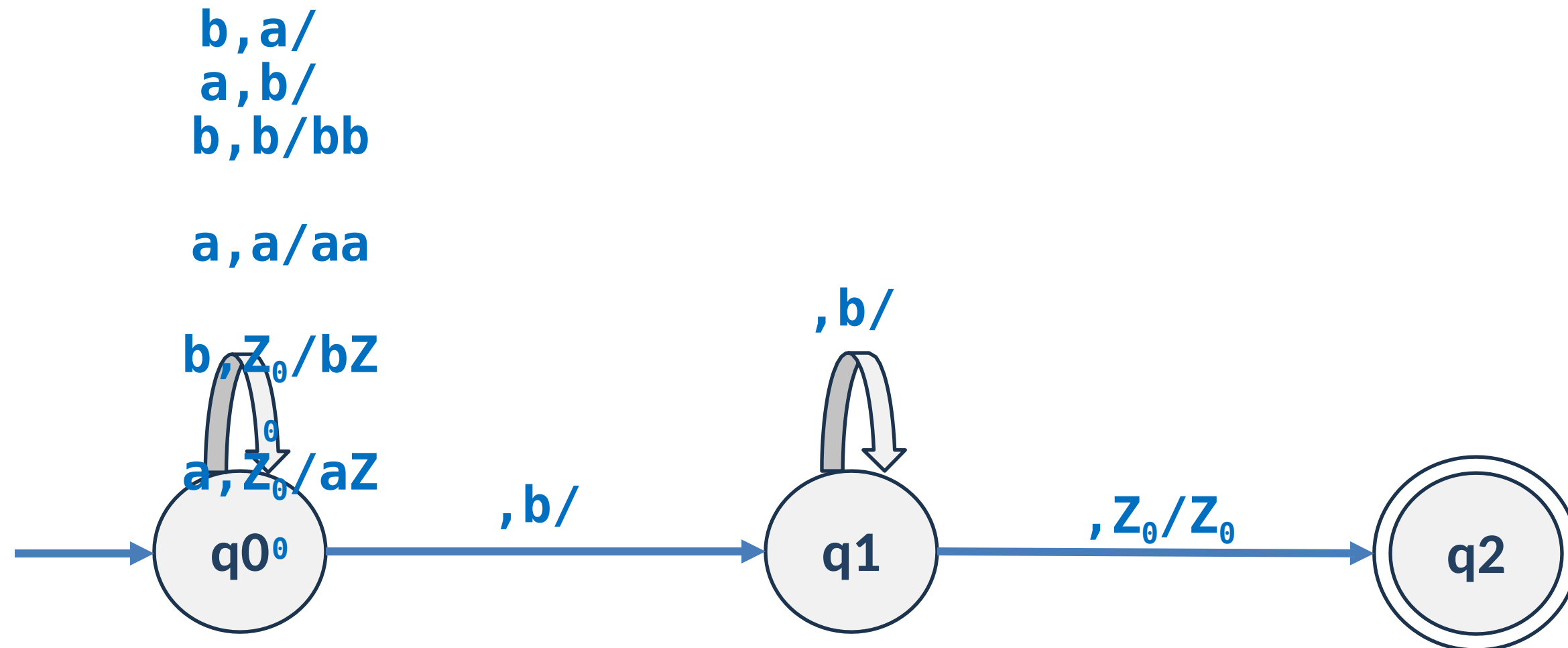


3. Construct a PDA for language $L = \{ w \mid n_a(w) < n_b(w) \text{ and } w \in \{a, b^*\} \}$

Solution :

3. Construct a PDA for language $L = \{ w \mid n_a(w) < n_b(w) \text{ and } w \in \{a, b^*\} \}$

Solution :

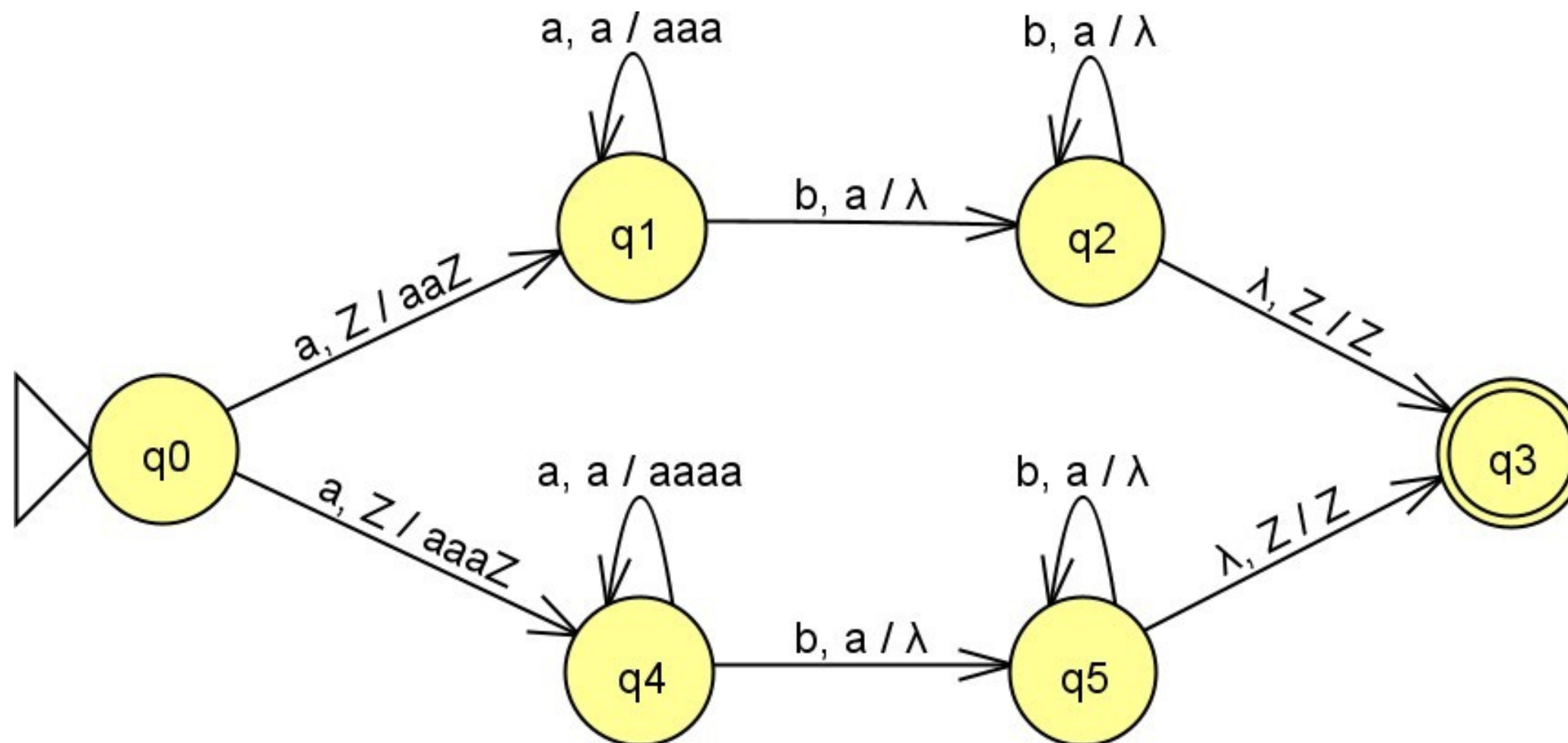


4. Construct PDA for language $L = \{ a^n b^{2n} \cup a^n b^{3n} \mid n \geq 1 \}$

Solution:

4. Construct PDA for language $L = \{ a^n b^{2n} \cup a^n b^{3n} \mid n \geq 1 \}$

Solution:

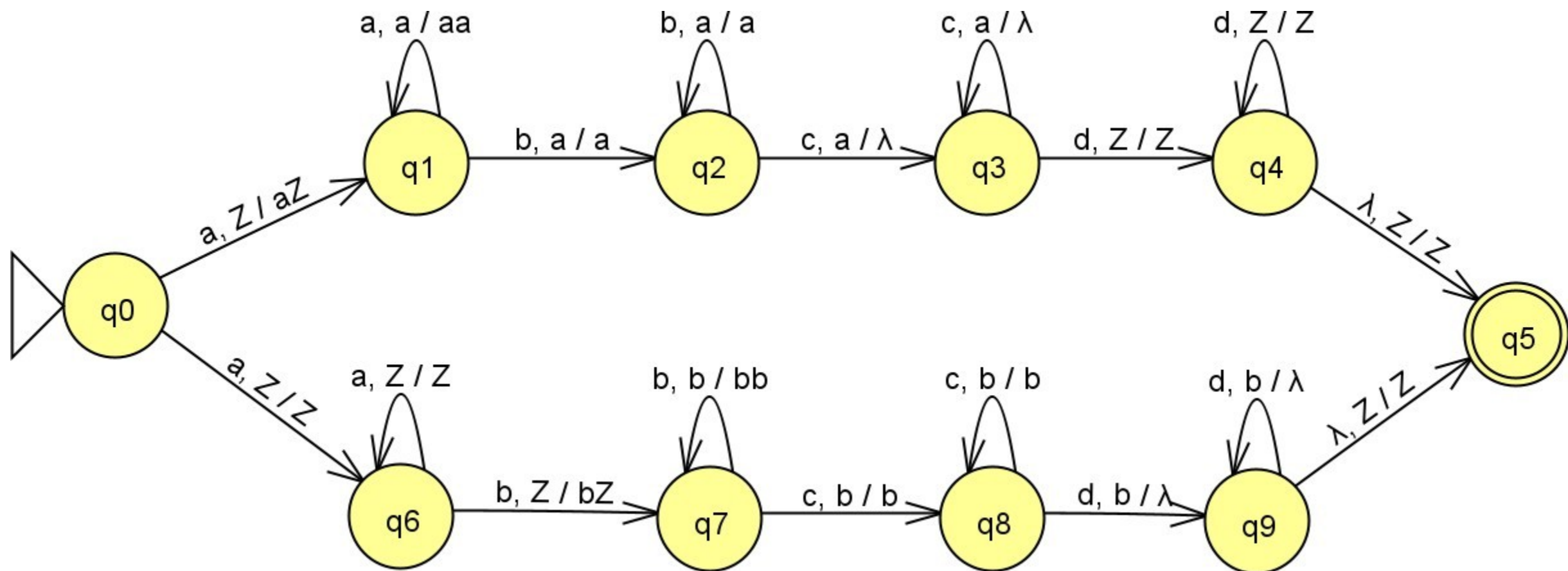


5. Construct PDA for the language $L = \{a^i b^j c^k d^l \mid i=k \text{ or } j=l \text{ and } i, j, k \geq 1\}$

Solution:

5. Construct PDA for the language $L = \{a^i b^j c^k d^l \mid i=k \text{ or } j=l \text{ and } i, j, k \geq 1\}$

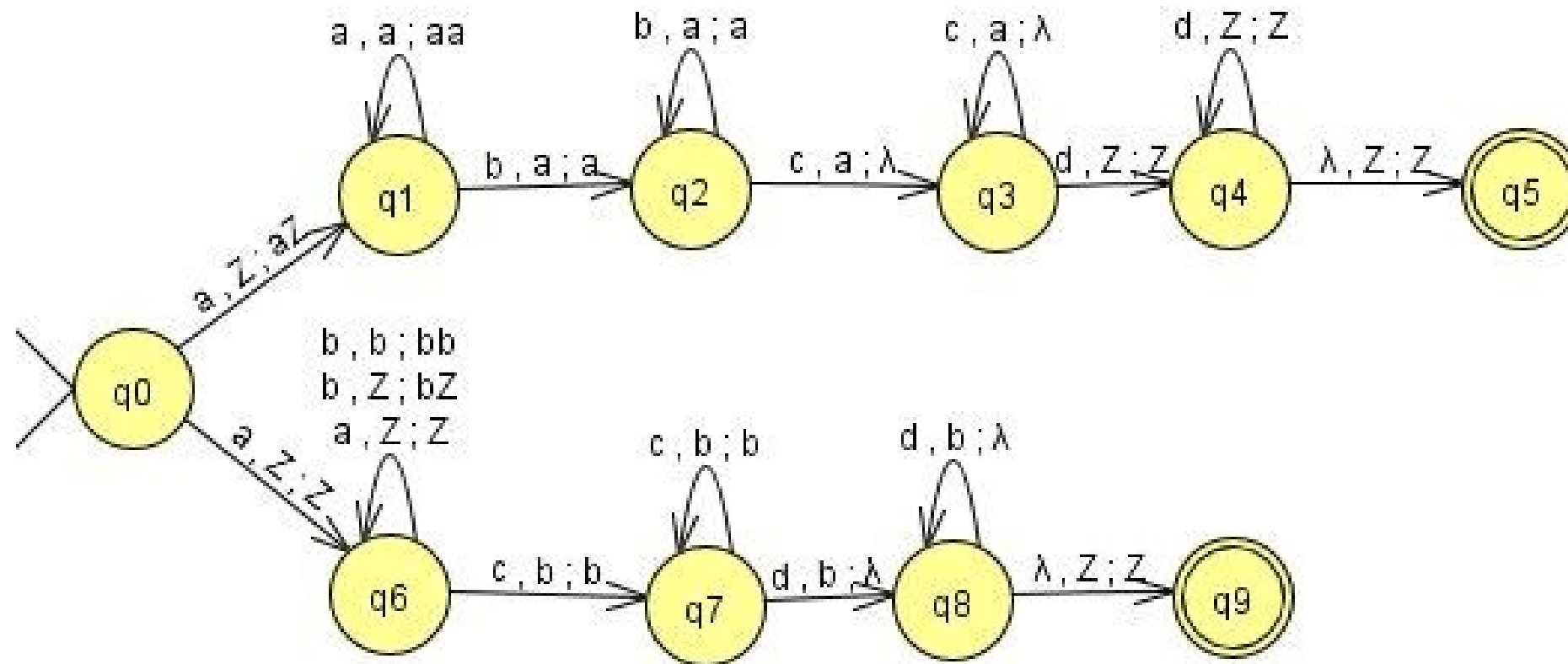
Solution: 1



Unit 2 - Non-Deterministic Pushdown Automata

5. Construct PDA for the language $L = \{a^i b^j c^k d^l \mid i=k \text{ or } j=l \text{ and } i, j, k \geq 1\}$

Solution: 2



Unit 2 - Non-Deterministic Pushdown Automata

Exercise:

- 1) How to convert a final state acceptance PDA into an empty stack acceptance PDA?
- 2) How to convert an empty stack acceptance PDA into a final state acceptance PDA?
- 3) Construct a PDA that accepts $\{ w w^R \mid w \text{ is any string of a's and b's} \}$ by empty stack.



THANK YOU

Prakash C O and Preet Kanwal

Department of Computer Science & Engineering